

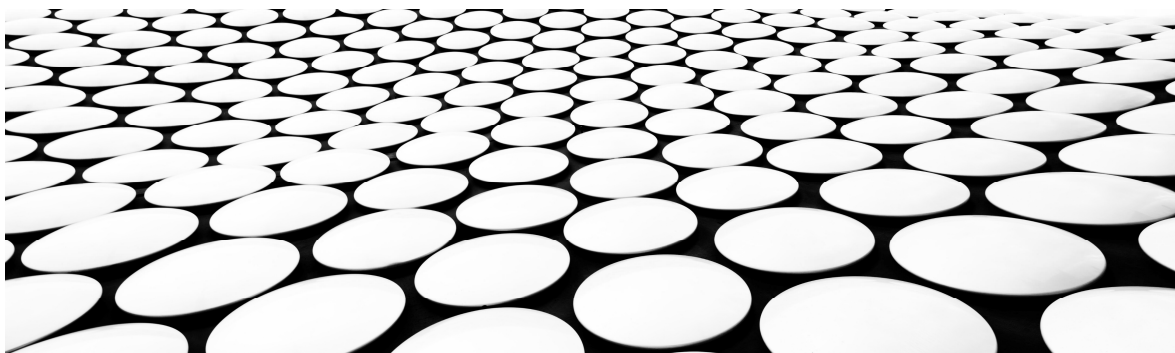


ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH  
**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**  
KHOA ĐIỆN TỬ - VIỄN THÔNG

# THIẾT KẾ VI MẠCH ĐIỆN TỬ

## CHAPTER 6: DIGITAL IC DESIGN

LÊ ĐỨC HÙNG



### CONTENT

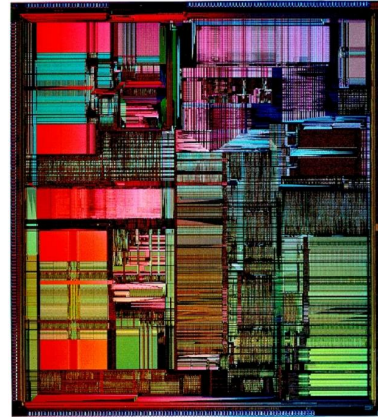
- Digital IC Design Flow – In General
- Digital IC Design Flow – In Detail
- Static Timing Analysis (STA)

---

## INTRODUCTION

### When we need ASIC (Digital ASIC Design)

- **Good**
  - The best implementation
  - Cheap (cost)
- **Bad**
  - Long design time
  - Chip configuration is difficult
  - Incorrect request



3

---

## INTRODUCTION

### Most Important Entities of IC Design

- Area
- Speed
- Power dissipation
- Design time
- Testability

4

## There are trade-offs between

- Cost / Area
- Speed / Throughput
- Energy Consumption / Power Density
- Design Time

## Expectation for an ASIC chip

- Area
- Power
- Speed

5

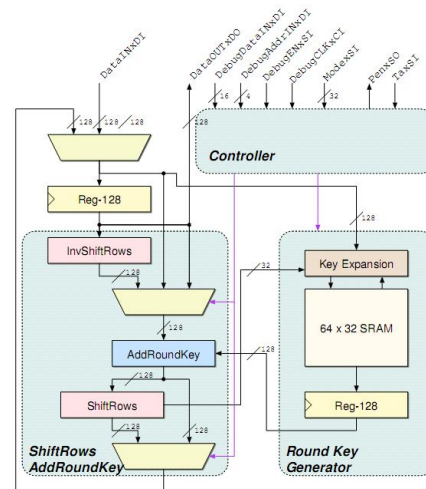
## INTRODUCTION

## Design Specifications

- Function
- Performance (speed, power, area)
- Input/Output

### Block Diagram Orientation

- Defining functions for each block
- Which blocks have high requirements (speed, power, area)
- Block interconnections
- Critical paths
- Sub-block diagram

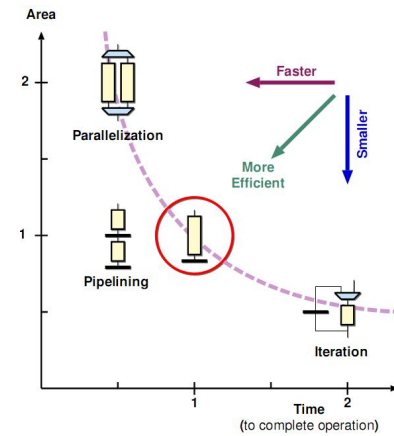


6

## INTRODUCTION

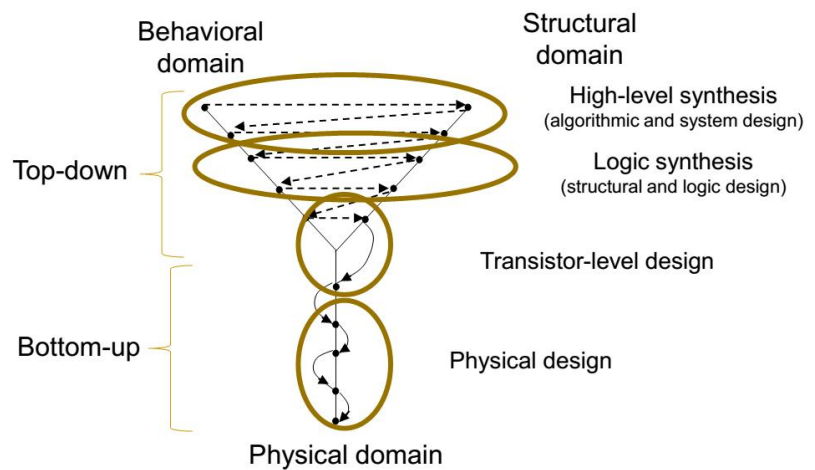
### Characteristics and influences of parameters:

- Area ( $\text{mm}^2$ )
- Clock rate (MHz)
- Throughput (data/sec)
- Latency (number of clock cycles)

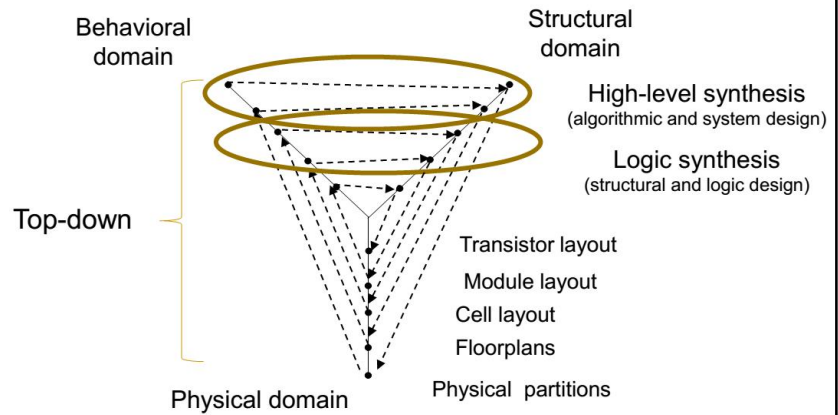


7

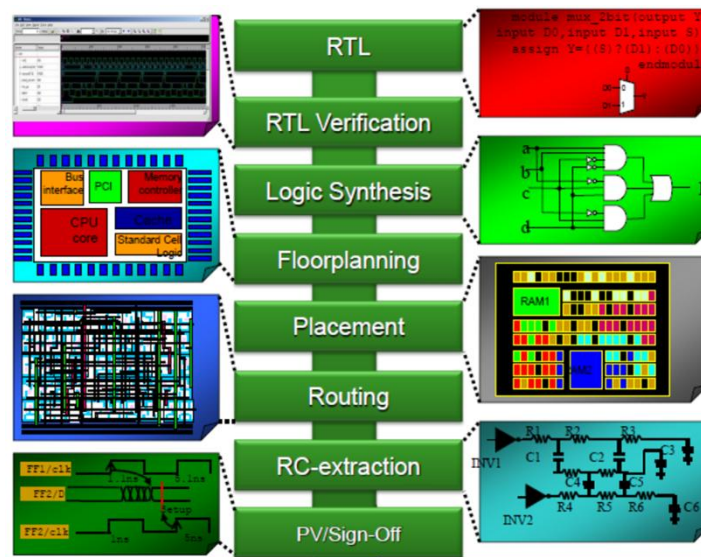
## TOP-DOWN AND BOTTOM-UP DESIGN METHODOLOGY



## ONLY TOP-DOWN DESIGN METHODOLOGY

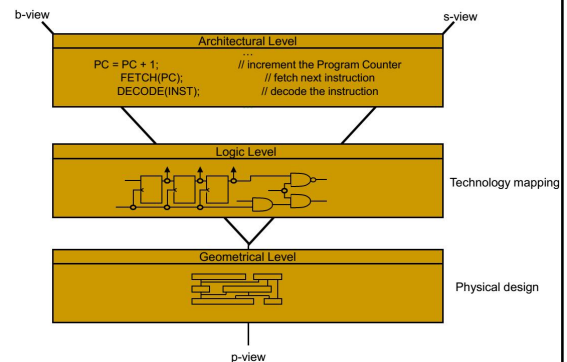
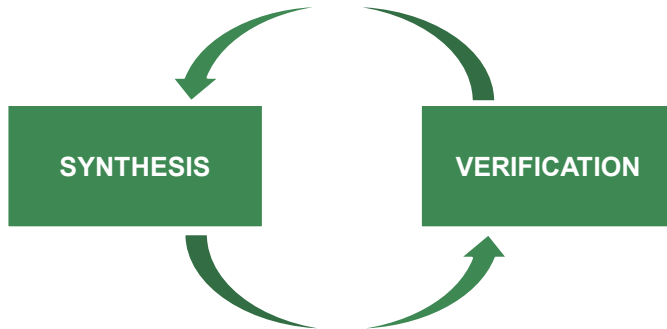


## DESIGN FLOW



## DESIGN PROCESS

- Design process cycles between synthesis and verification



### Synthesis:

- Make assumption, Circuit topology, device size,

### Verification:

- Simulation, circuit parameter measurement, verify assumptions

11

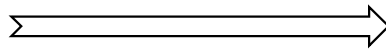
## DESIGN PROCESS

- Design is a continuous **trade-off among design parameters**

- Design parameters:

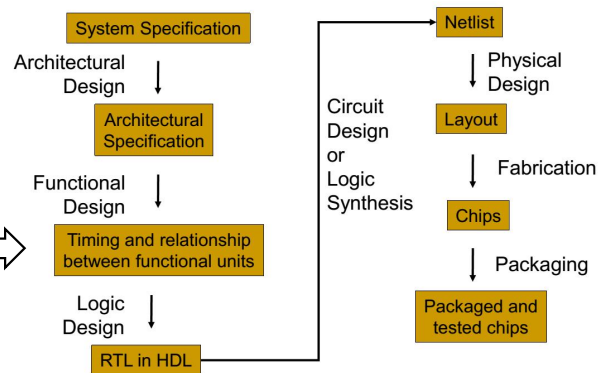
- Performance (speed, power, function, flexibility)
- Die size (cost of die)
- Design time
- Testability and ease of testing

- IC Design Cycle**



- Use EDA Tools for designing Digital IC Design

(EDA = Electronics Design Automation)



12

## HISTORY AND EVOLUTION OF THE EDA

### ■ 1960: HAND DESIGN



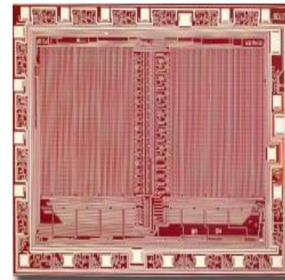
Courtesy TI  
122 Transistors

Paper designs



Courtesy RCA— CMOS Test  
Pattern

Cutting Rubylith artwork for mask generation



Courtesy Intel and NY  
Museum of Modern Art  
EPROM

13

## HISTORY AND EVOLUTION OF THE EDA

- **1970s. Automated pattern generation and circuit simulation**
- Digitizing systems (Calma)
- Spice for circuit simulation
- Automatic routing for printed circuit boards (Applicon, CV, RacalRedac)
- Captive IC routers (IBM, WE, ...)
- GDS II becomes the standard layout format (Calma)
- Schematic capture and logic simulation on workstations (Daisy, Mentor, Valid)

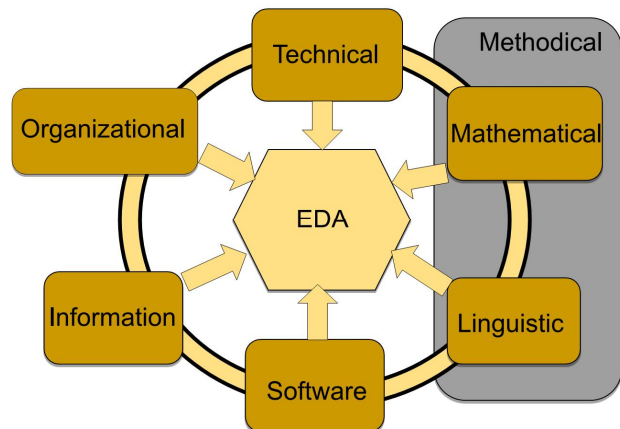
14

## ELECTRONICS DESIGN METHODOLOGY

### Components of Electronics Design

#### Core of EDA:

- Theory of processes occurring in the design
- Methods of analysis and synthesis of systems and their components
- Methods of analysis and synthesis of schemes and structures
- Mathematical models
- Mathematical methods and algorithms for solving equation systems

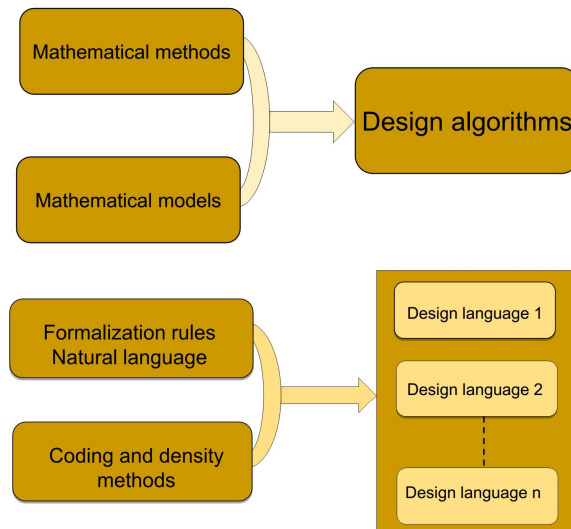


15

## ELECTRONICS DESIGN METHODOLOGY

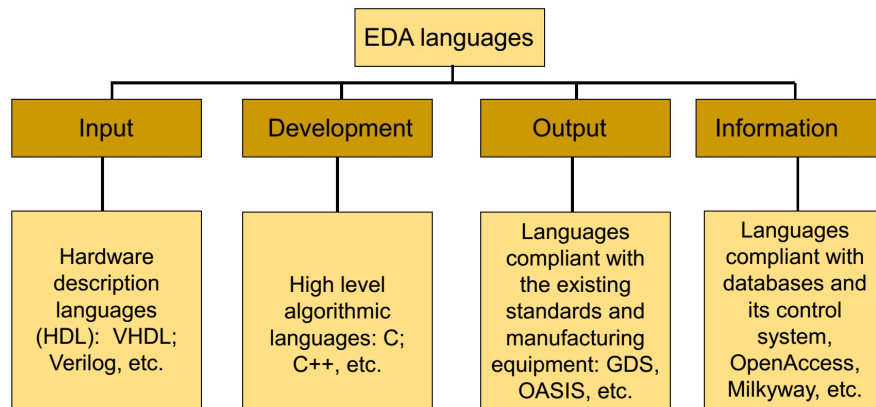
### Formal Design Language

- A group of mathematical methods, models and algorithms of design, necessary for design automation.
- A group of formal design languages (including transition from one language into another, coding and density methods), necessary for design automation.



## ELECTRONICS DESIGN METHODOLOGY

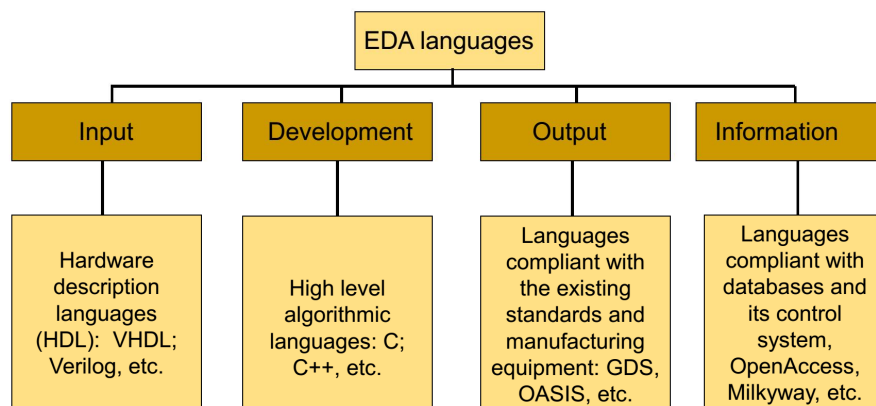
### ■ Formal Design Language



17

## ELECTRONICS DESIGN METHODOLOGY

### ■ Formal Design Language



18

## ELECTRONICS DESIGN METHODOLOGY

### Verilog Examples

```
module fulladder(input a, b, c,
                output s, cout);

    sum      s1(a, b, c, s);
    carry    cl(a, b, c, cout);
endmodule
```

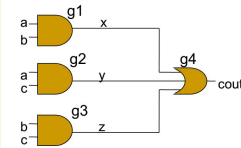
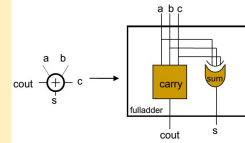
```
module carry(input a, b, c,
             output cout);

    assign cout = (a&b) | (a&c) |
                  (b&c);
endmodule
```

```
module carry(input a,b,c,
             output cout)

    wire x, y, z;

    and g1(x, a, b);
    and g2(y, a, c);
    and g3(z, b, c);
    or g4(cout, x, y, z);
endmodule
```



19

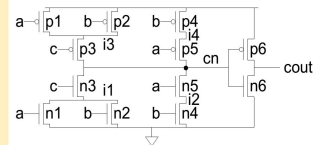
## ELECTRONICS DESIGN METHODOLOGY

### Transistor Level Netlist

```
module carry(input a, b, c,
             output cout);

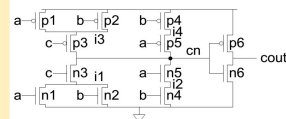
    wire i1, i2, i3, i4, cn;

    tranifl n1(i1, 0, a);
    tranifl n2(i1, 0, b);
    tranifl n3(cn, i1, c);
    tranifl n4(i2, 0, b);
    tranifl n5(cn, i2, a);
    tranifl p1(i3, 1, a);
    tranifl p2(i3, 1, b);
    tranifl p3(cn, i3, c);
    tranifl p4(i4, 1, b);
    tranifl p5(cn, i4, a);
    tranifl n6(cout, 0, cn);
    tranifl p6(cout, 1, cn);
endmodule
```



### SPICE Netlist

```
.SUBCKT CARRY A B C COUT VDD GND
MN1 I1 A GND GND NMOS W=1U L=0.18U AD=0.3P AS=0.5P
MN2 I1 B GND GND NMOS W=1U L=0.18U AD=0.3P AS=0.5P
MN3 CN C I1 GND NMOS W=1U L=0.18U AD=0.5P AS=0.5P
MN4 I2 B GND GND NMOS W=1U L=0.18U AD=0.15P
AS=0.5P
MN5 CN A I2 GND NMOS W=1U L=0.18U AD=0.5P AS=0.15P
MP1 I3 A VDD VDD PMOS W=2U L=0.18U AD=0.6P AS=1P
MP2 I3 B VDD VDD PMOS W=2U L=0.18U AD=0.6P AS=1P
MP3 CN C I3 VDD PMOS W=2U L=0.18U AD=1P AS=1P
MP4 I4 B VDD VDD PMOS W=2U L=0.18U AD=0.3P AS=1P
MP5 CN A I4 VDD PMOS W=2U L=0.18U AD=1P AS=0.3P
MN6 COUT CN GND GND NMOS W=2U L=0.18U AD=1P AS=1P
MP6 COUT CN VDD VDD PMOS W=4U L=0.18U AD=2P AS=2P
CII I1 GND 2FF
CII I3 GND 3FF
CA A GND 4FF
CB B GND 4FF
CC C GND 2FF
CCN CN GND 4FF
CCOUT COUT GND 2FF
.ENDS
```



20

## ELECTRONICS DESIGN METHODOLOGY

### File Format

| File | Expansion                          | Description                                                                                         |
|------|------------------------------------|-----------------------------------------------------------------------------------------------------|
| SDF  | Standard Delay Format              | Industry standard format giving pin to pin delays                                                   |
| PDEF | Physical Design Exchange Format    | Industry standard format for physical cluster and placement information (initiated by Synopsys)     |
| DEF  | Design Exchange Format             | Format for physical data exchange (mostly used for storing physical constraints, floor plan , etc.) |
| SPEF | Standard Parasitic Exchange Format | Industry standard format                                                                            |
| LEF  | Library Exchange Format            | Standard format used for exchanging layout data between programs                                    |
| PLEF | Parametric LEF                     |                                                                                                     |

```

DELAYFILE
(SDFVERSION "01/2.1")
(DESIGN "COUNTER")
(
  DATE "Wed Jul 27 08:18:43 2000"
  (VENDOR "class")
  (PROGRAM "Acme EDA Co.")
  (VERSION "2000.11")
  (DIVIDER 1)
  (VOLTAGE 5.00:5.00:5.00)
  (PROCESS "typical")
  (TEMPERATURE 25.00:25.00:25.00)
  (TIMESCALE 1ns)
)
(CELL
  (CELLTYPE "I/A")
  (INSTANCE U41)
  (DELAY
    (ABSOLUTE
      (IOPATH A Z (0.350:0.350:0.350) (0.340:0.340:0.340))
    ))
  )
)
  
```

Header Information

Min:Typical:Max

Rising delays

Falling delays

Delay from input pin to output pin

```

(CELL
  (CELLTYPE "FD1")
  (INSTANCE REG_BLK1/FO)
  (DELAY
    (ABSOLUTE
      (IOPATH CP Q (3.199:3.199:3.199) (2.126:2.126:2.126))
    ))
  (TIMINGCHECK
    (SETUP D CP (0.800:0.800:0.800))
    (HOLD D CP (0.400:0.400:0.400))
  )
)
(CELL
  (CELLTYPE "counter")
  (INSTANCE)
  (DELAY(ABSOLUTE
    (INTERCONNECT m1_reg_18/QN U31/I (.006:.006:.006)(.005:.005:.005))
    (INTERCONNECT U8/ZN U95/A2 (.235:.235:.235) (.250:.250:.250))
    (INTERCONNECT U8/ZN U97/A2 (.229:.229:.229) (.244:.244:.244))
  ))
)
  
```

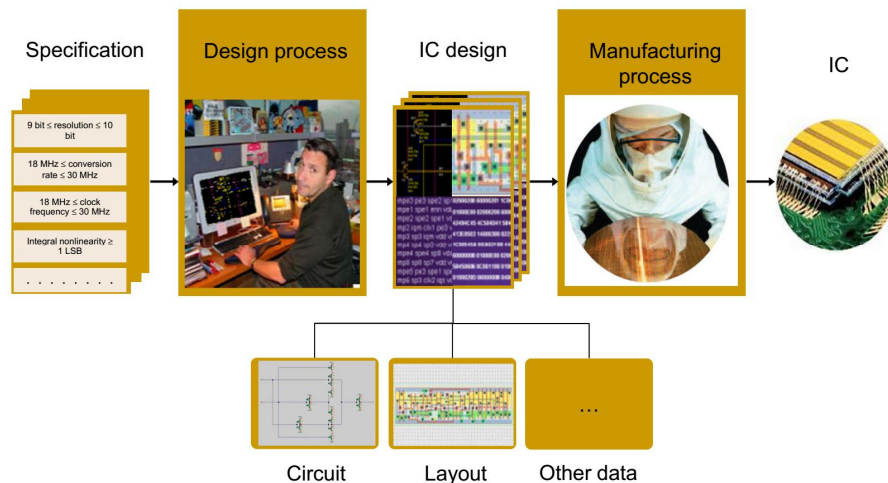
Setup/Hold constraints

Interconnect delays

21

## DIGITAL DESIGN FLOW

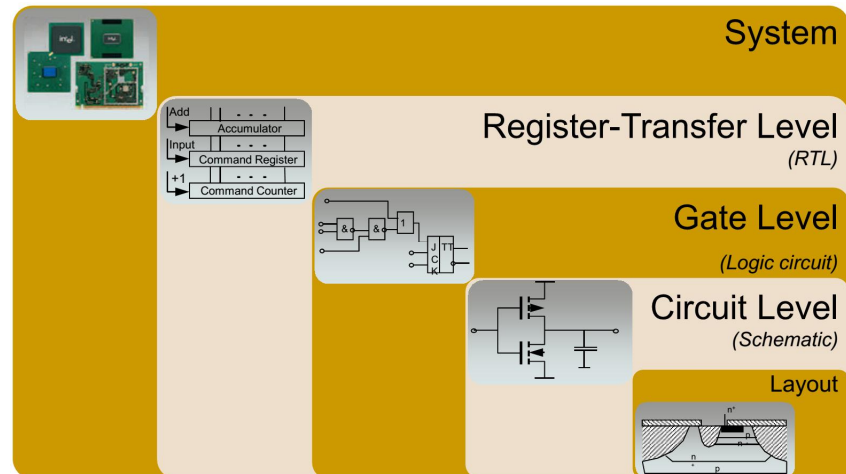
### Phases of IC Design



22

## DIGITAL DESIGN FLOW

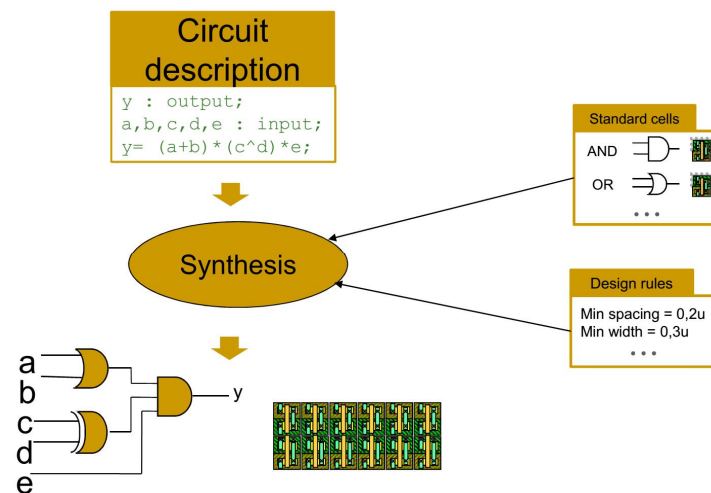
### Design Levels



23

## DIGITAL DESIGN FLOW

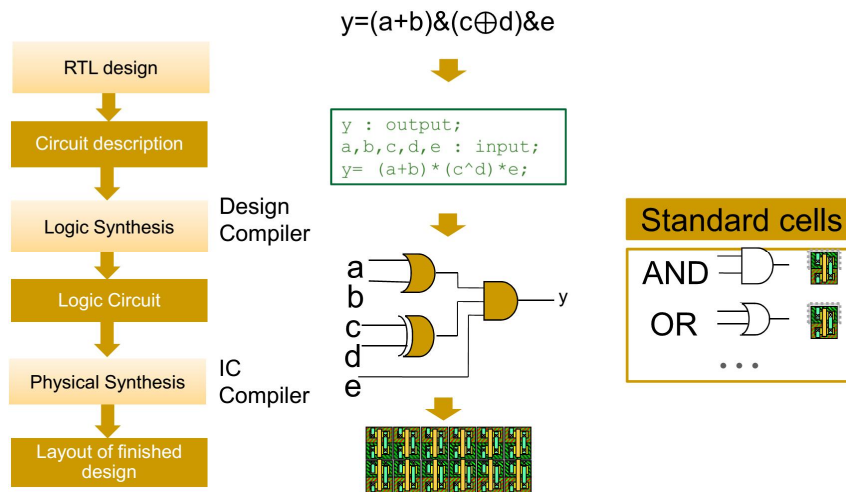
### Digital IC Synthesis



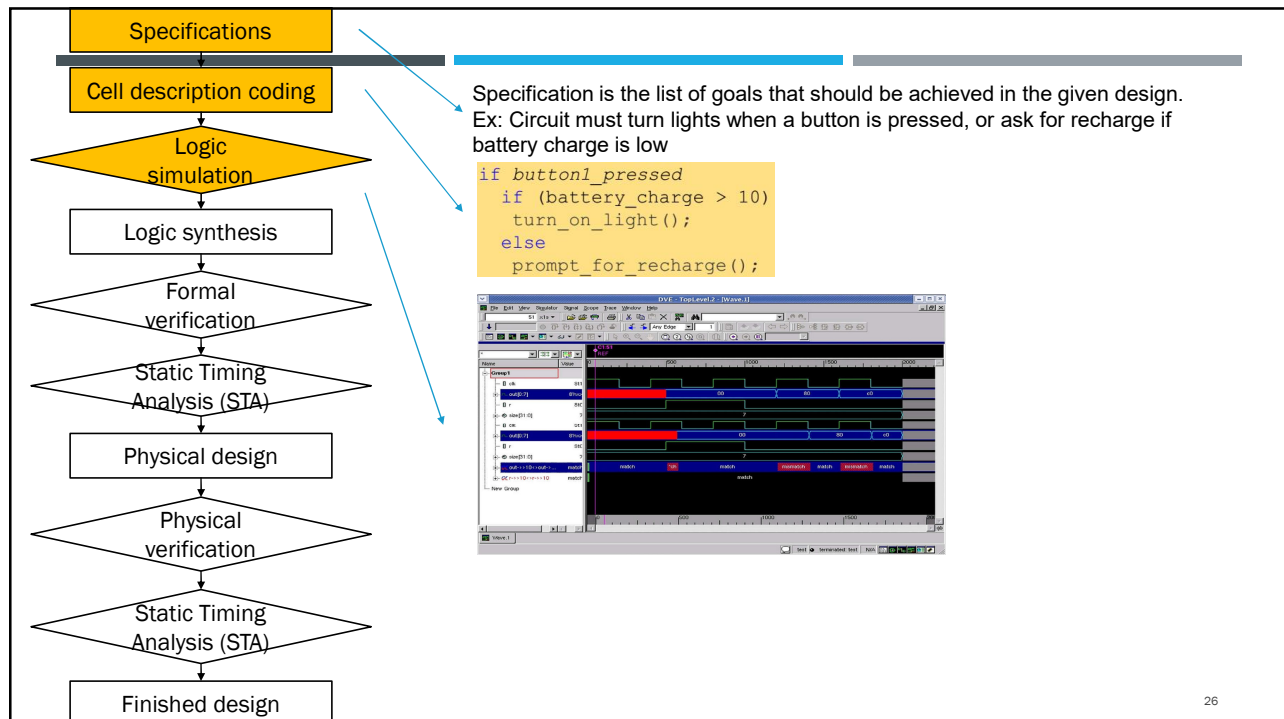
24

## DIGITAL DESIGN FLOW

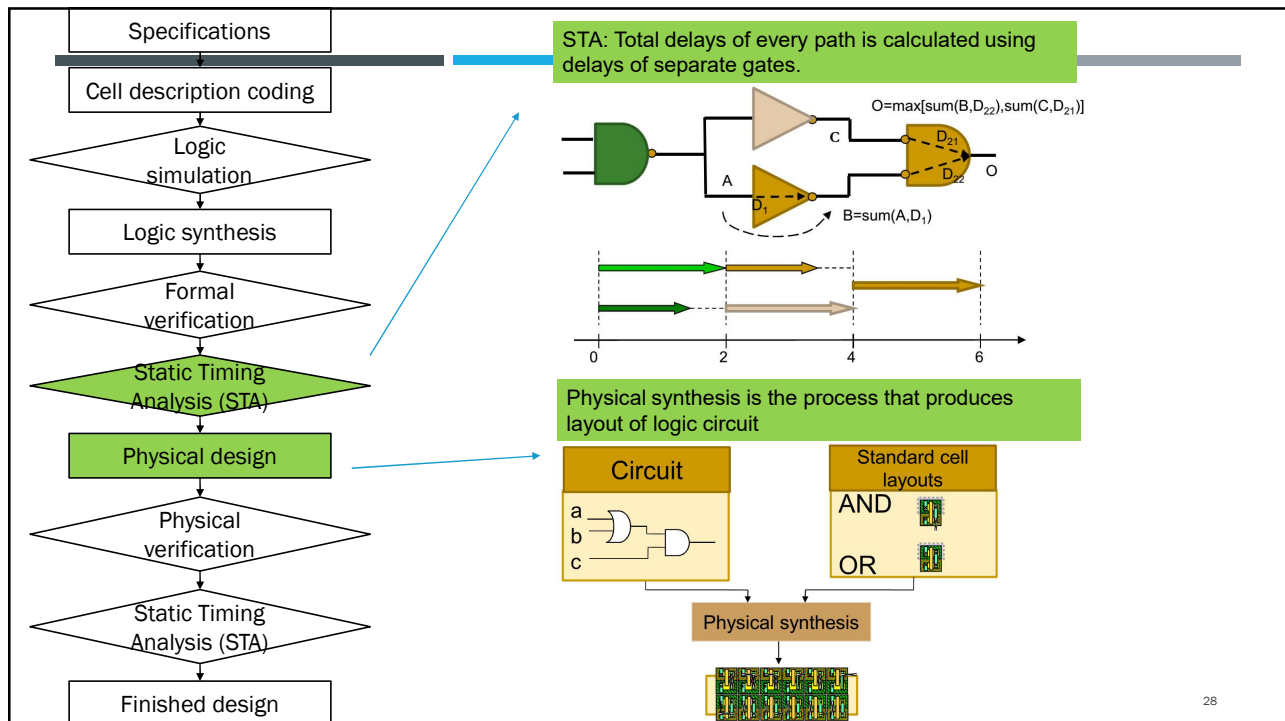
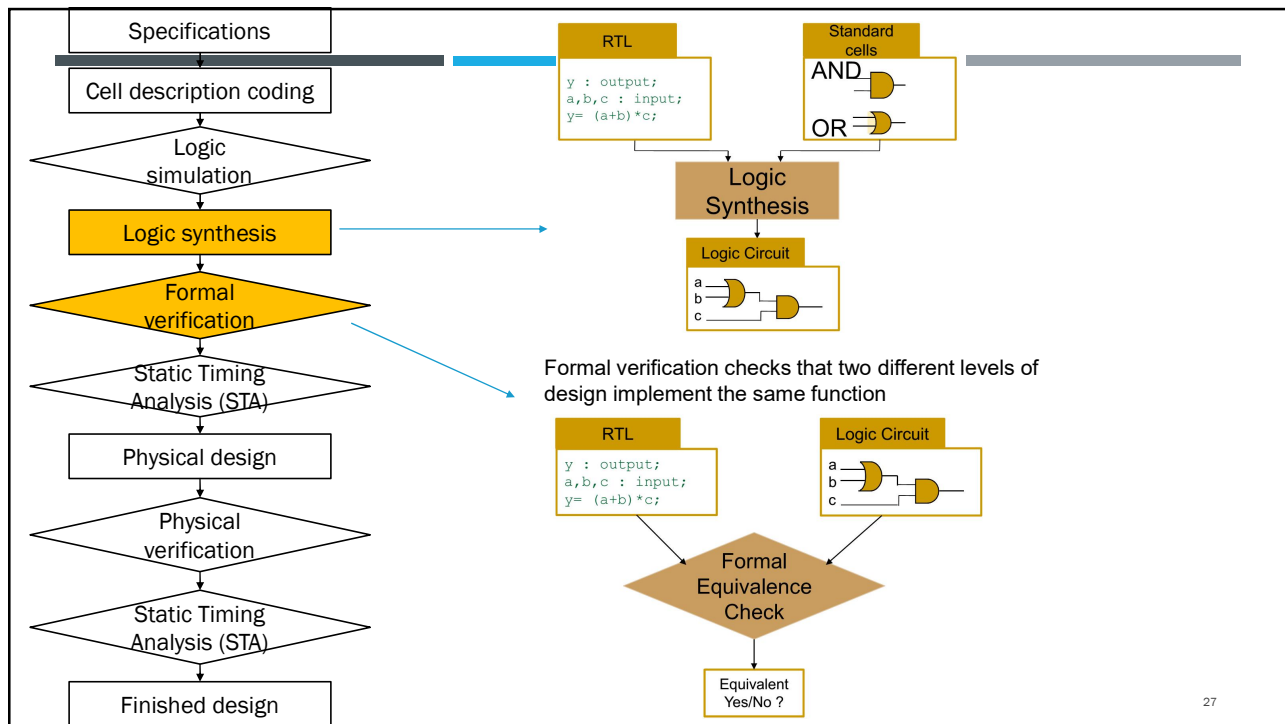
### Basics of Digital IC Design

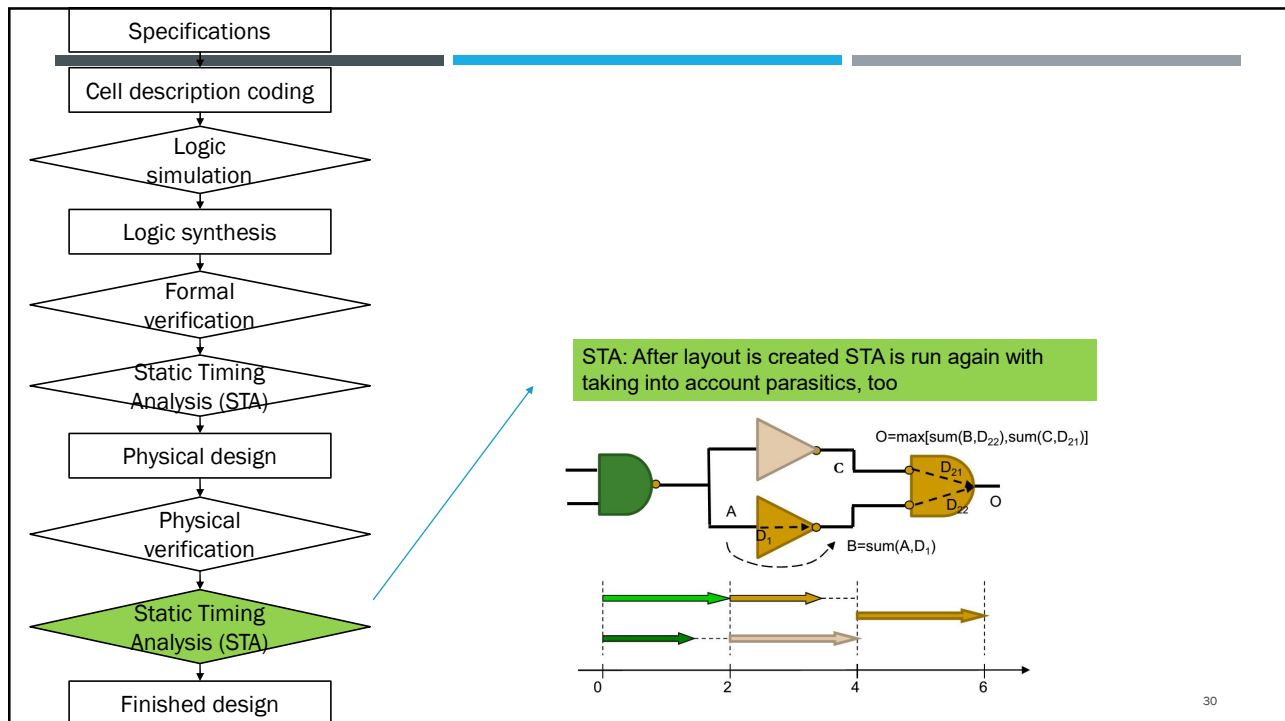
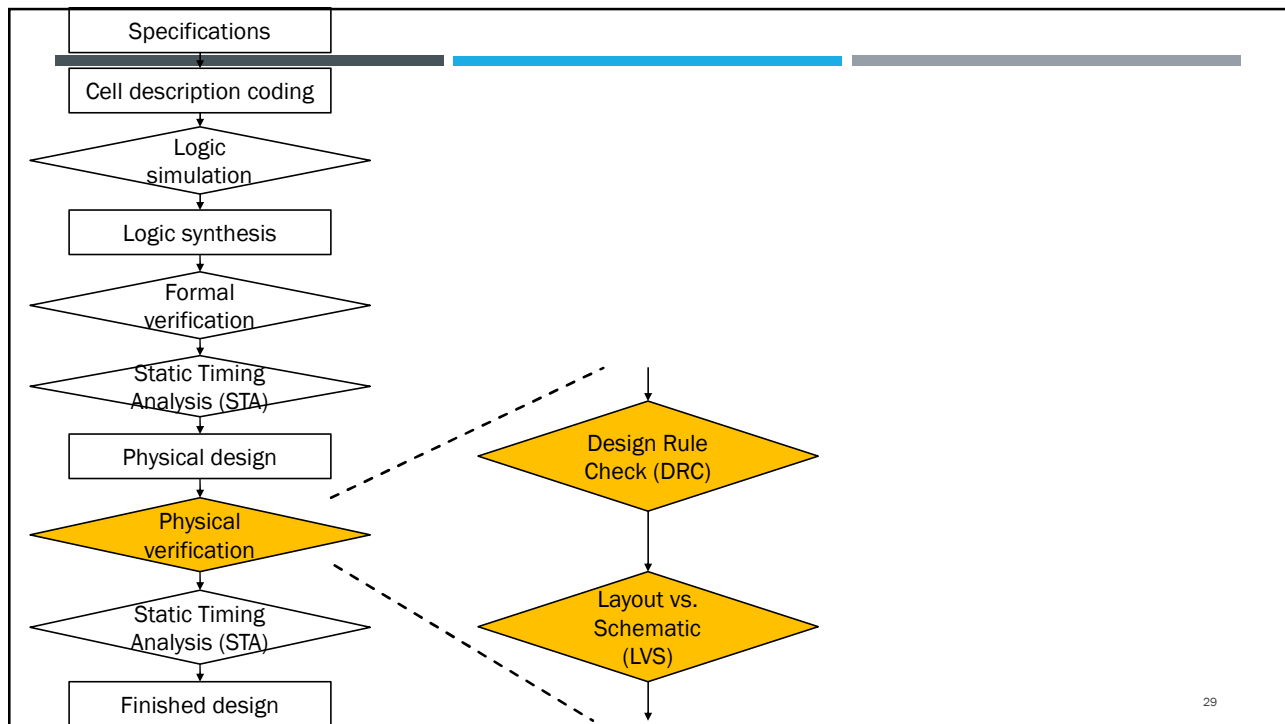


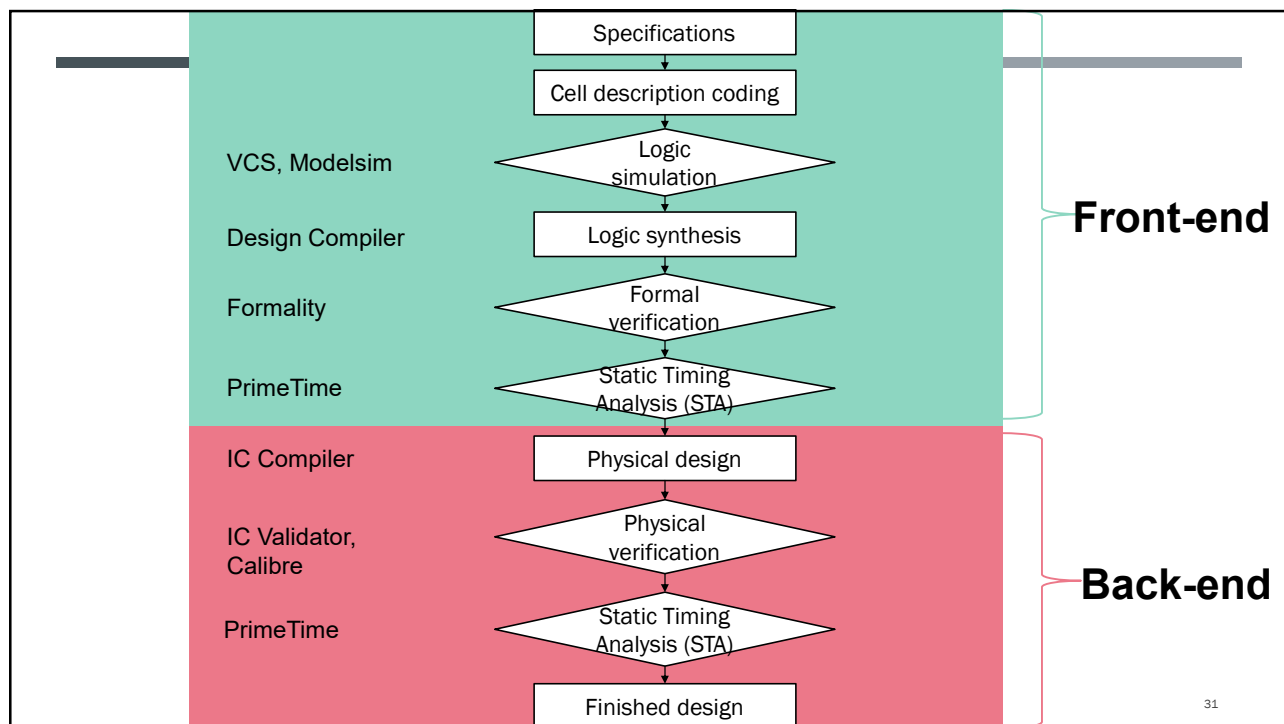
25



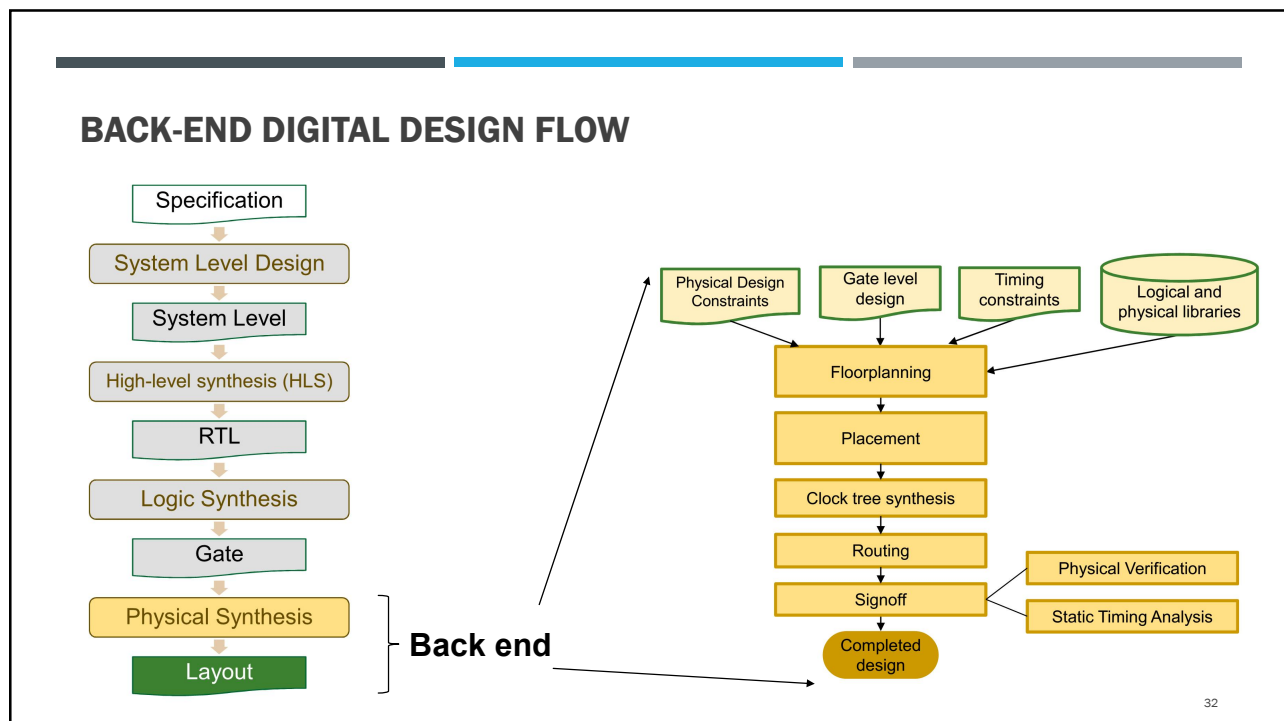
26







31



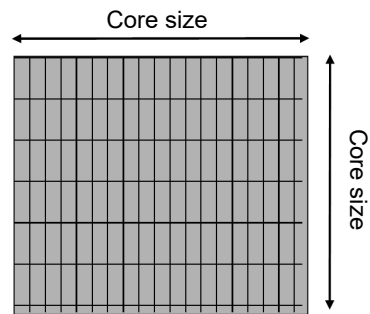
32

## BACK-END DIGITAL DESIGN FLOW

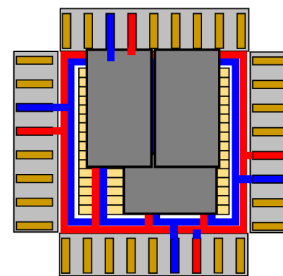
- **Floorplanning**

- During the floorplanning step

- The overall cell is defined, including: cell size, supply network, etc.
- The core size is defined



Floorplan



Power Planning

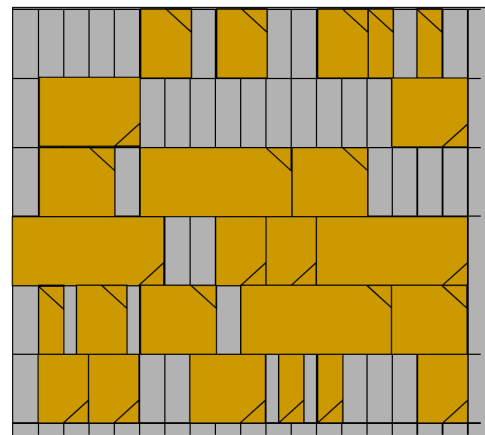
33

## BACK-END DIGITAL DESIGN FLOW

- **Placement**

- Cells are placed inside the floorplan with the optimization of

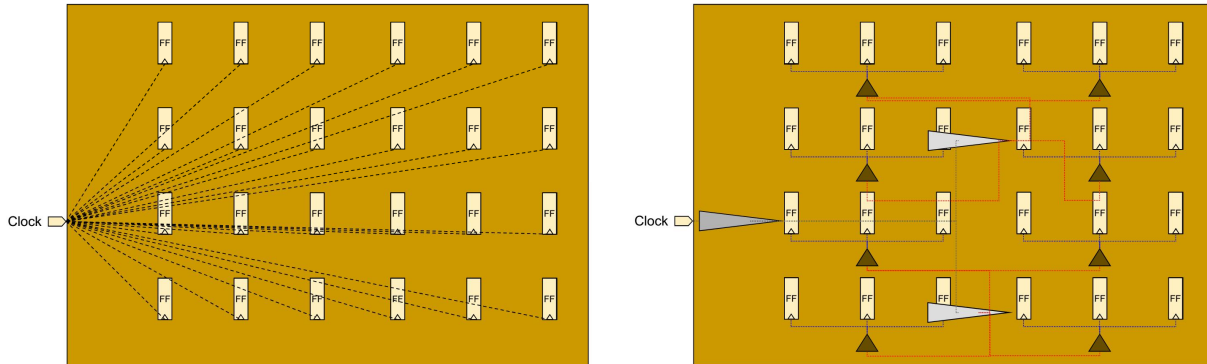
- Floorplan size
- Area
- Timing
- Power
- Clock path
- etc.



34

## BACK-END DIGITAL DESIGN FLOW

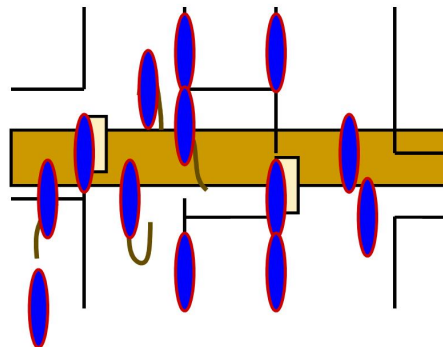
- **Clock Tree Synthesis (CTS)**
- A buffer tree is built to balance the loads and minimize the skew



35

## BACK-END DIGITAL DESIGN FLOW

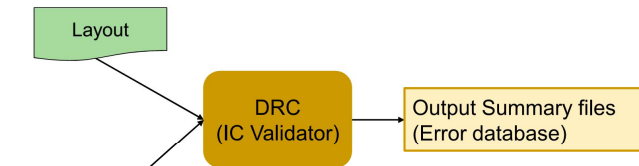
- **Routing**
- Routing creates physical connections to all clock and signal pins through metal interconnects
- Routed paths must meet setup and hold timing, max cap/trans, power and clock skew requirements
- Metal traces must meet physical DRC requirements



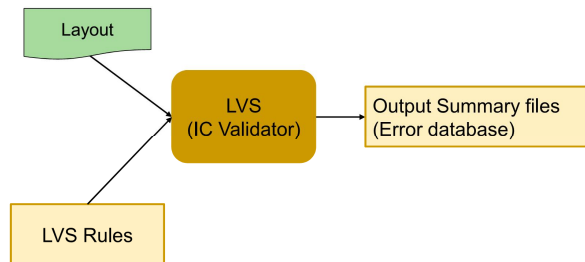
36

## BACK-END DIGITAL DESIGN FLOW

- Physical Verification
- Design Rule Check – DRC



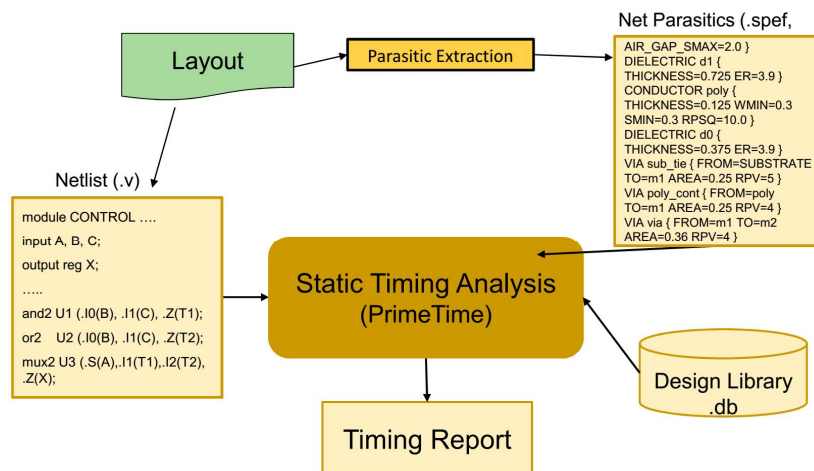
- Layout versus Schematic (LVS)



37

## BACK-END DIGITAL DESIGN FLOW

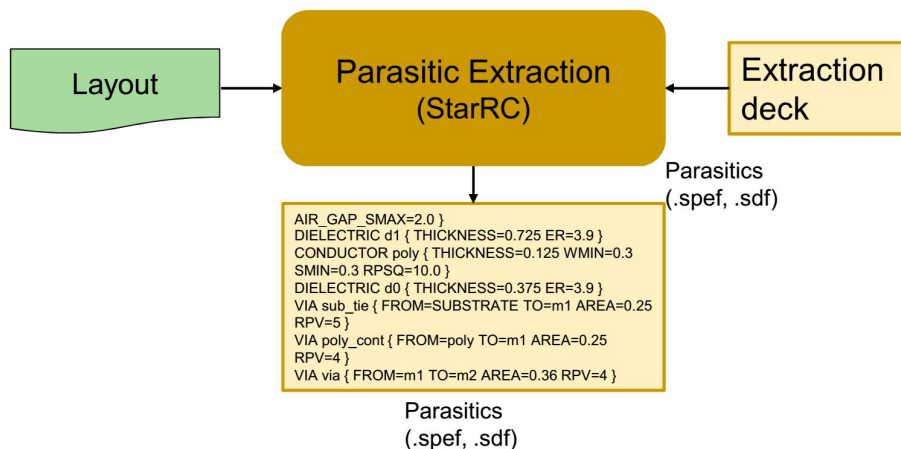
- Static Timing Analysis (STA)



38

## BACK-END DIGITAL DESIGN FLOW

### ■ Parasitic Extraction



39

## BACK-END EDA TOOLS

### Back-end EDA Tools (Digital IC design)

#### ■ SYNOPSYS

- Physical Design – **IC Compiler/ IC Compiler II (P&R), Design Compiler (Synthesis)**

#### Sign-off

- Physical Verification – **IC Validator**
- Parasitic Extraction – **StarRC**
- STA – **PrimeTime**

#### ■ CADENCE

- Layout – **Virtuoso**
- Physical Design – **Innovus**
- Physical Verification – **Assura**

#### ■ MENTOR GRAPHICS (SIEMENS ACQUIRED)

- Physical Design – **Aprisa (P&R), Oasys (Synthesis)**
- Physical Verification – **Calibre**

Designers must use automatic scripts (Tcl, Makefile, C-Shell, etc.) and commands in Back-end tools to implement Digital IC Design. All run on Linux/Unix.

40

## DESIGN FLOW

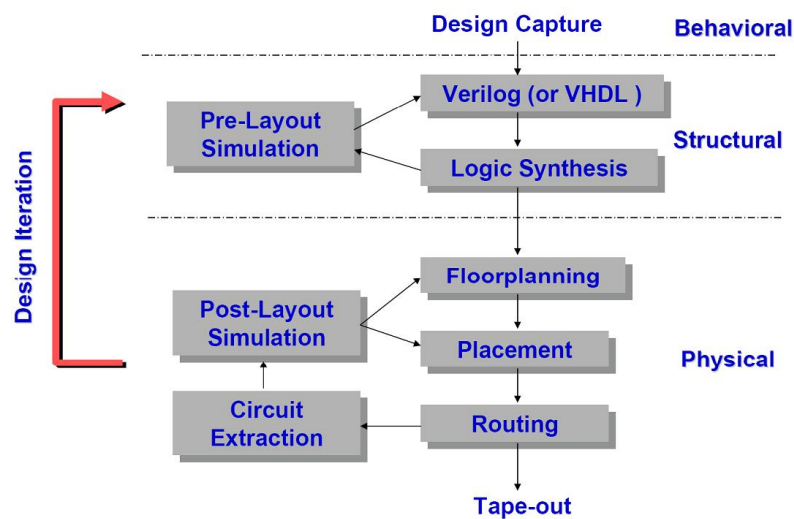
- **High Level Design**
    - Understanding specifications
    - Using C, C++, SystemC or SystemVerilog for behavior description
    - HW/SW Partition and IP selection
  - **RTL Design**
    - Verilog/VHDL
  - **System verification, timing response and timing issues**
- 
- **Physical Design**
    - Floorplanning, Place and Route, Clock insertion
  - **Performance and Manufacturability Verification**
    - Physic Parameter Extraction
  - **Verification of timing and signal integrity**
    - DRC/ LVS (Design Rule Checking/Layout versus Schematic)

FRONT END

BACK END

41

## DESIGN FLOW

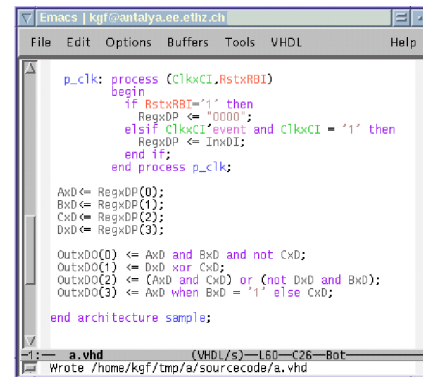


42

## FRONT-END DESIGN

### Hardware description language (HDL)

- Write RTL (Register Transfer Level) code to describe design ideas according to block diagrams
- Identify possible cases (test cases) and set up tests (testbench)
- Simulate blocks according to expected functionality and system functionality.
- Using Verilog HDL, VHDL, System Verilog, etc.



```
p_clk: process (clkxCI, RstxA9B1)
begin
    if RstxA9B1='1' then
        RegxD0 <= '0000';
    elsif clkxCI'event and clkxCI = '1' then
        RegxD0 <= InxD1;
    end if;
end process p_clk;

AxD <= RegxD0(0);
BxD <= RegxD0(1);
CxD <= RegxD0(2);
DxD <= RegxD0(3);

OutxD0(0) <= AxD and BxD and not CxD;
OutxD0(1) <= DxD xor CxD;
OutxD0(2) <= (AxD and CxD) or (not DxD and BxD);
OutxD0(3) <= AxD when BxD = '1' else CxD;

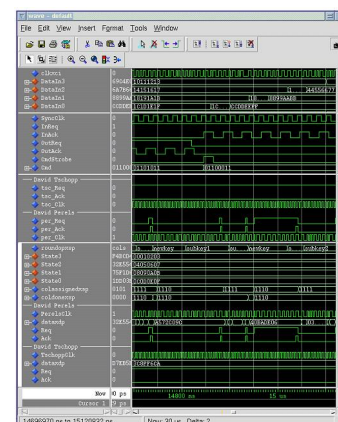
end architecture sample;
```

43

## FRONT-END DESIGN

### Simulation

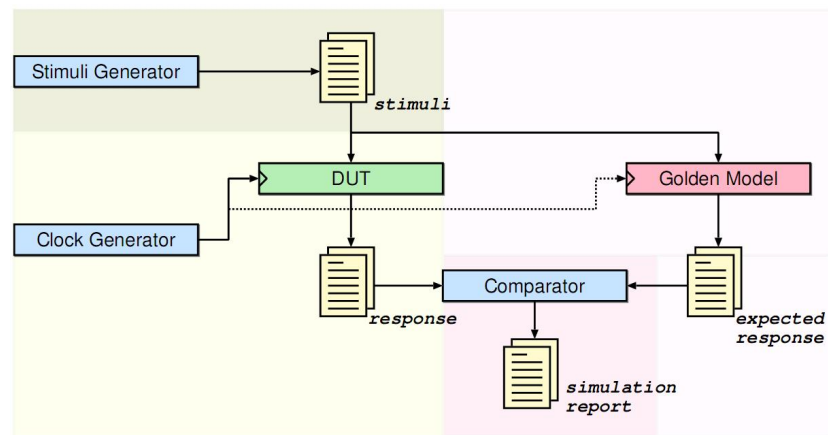
- Checking design working properly?
- Hardware Description
  - Not every architecture written in VHDL or Verilog can be implemented in hardware. The simulation does not show this.
- Behavioral simulation
  - No delay in processing
- Not characteristic of the parameter
  - Only function is simulated



44

## FRONT-END DESIGN

### Testbench



45

## FRONT-END DESIGN

### Verification

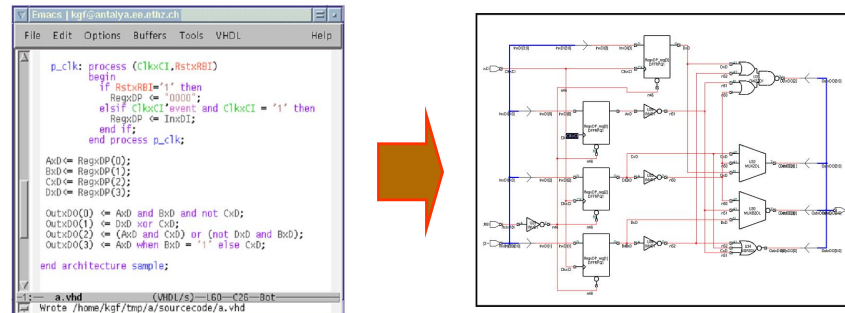
- Functional and faults testing
- Time consuming for testing the design.
- Full testing is impossible
  - The 32-bit adder has  $2^{64}$  possible cases. If every second tests 1,000,000,000, the testing time is 200 days
- The possibility of errors of writing codes
- Block diagrams can be wrong
- The hardware specifications are correct, but the standard model can be wrong.

46

## FRONT-END DESIGN

### Logic synthesis

- Compile hardware description code into hardware (association of logical elements)



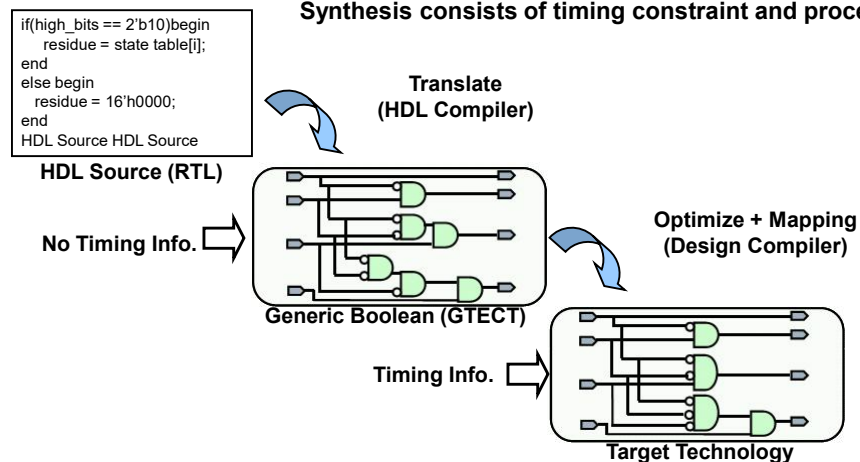
47

## FRONT-END DESIGN

### Logic synthesis

**Synthesis = translation + optimization + mapping**

Synthesis consists of timing constraint and process technology



48

## FRONT-END DESIGN

### Logic synthesis

- Circuit synthesis by using EDA tools
- This tool does not auto-generate the circuit by itself
- This tool only generates the exact circuit that the user wants to create
- The user must have a good idea with the generated circuit
- If the result is not as expected, the user should set the tool again so that the tool regenerates the correct circuit.

49

## FRONT-END DESIGN

### Standard Cell Libraries

- Standard cells of basic logic functions
  - NOT
  - AND, OR, XOR, NAND, NOR (2 → 5 inputs)
- Complex logic functions
  - Adders, And-Or-Invert (AOI), Or-And-Invert (OAI)
- Sequential function
  - Latches, Flip-flops
- Drive strengths: Each cell has at least 2 to 4 different current driving capabilities.
- These cells are designed with a specific process technology

AO21DL  
AND OR Gate

Description:  
And-Or gate, which has drive strength of 0.5x, with three data inputs (A1, A2, and B), and one output (Z).

#### Cell Attributes

| Area  | Static power (uW) | VDD  | Temp | Process |
|-------|-------------------|------|------|---------|
| 55.44 | 0.003             | 2.5V | 25°C | Typical |

#### Truth Table

| A1 | A2 | B  | Z |
|----|----|----|---|
| 1  | 1  | dc | 1 |
| dc | dc | 1  | 1 |
| 0  | dc | 0  | 0 |
| dc | 0  | 0  | 0 |

Propagation Delays(ns) for Sample Loads, standard load = 0.008pF slew = 0.005ns

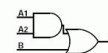
| Fanout | 1     | 2     | 3     | 4     | 8     | 16    |
|--------|-------|-------|-------|-------|-------|-------|
| Edge   | Rise  | Fall  | Rise  | Fall  | Rise  | Fall  |
| A1→Z   | 0.118 | 0.151 | 0.143 | 0.175 | 0.168 | 0.194 |
| A2→Z   | 0.122 | 0.169 | 0.147 | 0.193 | 0.171 | 0.213 |
| B→Z    | 0.091 | 0.141 | 0.115 | 0.165 | 0.138 | 0.184 |

Power Dissipation(pW) for Sample Loads, standard load = 0.008pF slew = 0.005ns

| Fanout | 1     | 2     | 3     | 4     | 8     | 16    |
|--------|-------|-------|-------|-------|-------|-------|
| Edge   | Rise  | Fall  | Rise  | Fall  | Rise  | Fall  |
| A2→Z   | 0.062 | 0.132 | 0.062 | 0.132 | 0.063 | 0.130 |
| A1→Z   | 0.067 | 0.114 | 0.067 | 0.113 | 0.067 | 0.113 |
| B→Z    | 0.040 | 0.092 | 0.040 | 0.092 | 0.040 | 0.091 |

#### Pin Descriptions

| Pin | Capacitance pF | Description |
|-----|----------------|-------------|
| A1  | 0.008          | data input  |
| A2  | 0.008          | data input  |
| B   | 0.007          | data input  |
| Z   |                | data output |



50

## STANDARD CELL LIBRARY

### Technology Library

Each library contains hundreds to thousands of standard cells, consisting of:

- Function
  - Gates – inverter, AND, NAND, NOR, XOR, AOI, OAI
  - Flops – Flip flops, Latches, Scan Flops, Gated Flops
  - I/O Cells – Input-Output pads, Bidirectional Pads, Complex
- Process
  - Mask layer options, gate shrinks, metals, special diffusions, thick metal, multiple oxides
- Cell selection
  - Drive strengths, sets, resets, scans, substrate ties, antenna diodes

51

## STANDARD CELL LIBRARY

### Technology Library

- Many options to select a cell in a library:
  - Area
  - Power: low power, low leakage, low voltage
  - Timing constraints (Setup/Hold time, Recovery/Removal time)
  - Timing requirements:
    - Input Capacitance
    - Transition time
    - (Output Slew)

52

## STANDARD CELL LIBRARY

### Technology Library

- Power extraction, IR-Drop analysis, EM analysis
- Dynamic Power (switching power)
- Static Power (leakage power)
- Passive power (internal power): power used by sequential cells (e.g. flip-flops) when inputs/clocks change without changing at outputs
- Delays
- Used for Place & Route
- Geometry sizes
- Pin locations
- Information relation to Foundries, IP Vendors, Fabless, IDMs

53

## STANDARD CELL LIBRARY

### Technology Library

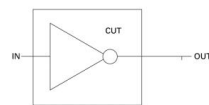
- Example: Inverter

| IEEE-1164 Verilog Logic States |       |                |
|--------------------------------|-------|----------------|
| Strength                       | State | Value          |
|                                | U     | Uninitialized  |
| Driven                         | X     | Unknown        |
| Driven                         | 0     | Low            |
| Driven                         | 1     | High           |
|                                | z     | High impedance |
| Resistive                      | W     | Weak X         |
| Resistive                      | L     | Weak 0         |
| Resistive                      | H     | Weak 1         |
|                                | --    | Don't care     |

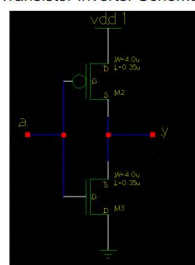
#### Verilog Language Description of Inverter

```
not i1 (out, in); // basic inverter
```

```
not #(5,3) i1 (out, in); // Rise time=5ns, Fall time=3ns
```



Transistor Inverter Schematic



### Schematic Netlist

```
*svc_inv.sch
```

```
M3 y a gnd gnd nmos L=0.35u W=4.0u
```

```
M2 y a vdd vdd pmos L=0.35u W=4.0u
```

```
.END
```

### Schematic Netlist with Parasitics

```
*svc_inv.sch
```

```
M3 y a gnd gnd nmos L=0.35u W=4.0u
```

```
M2 y a vdd vdd pmos L=0.35u W=4.0u
```

```
C1 .....
```

```
C2 .....
```

```
C3 .....
```

```
.END
```

54

## STANDARD CELL LIBRARY

### Technology Library

- **Cell Characteristics**
  - Cell Library Attributes
  - Measurements
  - Cell Library Model Quality
  - Liberty .lib/.db File Structure
  - Liberty .lib File Library Level Attributes
  - Operating Conditions
  - Cell Attributes in .lib File
  - Datasheet View
  - Pin Attributes
  - Setting Output Load Limits

55

## STANDARD CELL LIBRARY

### Technology Library

- **Cell Characterization Attributes**
- Pin types
  - Direction (input/output)
  - Function
- Load
  - Capacitive
  - Active
  - Fanout, wire loads
- Simulation
  - PWL for slope
  - Active drivers
- Parameters
  - Load
  - Input slope

```
pin(Z) {  
  direction : output ;  
  function : "!A";  
  max_capacitance : 0.08532 ;}
```

```
lu_table_template(wire_delay_table_template) {  
  variable_1 : fanout_number;  
  variable_2 : fanout_pin_capacitance;  
  variable_3 : driver_slew;  
  index_1 ("1.0 , 3.0");  
  index_2 ("0.12, 4.24");  
  index_3 ("0.1, 2.7, 3.12");  
}  
lu_table_template(trans_template) {  
  variable_1 : total_output_net_capacitance;  
  index_1 ("0.0, 1.5, 2.0, 2.5");  
}  
wire_load("05x05") {  
  resistance : 0 ;  
  capacitance : 1 ;  
  area : 0 ;  
  slope : 0.186 ;  
  fanout_length(1,0.39) ;  
  interconnect_delay(wire_delay_table_template)  
  values("0.00,0.21,0.3", "0.11,0.23,0.41", \  
    "0.00,0.44,0.57", "0.10 0.3, 0.41");  
}
```

56

## STANDARD CELL LIBRARY

### Technology Library

- **Cell Characterization Attributes**
- Capacitors
- Thresholds/switching points
- Rise Time
- Fall Time
- Delay (propagation + transition = cell)
- Power( static state dependent leakage, dynamic, short-circuit, hidden, internal)

57

## STANDARD CELL LIBRARY

### Technology Library

- **Liberty .lib File Library Level attributes**

```
library (name) {  
  technology (name) ;/* library-level attributes */  
  delay_model : generic_cmos | table_lookup |  
  cmos2 | piecewise_cmos | dcm |  
  polynomial ;  
  bus_naming_style : string ;  
  routing_layers(string);  
  time_unit : unit ;  
  voltage_unit : unit ;  
  current_unit : unit ;  
  pulling_resistance_unit : unit ;  
  capacitive_load_unit(value,unit);  
  leakage_power_unit : unit ;  
  operating_conditions(){  
    process: value;  
    temperature: value;  
    voltage: value;  
  }  
}
```

```
library(1533EF13SA_ff) {  
  /* general attributes */  
  delay_model : table_lookup ;  
  in_place_swap_mode : match_footprint ;  
  library_features(report_delay_calculation);  
  /* unit attributes */  
  time_unit : 1ns ;  
  voltage_unit : 1V ;  
  current_unit : 1mA ;  
  pulling_resistance_unit : 1kohm ;  
  leakage_power_unit : 1pW ;  
  capacitive_load_unit(1, pf);  
  /* operation conditions */  
  operating_conditions(fast) {  
    process : 1.0 ;  
    temperature : -40 ;  
    voltage : 1.65 ;  
    tree_type : balanced_tree ;  
  }  
}
```

58

## STANDARD CELL LIBRARY

### Technology Library

- **Operation Conditions**

- **Name**

- Name of operation condition in a library (e.g. WCCOM)

- **Process**

- Due to errors during fabrication, the devices will be affected. We usually choose at **typical** mode (fast, typical, slow).

- **Voltage**

- Operation voltages

- **Temperature**

- Ambient temperature affects design's performance

- **Power\_rail**

- Voltage values for power

```
operating_conditions(fast) {  
  process : 1.0 ;  
  temperature : -40 ;  
  voltage : 1.65 ;  
  tree_type : balanced_tree ;  
}
```

```
library (example) {  
  operating_conditions (WCCOM) {  
    process : 1.5 ;  
    temperature : 70 ;  
    voltage : 4.75 ;  
    tree_type : worst_case_tree ;  
    power_rail(v1, 2.5);  
    power_rail(v2, 3.9) ;  
  }  
}
```

59

## STANDARD CELL LIBRARY

### Technology Library

- **Cell Attributes in .lib technology file**

- **Structures**

- Each cell is described in structures, bus, pin

- **Function**

- Each function is described with the corresponding function

- **Timing**

- Timing analysis and optimization information about pin-pin, delay, propagation, timing constraints in sequential circuits

- **Source**

- State-dependent model and path-dependent model

- **Other parameters**

- Area, design rules

60

## STANDARD CELL LIBRARY

### Technology Library

```
cell(IVD1){
  area : 4.7601 ;
  cell_footprint : IV ;
  cell_leakage_power : 6.46088 ;
}
pin(A) {
  capacitance : 0.00479699 ;
  direction : input ;
}
pin(Z) {
  direction : output ;
  function : "IA" ;
  max_capacitance : 0.08532 ;
}
fall_power(pwr_tin_load_7x7) {
  index_1("0.004, 0.017419, 0.064524, \
");
  index_2("0.00079, 0.0023106, 0.007648, ");
  values("-0.000607554, -0.000456619, -0.000255261", \
"-0.00112551, -0.00100802, -0.000732359", \
"-0.00100449, -0.00102865, -0.000974434");
}
rise_power(pwr_tin_load_7x7) {
  index_1("0.004, 0.017419, 0.064524");
  index_2("0.00079, 0.0023106, 0.007648");
  values("0.0121026, 0.0122204, 0.0124768", \
"0.0115487, 0.0115934, 0.0118285", \
"0.0118955, 0.0119009, 0.0118277");
}
}
```

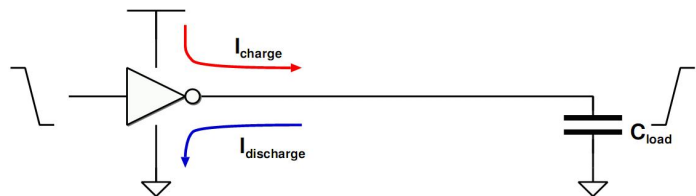
```
timing() {
  related_pin : "A";
  timing_sense : negative_unate ;
  cell_fall(tmg_ntin_load_7x7) {
    index_1("0.004, 0.017419, 0.064524");
    index_2("0.00079, 0.0023106, 0.007648");
    values("0.011482, 0.013873, 0.021955", \
"0.017427, 0.020706, 0.029807", \
"0.025032, 0.030851, 0.047121");
  }
  cell_rise(tmg_ntin_load_7x7) {
    index_1("0.004, 0.017419, 0.064524");
    index_2("0.00079, 0.0023106, 0.007648");
    values("0.014198, 0.017529, 0.028794", \
"0.021899, 0.025822, 0.037284", \
"0.036564, 0.04352, 0.062827");
  }
  fall_transition(tmg_ntin_load_7x7) {
    index_1("0.004, 0.017419, 0.064524, ");
    index_2("0.00079, 0.0023106, 0.007648, ");
    values("0.004958, 0.006865, 0.013566, ", \
"0.00759, 0.009166, 0.014178", \
"0.014867, 0.017672, 0.025296", \
"0.023954, 0.027858, 0.039089");
  }
  rise_transition(tmg_ntin_load_7x7) {
    index_1("0.004, 0.017419, 0.064524");
    index_2("0.00079, 0.0023106, 0.007648");
    values("0.007111, 0.009852, 0.019457", \
"0.009401, 0.011327, 0.01952", \
"0.01759, 0.020894, 0.029954");
  }
} }
```

61

## FRONT-END DESIGN

### Timing constraints

- To charge capacitor as fast as possible
  - Decrease load capacitor:  $C_{load}$
  - Increase driving (strength) current



$$I_{D,lin} = \mu \cdot C_{ox} \cdot \frac{W}{L} \cdot [(V_{GS} - V_T) \cdot V_{DS} - \frac{V_{DS}^2}{2}]$$

$$I_{D,sat} = \mu \cdot C_{ox} \cdot \frac{W}{L} \cdot \frac{(V_{GS} - V_T)^2}{2}$$

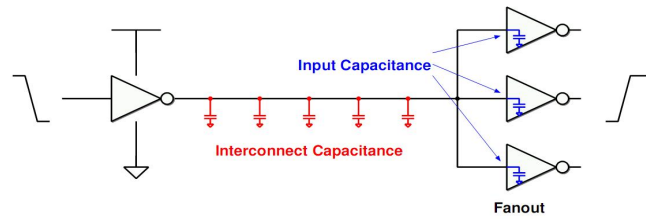
- Driving current depends on W/L ratio transistors.
- W/L aspect ratio increases → increased current → timing is faster

62

## FRONT-END DESIGN

### Timing constraints

- Capacitive Loads
  - Sources of generating load capacitors: input capacitances of the next input, interconnect capacitance
- Basic concepts of timing
  - If the load is constant, the wider the transistor the faster timing.
  - The larger a transistor, the higher input parasitic capacitor → It makes it more difficult to control the transistor
  - There must be a balance between current and input parasitic capacitors
  - The fanout of the transistor and the length of the connection wire will determine the switching speed.
  - Interconnection length can be estimated.



63

## FRONT-END DESIGN

### Circuit constraints

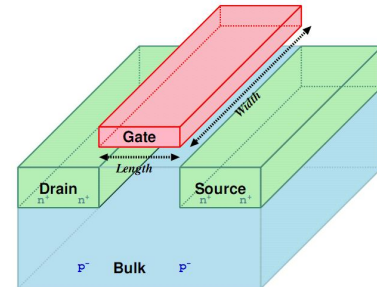
- Synthesis constraints
- Max. delay
- Max. area
- Set constraints but do not guarantee circuit results

64

## FRONT-END DESIGN

### Circuit constraints

- **Input capacitance**
- MOS Capacitance
  - The most influential parasitic capacitor is the gate capacitor.
  - The gate capacitor is proportional to the area of the gate.
  - The larger the width, the larger its parasitic capacitor
- **Sequential timing**
- Timing paths: there are 4 timing paths in sequential circuits:
  - Register to register
  - Input to register
  - Register to output
  - Input to output

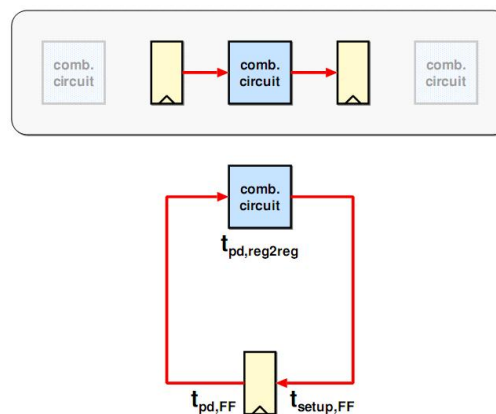


65

## FRONT-END DESIGN

### Circuit constraints

- **Sequential timing**
  - Register to register



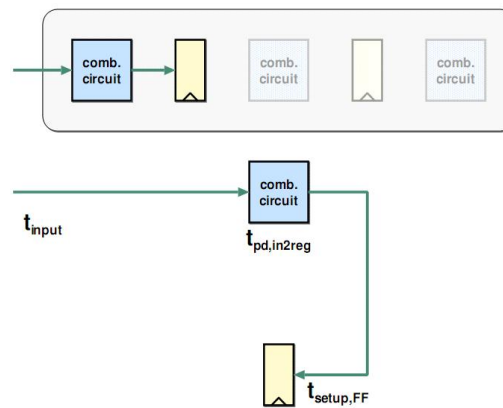
66

## FRONT-END DESIGN

### Circuit constraints

- Sequential timing

- Input to register



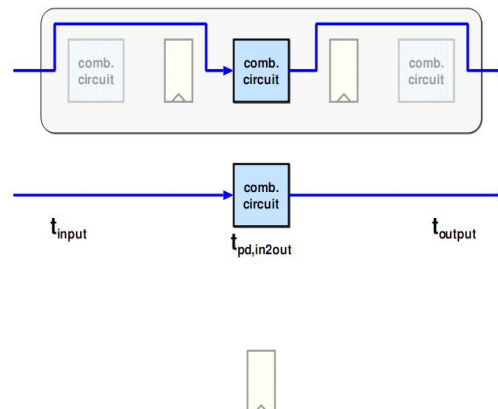
67

## FRONT-END DESIGN

### Circuit constraints

- Sequential timing

- Register to output
- Input to output



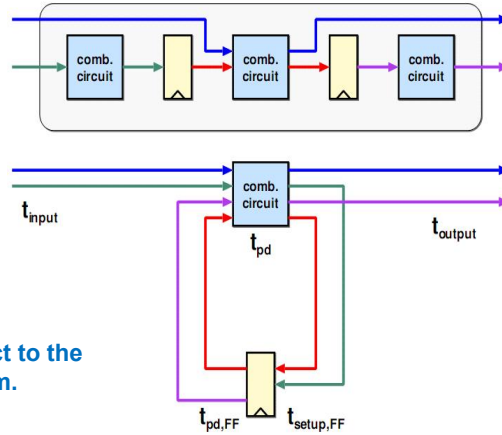
68

## FRONT-END DESIGN

### Circuit constraints

- **Sequential timing**

- Register to register
- Input to register
- Register to output
- Input to output



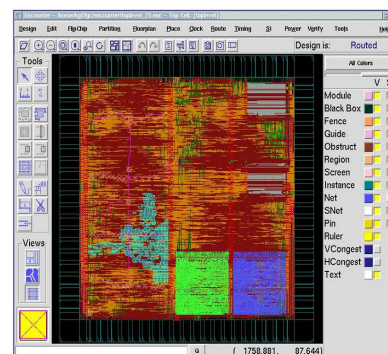
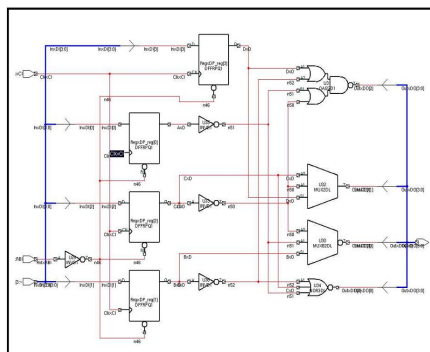
These timing paths will affect to the operation of the system.

69

## BACK-END DESIGN

### Physical Design

- Assume that netlist is available (results after circuit synthesis), then convert netlist to layout (physical design).

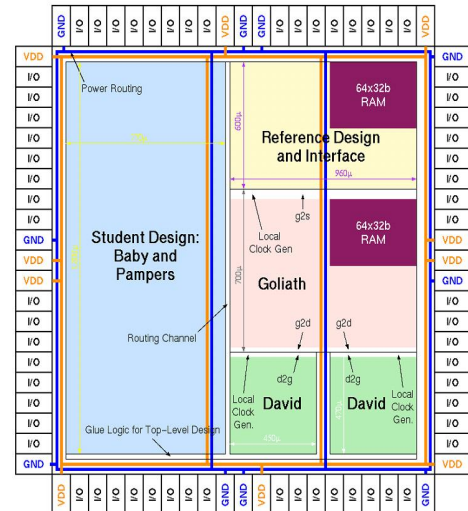


70

## BACK-END DESIGN

### Floorplanning

- Determine the total area, geometry of the chip
- Place I/O cells
- Place macro blocks (IPs/hard cores)
- Try net routing, power wiring, and optimization
- This step is repeated many times until achieving the best optimized results

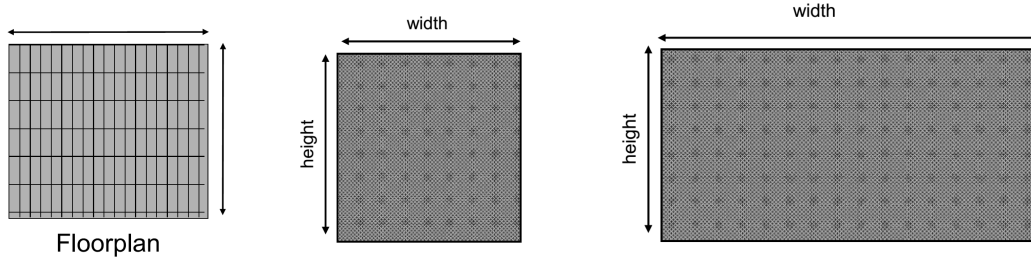


71

## BACK-END DESIGN

### Floorplanning

- During the floorplanning step the overall cell is defined, including: cell size, supply network, etc
- Aspect ratio is the height to width ratio of a block
  - Defines the block shape
  - Default aspect ratio is 1



72

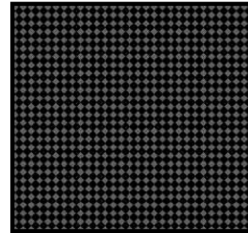
## BACK-END DESIGN

### Floorplanning

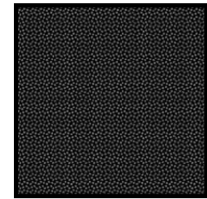
- Area Utilization dictates how densely user wants cells to be placed within the block

$$Utilization = \frac{\sum(Cell\ Area)}{Core\ Area}$$

- The remaining space is needed for ease of routing
- Increasing utilization will reduce the core area
- Note:** Default Utilization is 0.6



set\_utilization 0.6



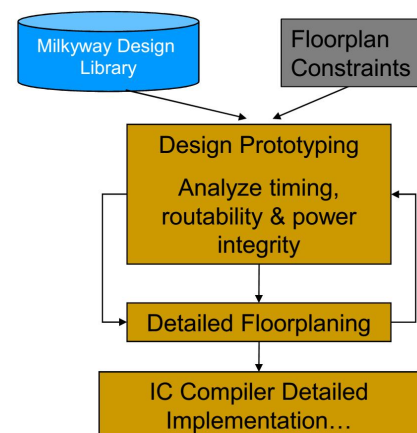
set\_utilization 0.85

73

## BACK-END DESIGN

### Floorplanning

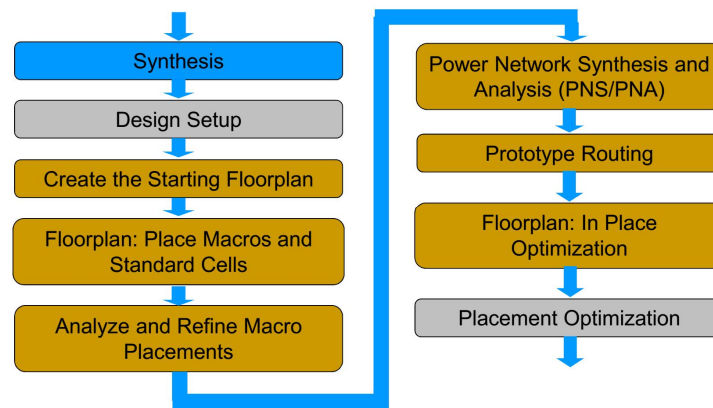
- The Milkyway library (Synopsys) was created during the design setup stage.
- It contains the design connectivity (netlist, timing/area/power constraints) pointers to refer libraries, TLU+ models for process and the UNIX directory structures to hold it all.



74

## BACK-END DESIGN

### Design Planning – Basic Steps



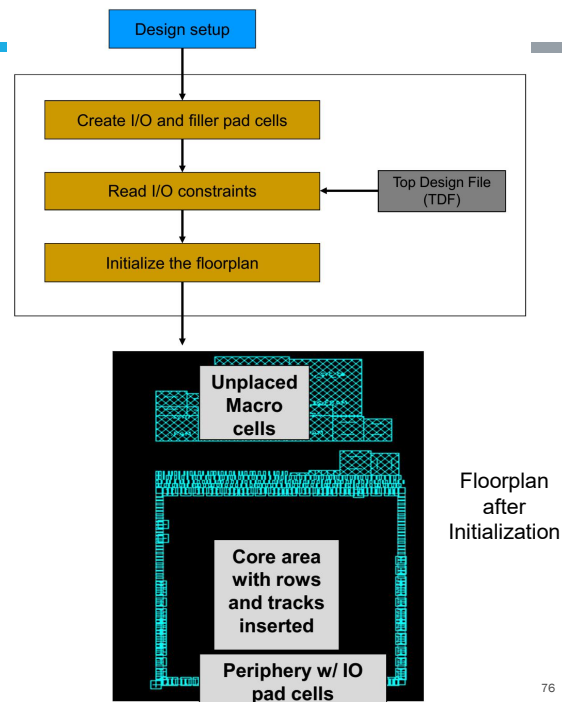
75

## BACK-END DESIGN

### Floorplanning

The steps in initializing the floorplan:

- Reading the I/O Constraints
- Defining the Core and Placing the I/O Pads
- Creating Rectilinear-Shaped Blocks
- Writing I/O Constraint Information
- Adding Cell Rows
- Removing Cell Rows
- Saving the Floorplan Information



76

## BACK-END DESIGN

### Floorplanning

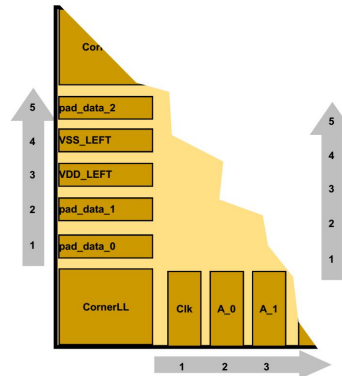
- TDF (Top Design File) is used to input I/O constraints
  - pin commands control order and side placement of top I/Os
- Read the TDF file: `read_io_constraints`
  - Loads the TDF file to define IO constraints such as pin/pad location, edge, layer, order etc.
  - Constraints are honored in `initialize_floorplan` and other IO related commands
- Write out a TDF file: `write_io_constraints`
  - Write (dump) I/O pad or pin constraint information to a file

```
; tdf file

; Place the corner cells
pad "cornerLL" "bottom"
pad "cornerLR" "right"
pad "cornerTR" "top"
pad "cornerTL" "left"

; place io and power pads
; left and right sides count from to top (excluding corner)
pad "pad_data_0" "left" 1
pad "pad_data_1" "left" 2
pad "VDD_LEFT" "left" 3
pad "VSS_LEFT" "left" 4
pad "pad_data_2" "left" 5

; Bottom and top sides count from left to right(excluding corner)
pad "Clk" "bottom" 1
pad "A_0" "bottom" 2
pad "A_1" "bottom" 3
```

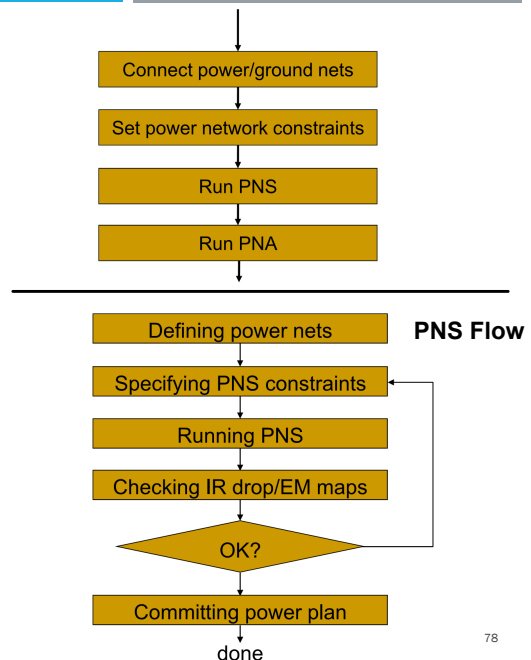


77

## BACK-END DESIGN

### Power Network Synthesis and Analysis (PNS/PNA)

- **Power Network Synthesis**
  - Create power/ground rings and straps
  - Creates macro power rings
  - Creates the power grid
  - Power topology definition
  - Calculation of the width and the number of power straps to meet IR constraints
  - Performance of detailed P/G connections and via placement
- Validation with PNA



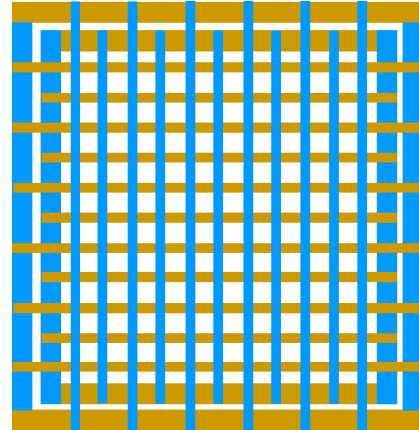
78

## BACK-END DESIGN

### Power Network Synthesis and Analysis (PNS/PNA)

#### Power Network Synthesis

- Supported topology with PNS
- PNS currently creates a power plan with/without a core ring connected to a power mesh
- User can specify:
  - Number of straps: min., max.
  - Width of straps: min., max.
  - Width of ring
  - Layer
  - IR drop constraint

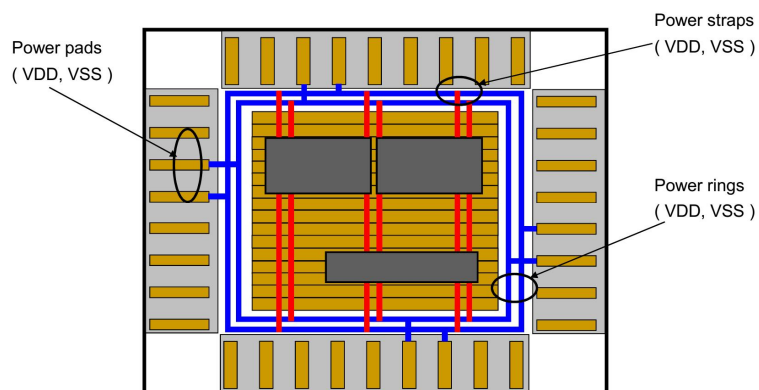


79

## BACK-END DESIGN

### Power Network Synthesis and Analysis (PNS/PNA)

#### Power Network Synthesis

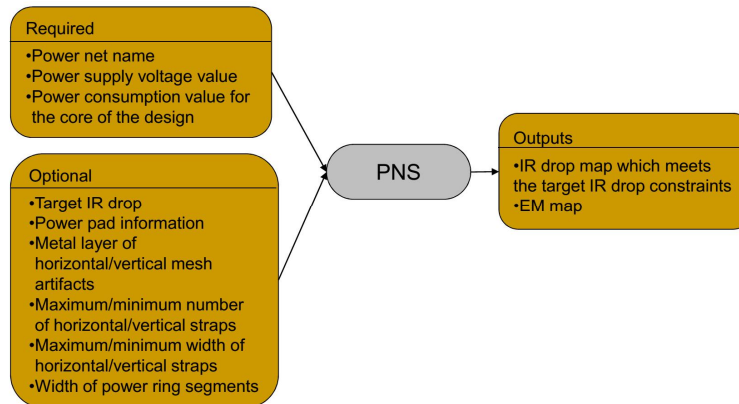


80

## BACK-END DESIGN

### Power Network Synthesis and Analysis (PNS/PNA)

#### Inputs and Outputs of Power Network Synthesis



81

## BACK-END DESIGN

### Power Network Synthesis and Analysis (PNS/PNA)

#### Power Network Analysis

- Power Network Analysis is needed for power network planning and synthesis as it forms the core of the IR and EM analysis that is required to ensure that PNS goals are met.
- Electromigration analysis
  - Current density analysis for every power segment
  - Too high current density leads to damaged interconnect, reduced useful life of chip and reliability issue
- IR/Voltage Drop analysis
  - Voltage drop at each node of power network
  - Too low power supply voltage leads to increased delay and timing issue

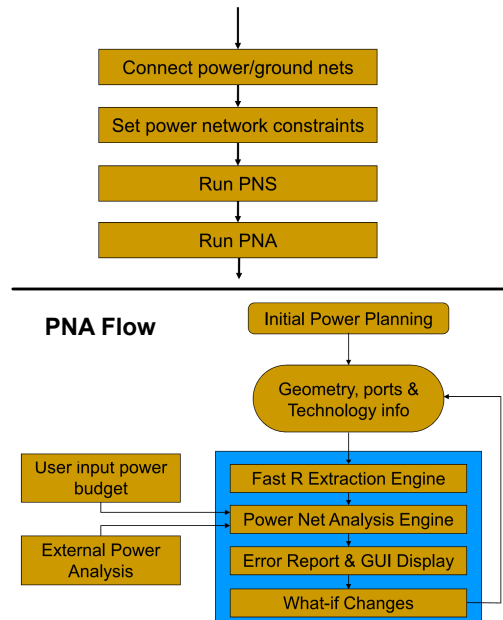
82

## BACK-END DESIGN

### Power Network Synthesis and Analysis (PNS/PNA)

#### Power Network Analysis

- After the creation of power network rings, straps and IO connections:
  - Power network does not have to be completed
- Define virtual power pads if power supply does not exist on the chip
- Run PNA to analyze power network
  - Analyze IR drop, voltage drop and electromigration
- Adjust power network to solve reported issues
  - Modify virtual power pads to improve voltage drop map



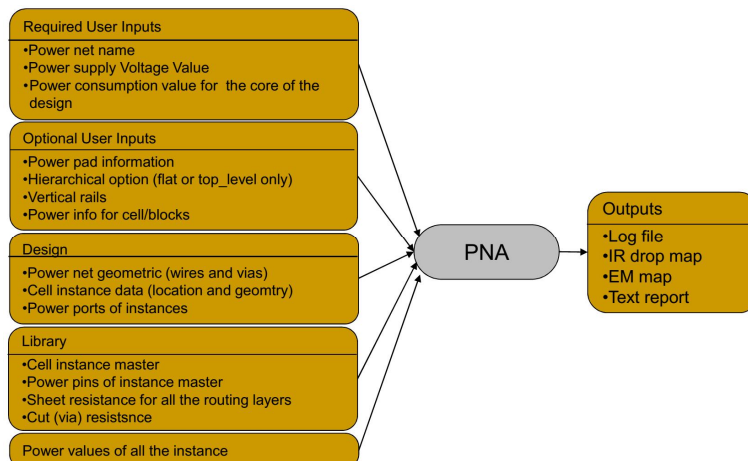
83

## BACK-END DESIGN

### Power Network Synthesis and Analysis (PNS/PNA)

#### Inputs and Outputs of

#### Power Network Analysis



84

## BACK-END DESIGN

### In Placement Optimization

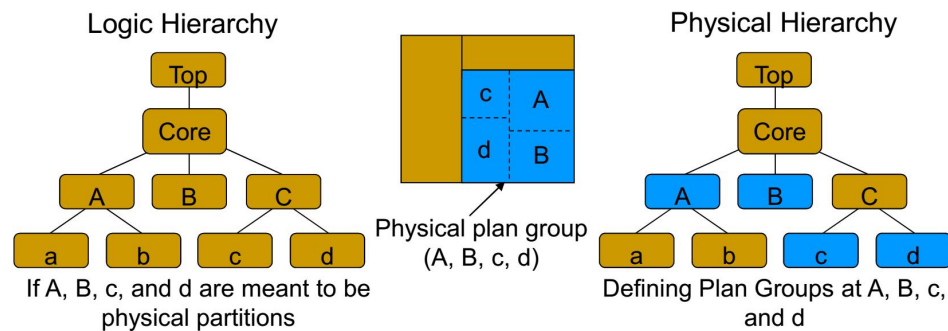
- In-place optimization is an iterative process that is based on virtual routing
- In-place optimization can be run before or after global routing. If in-place optimization is run after global routing, the global routes must first be deleted.
- High Fanout buffer trees are synthesized automatically
- Analyzation using: Constraint reports, Timing reports, Congestion maps
- Types of optimizations that are performed:
  - Improve overall timing
  - Area recovery
  - Fixing DRC violations

85

## BACK-END DESIGN

### Relationship between Logic Hierarchy and Physical Hierarchy

- Each group of modules in logic hierarchy, which in physical design must be figured together, are implemented as a physical block in physical hierarchy.



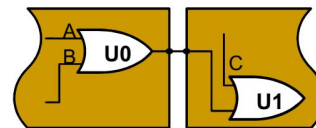
86

## BACK-END DESIGN

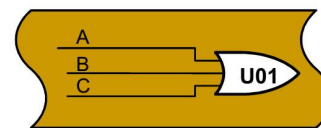
### Relationship between Logic Hierarchy and Physical Hierarchy

- Logical hierarchy
  - Irrelevant for placement
  - But affects logic optimization
- Logic optimization
  - May impact timing optimization negatively
  - Ungrouping design hierarchies to improve optimization
- Physical hierarchy
  - Relevant for placement
  - Placement → flat and hierarchy ignored
- Physical optimization
  - May impact timing optimization positively
  - Ungrouping design hierarchies to determine optimization

#### Logic Hierarchy



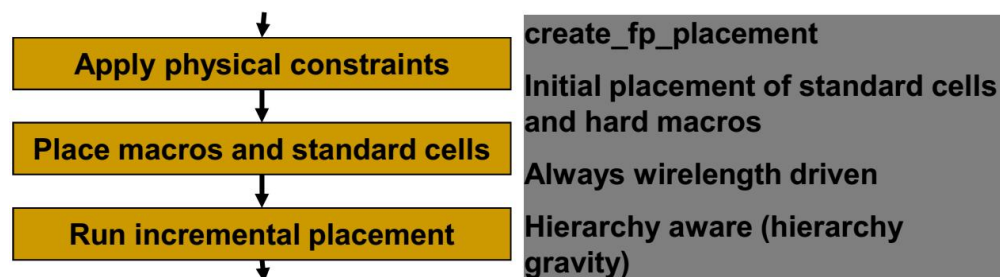
#### Physical Hierarchy



87

## BACK-END DESIGN

### Floorplan: Place Standard and Macro Cells

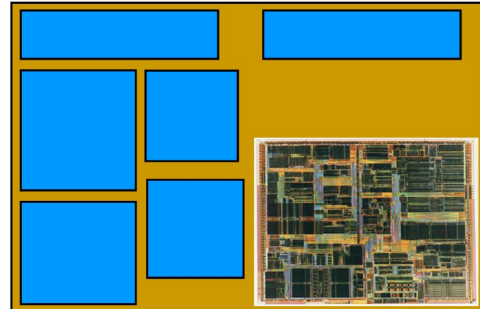


88

## BACK-END DESIGN

### Floorplan: Place Standard and Macro Cells – Hard Macros

- Hard macros is a physical plan group which is parameterized by rectangular coarse sizes and pin order.
- As a rule, they are IP blocks.
- The easiest path to SoC
- Hard macros make the assembly more difficult
- No resizing capability to fix timing during assembly



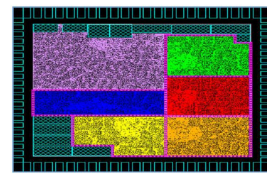
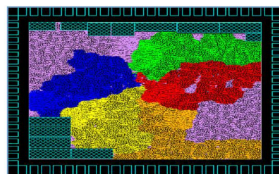
89

## BACK-END DESIGN

### Floorplan: Place Standard and Macro Cells – Soft Macros

- Soft macros is a plan group which is isolated from logical hierarchy and belongs to a certain physical hierarchy
- Soft macros are not parameterizable and has a certain area, but it lacks certain shape.
  - Soft macros will allow better global optimization
  - Final assembly may solve shape, pin, and global timing problems causing reduced design iterations.

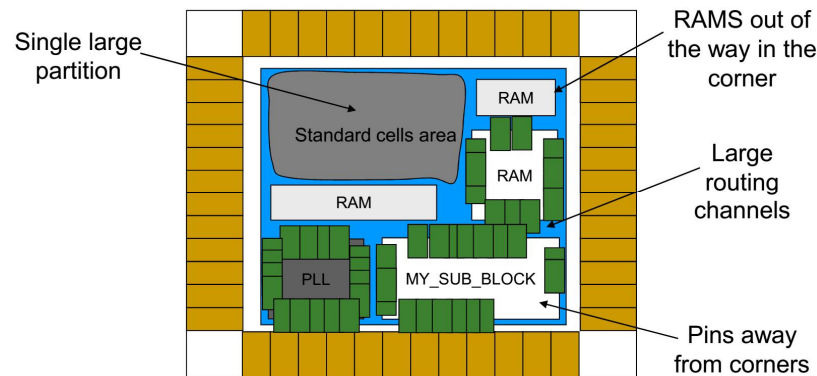
Soft macros before parameterization      Soft macros after parameterization



90

## BACK-END DESIGN

### Floorplan: Place Standard and Macro Cells – Consistency of Macros and Standard Cells

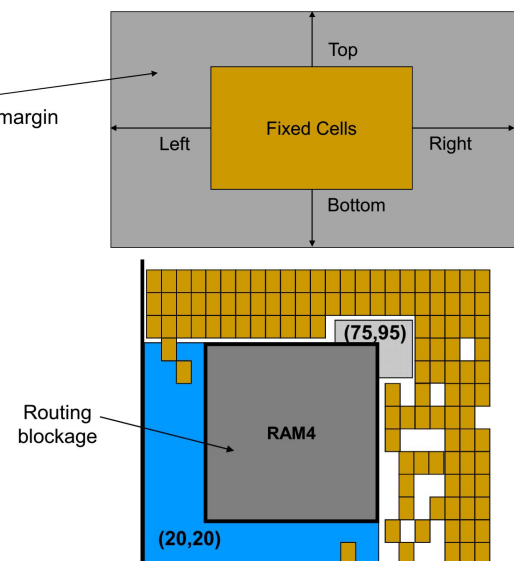


91

## BACK-END DESIGN

### Floorplan: Place Standard and Macro Cells – Placement and Routing Blockages

- **Placement blockages** are areas that leaf cells must void during placement and legalization, including overlapping any part of the placement blockage
- **Routing blockage** – areas where routing is not allowed

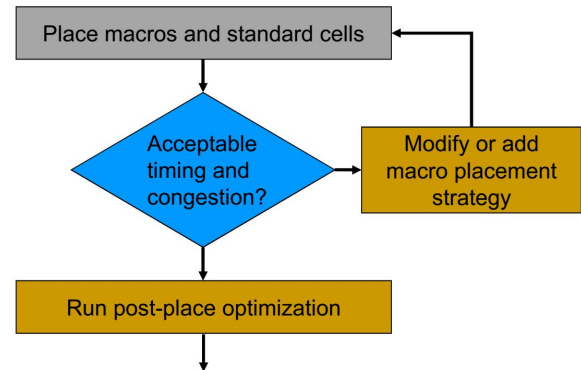


92

## BACK-END DESIGN

### Macro placement constraints

- Macro placement constraints have an impact on placing and further global routing of standard cells
- Some basic constraints of macro placement are
  - Alignment of macros by edges
  - Grouping macros - For standard cells, rectangular area should be left as much as possible
- Some basic parameters of macros placement
  - Routability
  - Timing
  - Wire length
  - Area for standard cells
  - Congestion problems
  - Timing problems

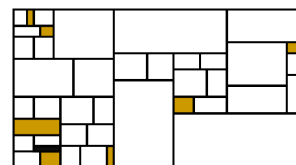


93

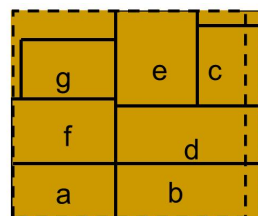
## BACK-END DESIGN

### Floorplanning

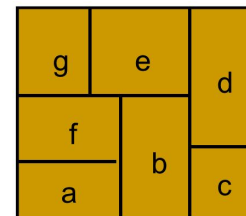
- The floorplanning problem is to plan the positions and shapes of the modules at the beginning of the design cycle to optimize the circuit performance:
  - Chip area
  - Total wirelength
  - Delay of critical path
  - Routability



■ Deadspace



A non-optimal floorplan

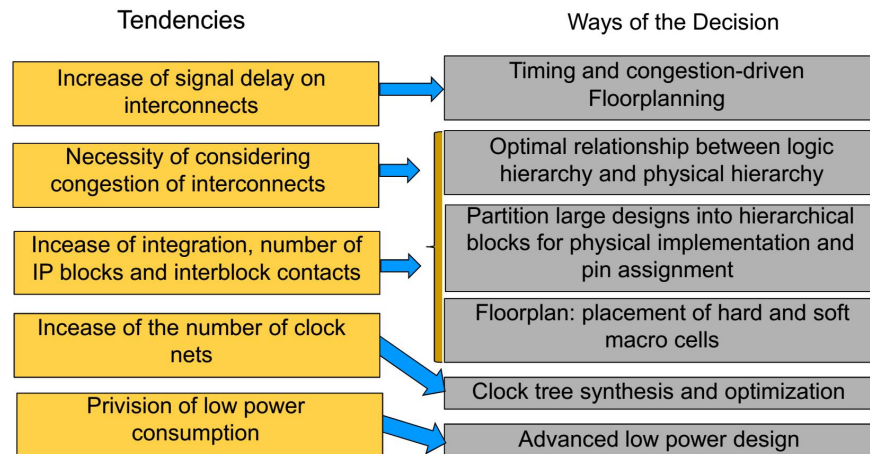


An optimal floorplan in terms of area

94

## BACK-END DESIGN

### New Tendencies of Floorplanning and Ways of the Decision

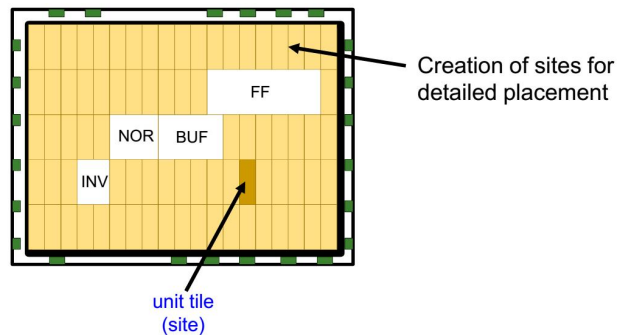


95

## BACK-END DESIGN

### Standard Cell Placement

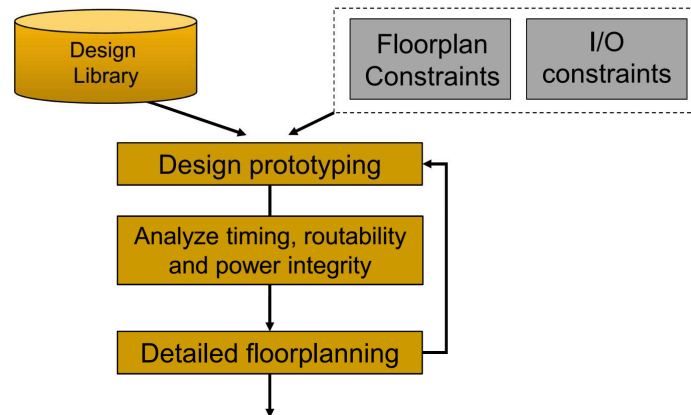
- Placement requires grid in which cells will be placed
- Floorplanning uses 'unit tile' cell to build this grid
  - It is defined by a library developer and library cells are designed to be multiple of unit tile



96

## BACK-END DESIGN

### Design Planning

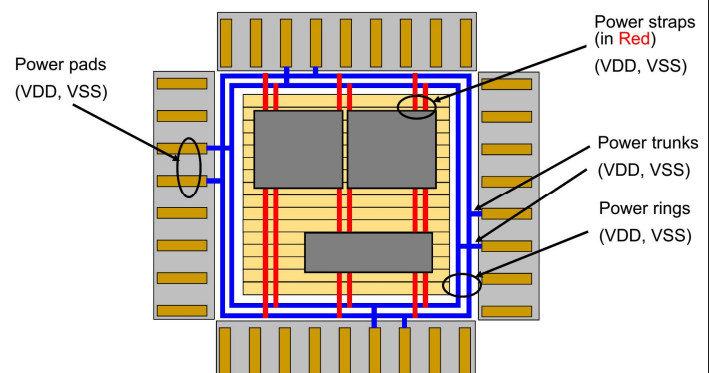


97

## BACK-END DESIGN

### Power Planning

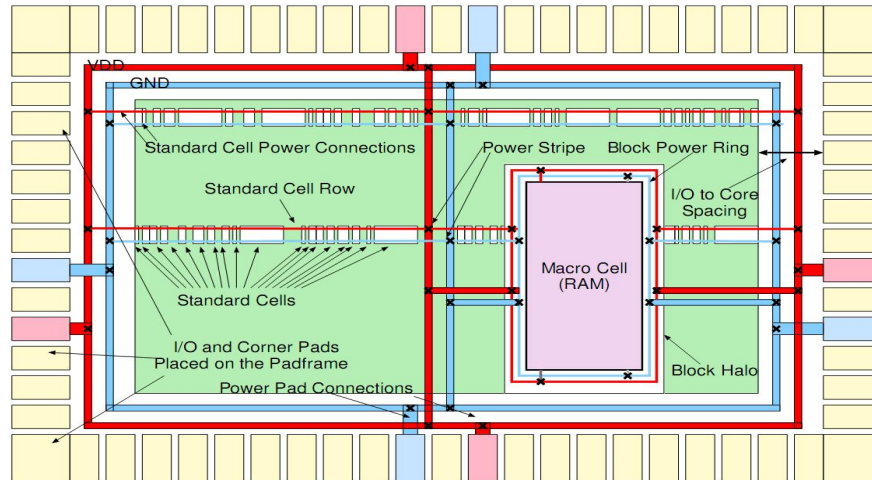
- Steps for Power Planning
  - Determine the number of power pads
  - Determine which metal layers will be used for power routing
  - Define the width of the top-level power bus
  - Determine the structure for block and macrolevel power routing
  - Power Network Analysis (PNA)



98

## BACK-END DESIGN

### Power Planning

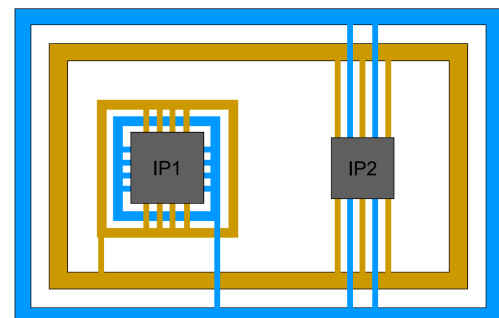
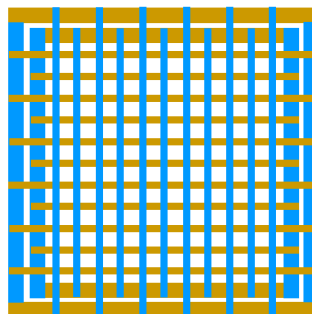


99

## BACK-END DESIGN

### Power Network

- User can specify
  - Number of straps: Min, max
  - Width of straps: Min, Max
  - Width of ring
  - Layers

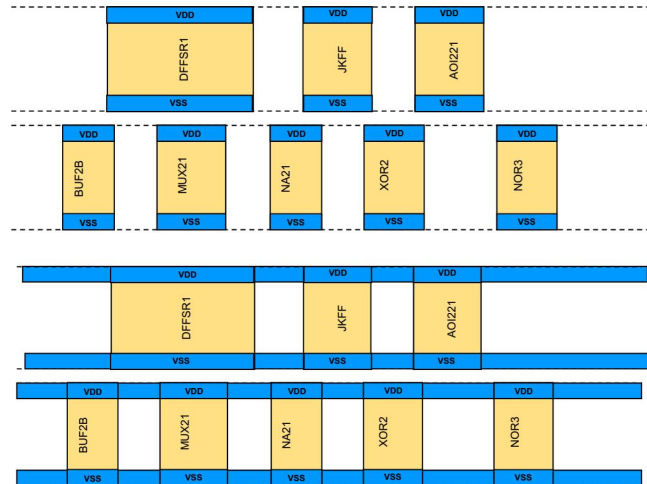


100

## BACK-END DESIGN

### Standard cell PG (Power/Ground) Preroute

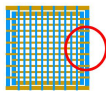
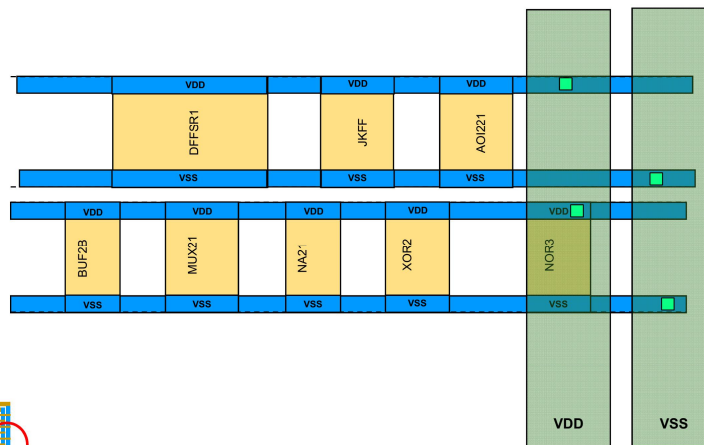
- Check that the PG connections are correct
  - To standard cells
  - To macros
  - To pads
- For macro and pad PGs
  - Can verify that all PG pins are connected
  - Can verify that at least one PG pin is connected
  - Can ignore



101

## BACK-END DESIGN

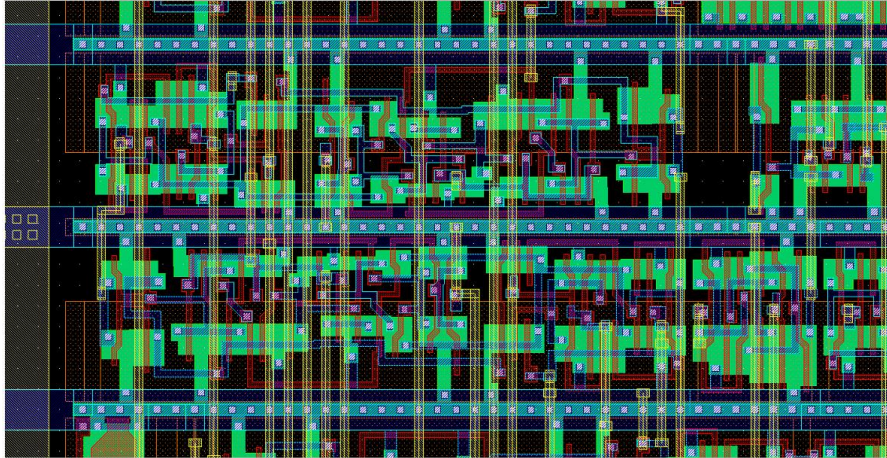
### Committing power plan



102

## BACK-END DESIGN

### Standard Cell Rows

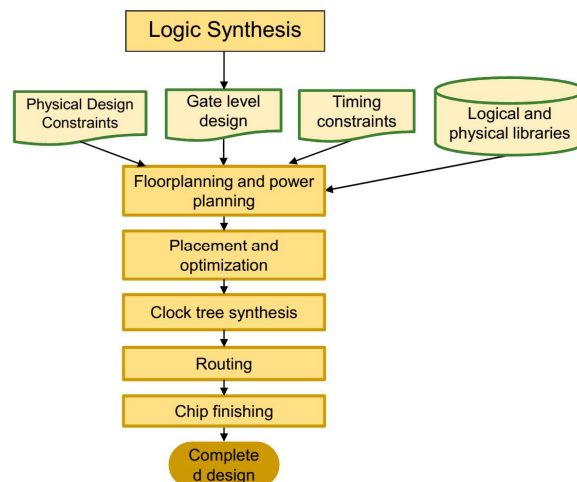


103

## BACK-END DESIGN

### Placement

#### ■ Placement in Digital IC flow

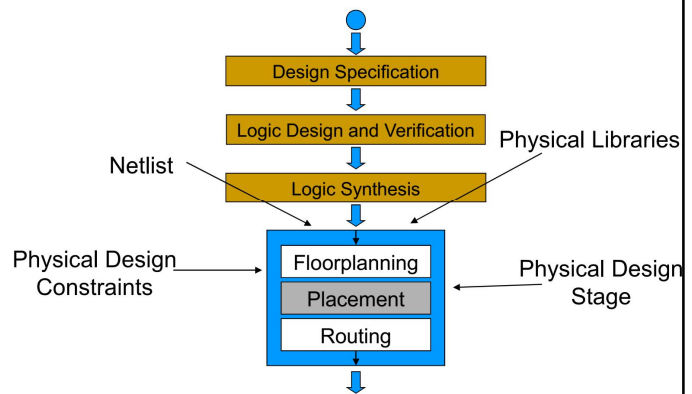


104

## BACK-END DESIGN

### Placement

- The goal of placement is to minimize the total area and interconnect cost
- The quality of the attainable routing is highly determined by the placement
- Circuit placement becomes very critical in 32/28nm and below technologies

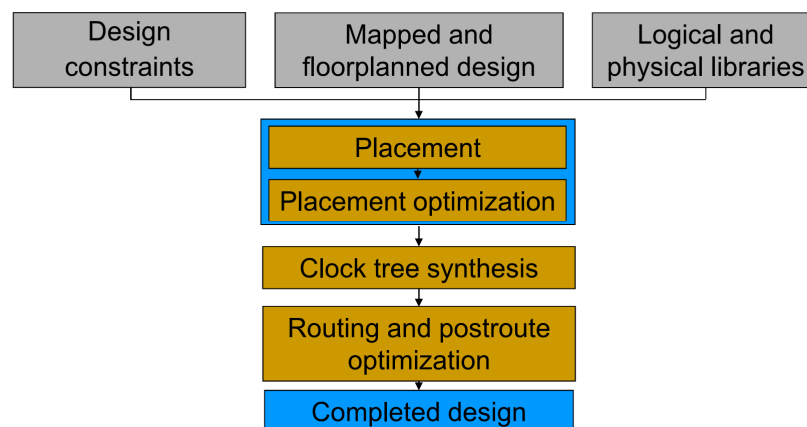


105

## BACK-END DESIGN

### Placement

- Placement in a Typical IC Physical Design Flow

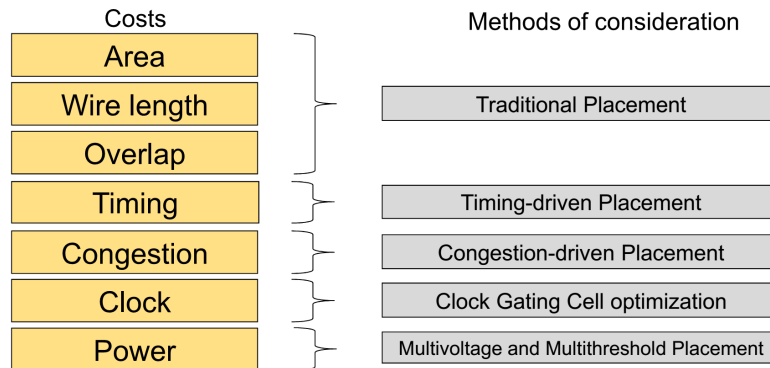


106

## BACK-END DESIGN

### Placement

- New Tendencies of Physical Design and Placement Cost Components

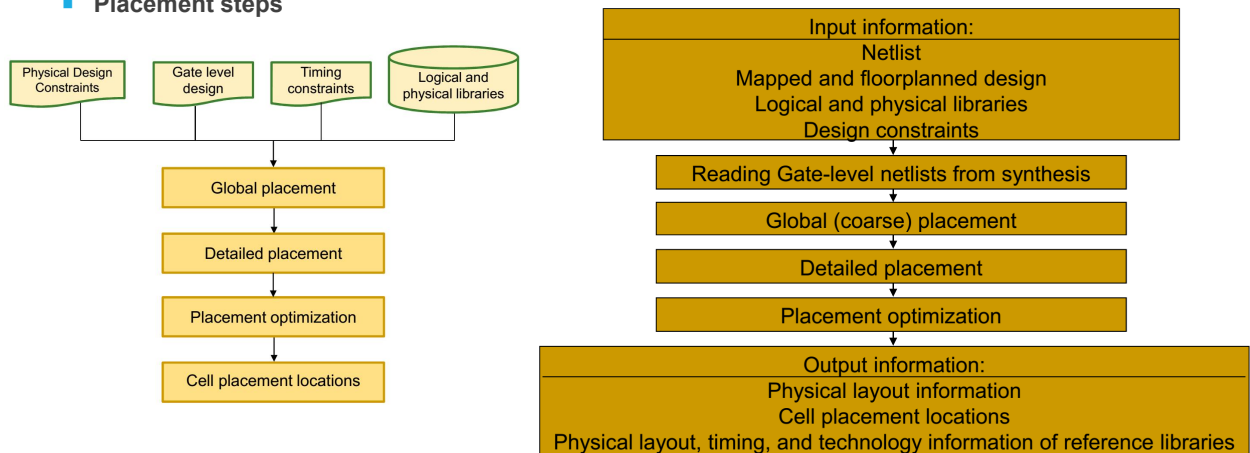


107

## BACK-END DESIGN

### Placement

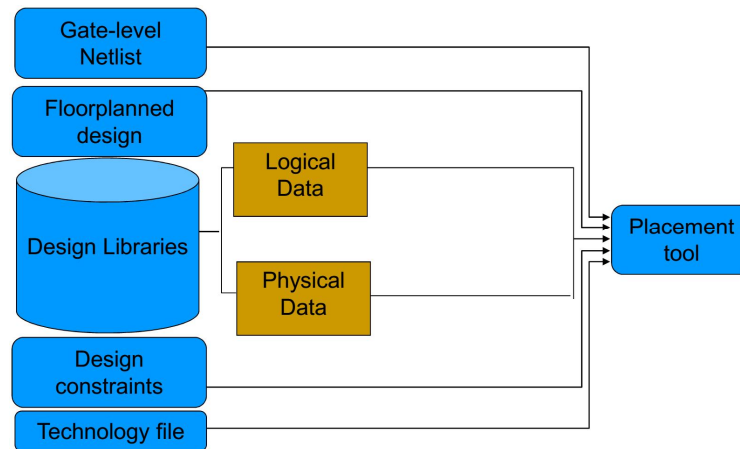
- Placement steps



## BACK-END DESIGN

### Placement

- Input Information for Placement Tool



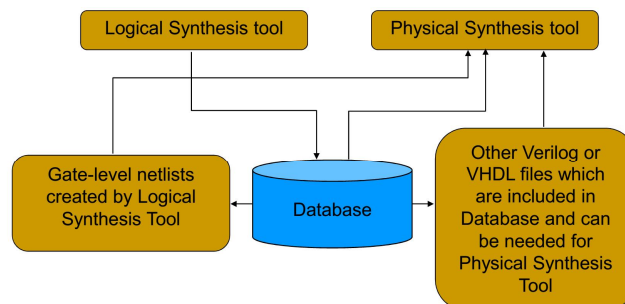
109

## BACK-END DESIGN

### Placement

- Reading Gate-Level Netlists from Synthesis

- Gate-level netlists contain references to standard cells and macros which are stored in the logical libraries as well as in other hierarchical logic blocks



110

## BACK-END DESIGN

### Placement

#### Logical and Target Libraries

##### Logical Libraries

Provide timing and functionality information for all standard cells (AND, OR, flip-flop, ...)

Provide timing information for hard macros (IP, ROM, RAM, ...)

Logical Libraries

##### Target Libraries

Technological library of standard cells, provide information about their characteristics for concrete foundries

Provide information about Level shifters and isolation cells

Target Libraries

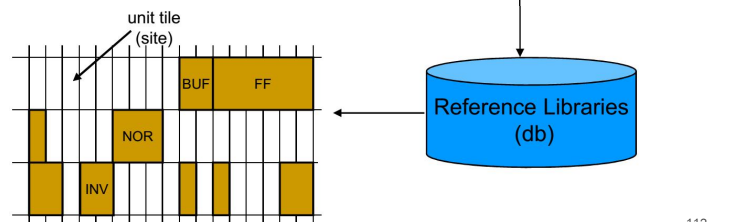
111

## BACK-END DESIGN

### Placement

#### Logical and Target Libraries

- Logic libraries provide timing and functionality information for all standard cells. In addition, logic libraries can provide timing information for hard macros such as RAMs.
- Physical libraries contain physical information of standard and macro cells necessary for placement, define placement unit tile from the length of standard cells.

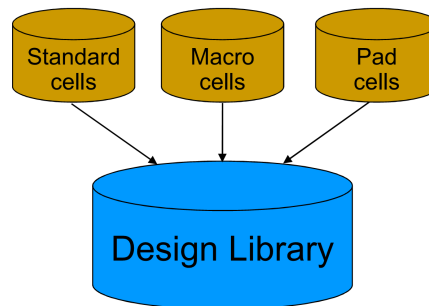


112

## BACK-END DESIGN

### Placement

- **Reference Libraries**
- Contain sub-design or cells used by many other designs
- Referenced by pointers in the design library for memory efficiency



113

## BACK-END DESIGN

### Placement

- **Standard Cell Library**
  - A standard cell is a predesigned layout of one specific basic logic gate
  - Each cell usually has the **same standard height**
  - A standard cell library contains a varied collection of standard cells
  - Libraries are usually supplied by an IC vendor or library group

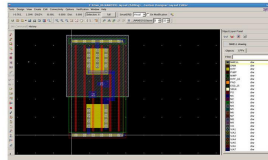
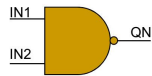
114

## BACK-END DESIGN

### Placement

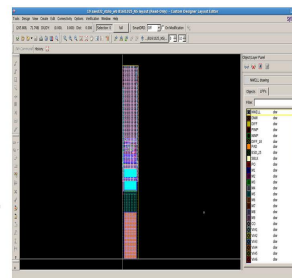
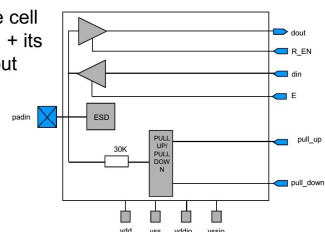
- Digital Standard Cell Library

NAND2  
symbol +  
its layout



- IO Cell Library

Simple cell  
symbol + its  
layout

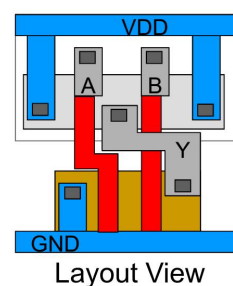
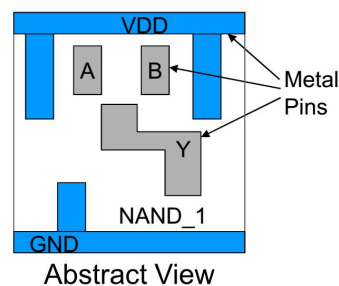


115

## BACK-END DESIGN

### Placement

- Abstract View of Cells in Standard Cell Library
- A standard cell library also contains a corresponding abstract view for each layout view
- Abstract views contain only the minimal data needed for placement

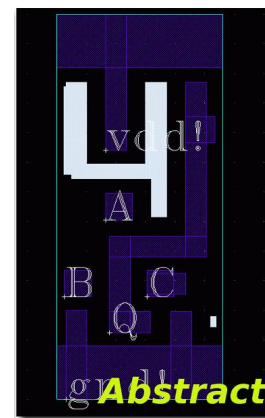
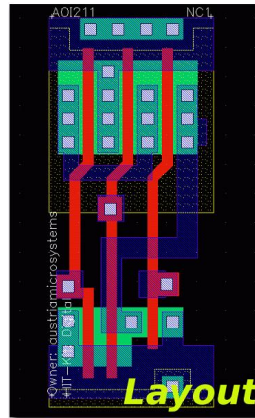
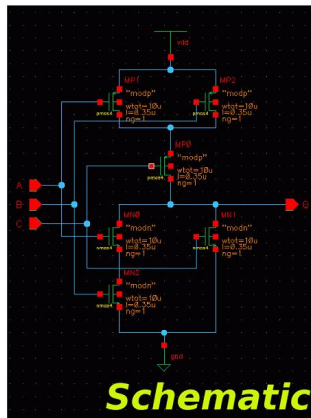


116

## BACK-END DESIGN

### Placement

#### ■ Digital Standard Cell Library



117

## BACK-END DESIGN

### Placement

- Technology File
- Technology File is unique to each process/technology
- Contains metal layer technology parameters
  - Number and name designations for each layer/via
  - Dielectric constant for technology
  - Physical and electrical characteristics of each layer/via
  - Design rules for each layer/Via (Minimum wire widths and wire-to-wire spacing, etc.)
  - Units and precision for electrical units
  - Colors and patterns of layers for display
  - etc.

#### Example of a Technology File

```
Technology {
  dielectric = 3.7
  unitTimeName = "ns"
  timePrecision = 1000
  unitLengthName = "micron"
}

...

Layer "m1" {
  layerNumber = 16
  maskName = "metal1"
  pitch = 0.56
  defaultWidth = 0.23
  minWidth = 0.23
}
```

118

## BACK-END DESIGN

### Placement

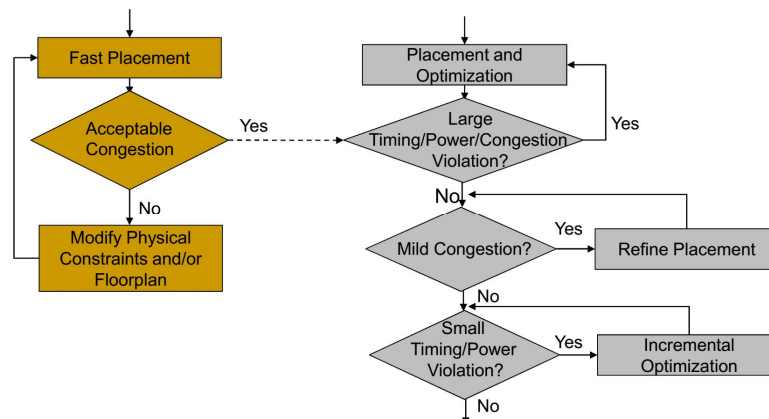
- **Committing the Physical Hierarchy**
- Converts the (virtual) flat design to a two-level hierarchical design
- Removes standard cells and hard macros belonging to a plan group (design) from the top cell and copies them down to the soft macro CEL which is created for the plan group
- Creates nets in the soft macros view based on the Hierarchy Preservation data
- Pushes placement blockages and route guides down to the soft macros views
- (Optional) Pushes power, ground straps and standard cell preroutes down into each soft macros view

119

## BACK-END DESIGN

### Placement

- **IC Compiler Placement Methodology**



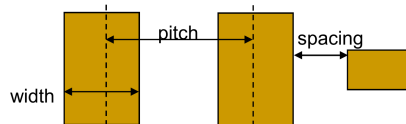
120

## BACK-END DESIGN

### Placement

- Example of Physical Technology Data

```
icc_shell> check_physical_constraints
...
Physical Library: design_lib_orca
Routing layer : METAL width: 160 pitch: 410 space: 180
Routing Layer : METAL Resistance : 6.4e-05 Capacitance : 4.19e-05
Routing layer : METAL2 width: 200 pitch: 410 space: 210
Routing Layer : METAL2 Resistance : 3.7e-05 Capacitance : 2.23e-05
Routing layer : METAL3 width: 200 pitch: 515 space: 210
Routing Layer : METAL3 Resistance : 3.7e-05 Capacitance : 1.39e-05
Routing layer : METAL4 width: 440 pitch: 970 space: 460
Routing Layer : METAL4 Resistance : 1.9e-05 Capacitance : 1.02e-05
...
```



121

## BACK-END DESIGN

### Placement

- Standard Cell Library Components

| File/Database     | Description                                          | Require/Optional |
|-------------------|------------------------------------------------------|------------------|
| Milkyway Techfile | Milkyway techfile for ICC PNR                        | Required         |
| Milkyway Database | Milkyway database with CEL and FRAM                  | Required         |
| Timing .LIB/.DB   | Synopsys timing libraries                            | Required         |
| Antenna Techfile  | For ICC antenna fix                                  | Required         |
| TLUPlus Tables    | For ICC interconnect parasitic estimation            | Required         |
| TLUPlus Mapfile   | For layer mapping between TLUP and MW                | Required         |
| Verilog Models    | For gate level simulations                           | Required         |
| CDL Netlist       | To run LVS                                           | Required         |
| GDS               | GDS, may be needed if no CEL view                    | Required         |
| LEF               | May be used for Milkyway database FRAM re-generation | Optional         |
| Spice LPE Models  | Layout parasitic extracted gate models               | Optional         |

122

## BACK-END DESIGN

### Standard Cell Placement

#### *Which criteria to place standard cells in a floorplan ?*

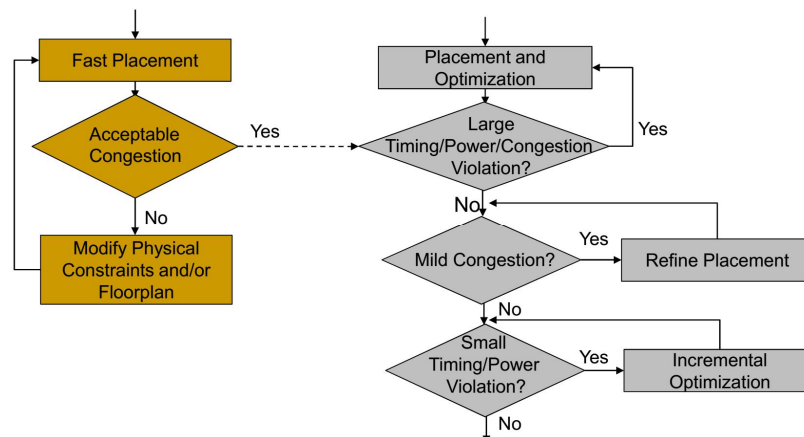
- The shortest critical path
- The longer wire the larger parasitic capacitance → As the shortest wire as possible
- Placement for possible routing
  - Not any placement leading to routing possibility
- The smallest area
  - Routing may lead the larger area.

123

## BACK-END DESIGN

### Placement

#### ▪ IC Compiler Placement Methodology

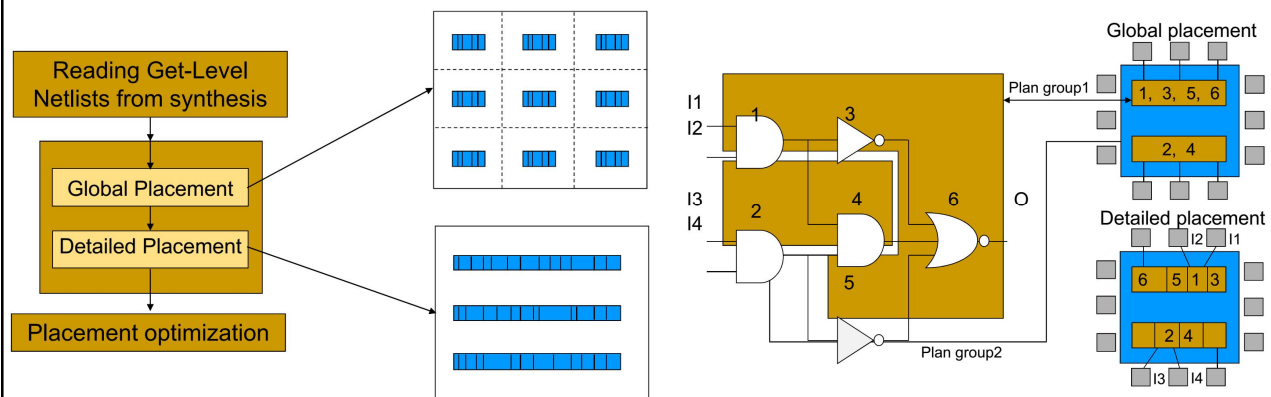


124

## BACK-END DESIGN

### Placement

#### Global and Detailed Placement



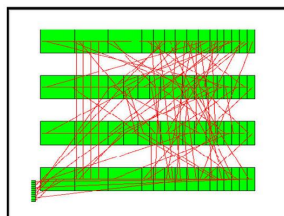
125

## BACK-END DESIGN

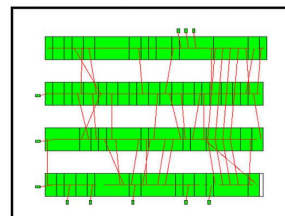
### Placement

#### Global Placement

- Standard cells must be in groups in such a way that the number of connections between groups is minimum
- This issue is solved through circuit partitioning
- As a basic criterion, the minimum is taken among group connections



Bad Placement



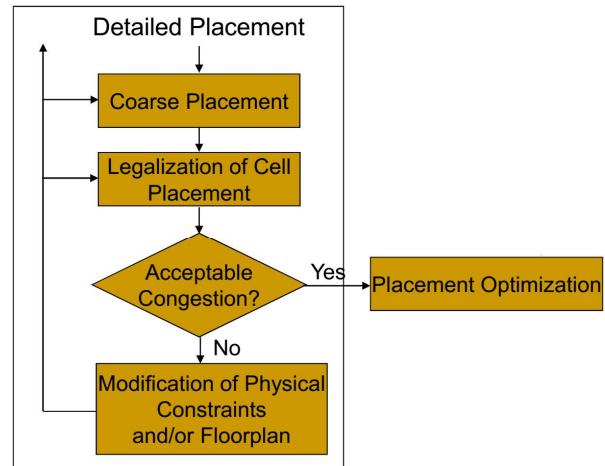
Good Placement

126

## BACK-END DESIGN

### Placement

- **Detailed Placement**
- Detailed placement is solved in two stages:
  - Coarse placement
  - Legalization of cell placement

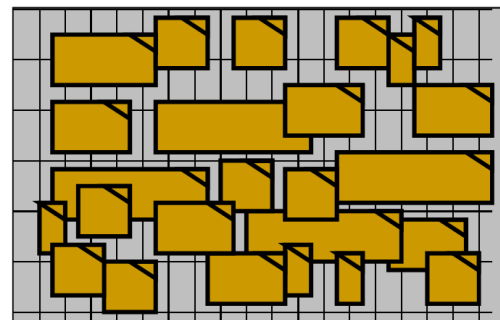


127

## BACK-END DESIGN

### Placement

- **Coarse Placement**
- All the cells are placed in the approximate locations, but they are not legally placed
- No logic optimization is done
- Cells overlap and are not on-grid.
- Large cells (e.g. RAMs) form large placement blockages for other smaller leaf cells.
- Power routing forms routing layer blockages that will also be checked and avoided if specified.

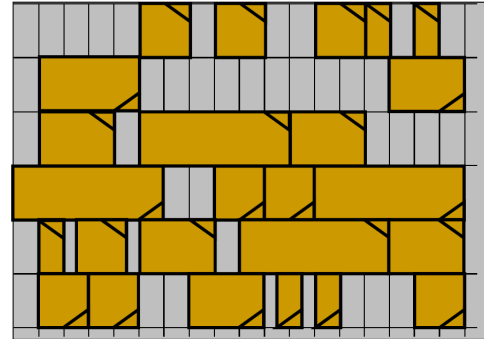


128

## BACK-END DESIGN

### Placement

- **Legalized Cell Placement**
- Legal placement of cells is not required for analyzing routing congestion at an early stage
- Legal placement is done before saving the design
- Cells are not overlapped and are on-grid.



129

## BACK-END DESIGN

### Placement

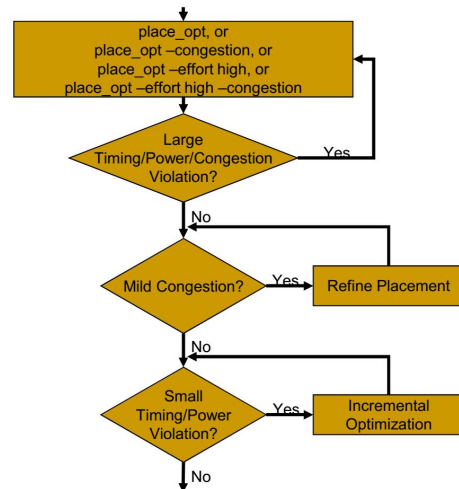
- **Macro Placement Constraints**
- Macro placement constraints have an impact on placing and further global routing of standard cells
- Some basic constraints of macro placement
  - Alignment of macros by edges
  - Grouping macros in a way that for standard cells rectangular area should be left as much as possible
  - Alignment of macros with core boundary
- Some basic parameters of macros placement
  - Routability
  - Timing
  - Wire length
  - Area for standard cells

130

## BACK-END DESIGN

### Placement

- **Placement Optimization**
- Use optimization options
- Congestion-driven placement: to perform congestion driven placement if the design is found to be congested during the exploration phase.
- Timing-driven placement: to perform time driven placement if the design is found to be violated in timing.

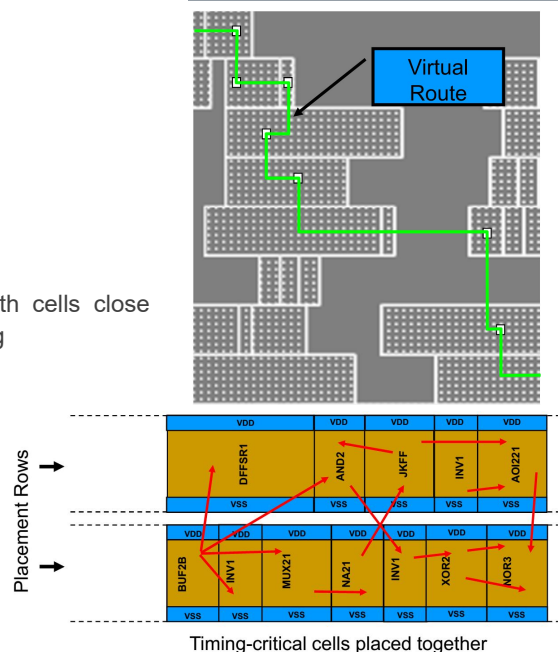


131

## BACK-END DESIGN

### Placement

- **Timing-Driven Placement**
- All steps including placement are timing-driven
- Timing-driven placement tries to place critical path cells close together to reduce net RCs and to meet setup timing
- RC nets are based on Virtual Route (VR) estimates
- Standard cells are placed in “placement rows”
- Cells in a timing-critical path are placed close together to reduce routing-related delays
  - timing-driven placement



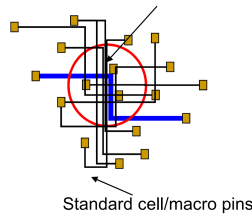
## BACK-END DESIGN

### Placement

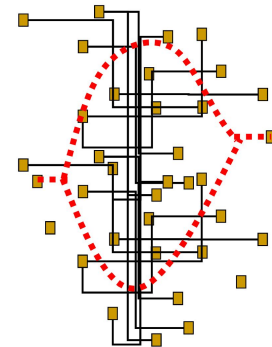
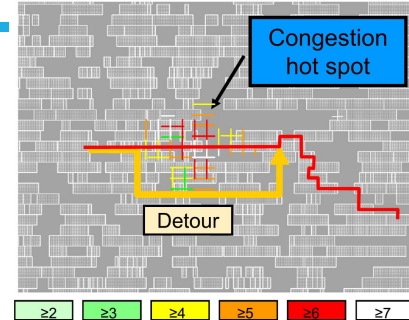
#### ■ Congestion-Driven Placement

- If congestion is not too severe, the actual route can be detoured around the congested area
- The detoured nets will have worse RC delay compared to the VR estimates
- In highly congested areas, delay estimates during placement will be optimistic
- It is important to minimize or eliminate congestion
- Severe congestion can cause a design to be un-routable

Limit to the number of nets that can be routed through this small area



Congestion Map

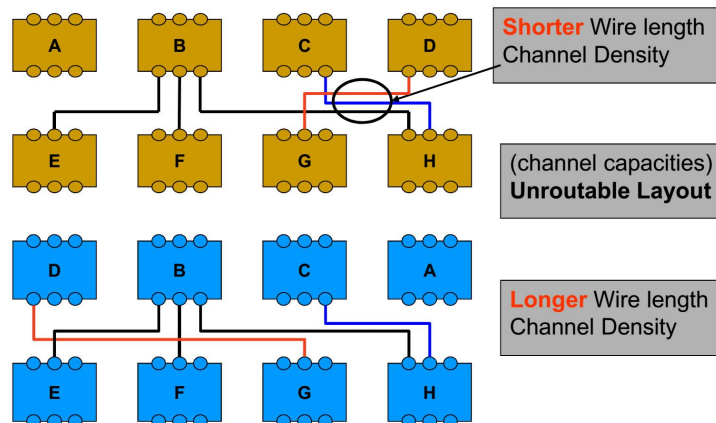


13

## BACK-END DESIGN

### Placement

#### ■ Standard Cells Congestion Placement Optimization

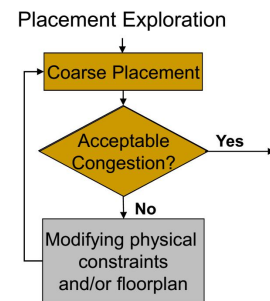


134

## BACK-END DESIGN

### Placement

- **Congestion vs. Timing-Driven Placement**
- Cells along timing critical paths can be spread apart to reduce congestion. These paths may now violate timing.
- Small timing violations can be resolved later by incremental logic optimization.
- **Strategies to Fix Congestion**
- Rerunning coarse placement with congestion-driven option
- Modifying physical constraints
  - Adjusting cell density in congested areas, Adding/modifying blockages
- Modifying the floorplan
  - Moving macros, Changing/increasing core shape/size, Moving pins/pads, etc.

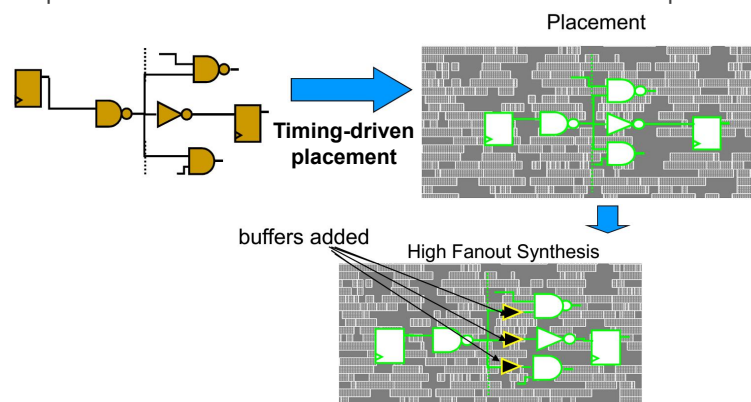


135

## BACK-END DESIGN

### Placement

- **High Fanout Synthesis (HFS)**
- The number of placed cells increases. It leads to the violation of the initial placement.



136

## BACK-END DESIGN

### Placement

- **Power Optimization**
- Good placement can help power reduction of a design.
- Power Optimization Techniques
  - Clock Gating: Clock gate reduces switching activity on the clock tree
  - Leakage Power Optimization
  - Dynamic Power Optimization

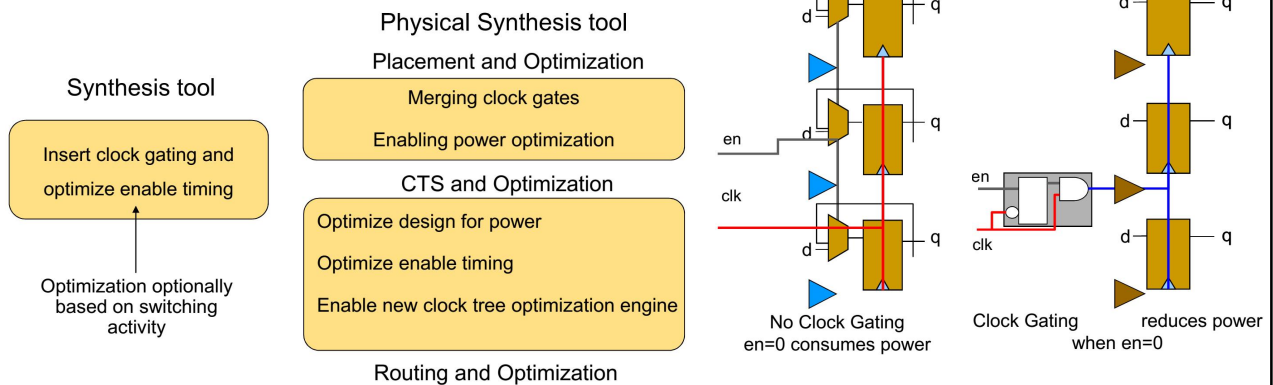
$$P = \underbrace{A n C V^2 f}_{\text{Dynamic Power (AC)}} + \underbrace{n I_L V}_{\text{Static Power (DC)}}$$

137

## BACK-END DESIGN

### Placement

- **Low Power Clock Gating Methodology**

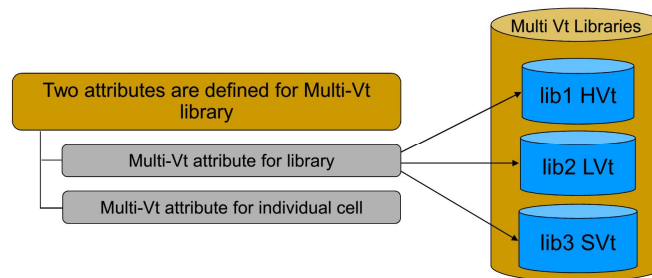


138

## BACK-END DESIGN

### Placement

- Leakage Power Reduction Techniques
- A multi- $V_{th}$  library is the key factor for leakage power optimization (HVT, LVT, SVT)
- Using low  $V_{th}$  cells on critical paths helps to improve timing
- Using high  $V_{th}$  cells on non-critical paths helps to save power

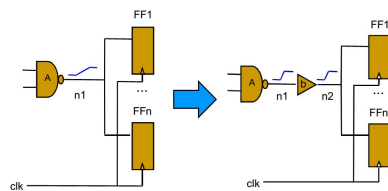


139

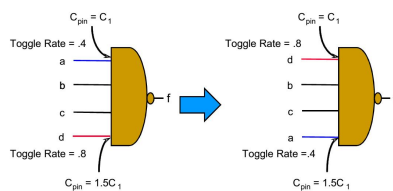
## BACK-END DESIGN

### Placement

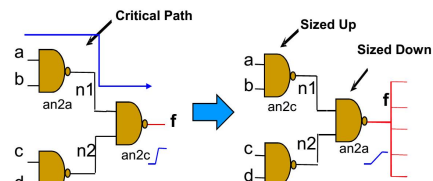
- Dynamic Power Optimization



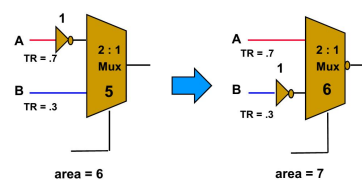
Buffer Insertion



Pin Swapping



Cell Sizing



Phase Assignment

140

## BACK-END DESIGN

### Placement

- **Dynamic Power Optimization**
- Setting up Clock Tree options completely before placement optimization
- Low Power Placement minimizes capacitance between nets by shortening the nets
  - Physical synthesis tool uses clock tree synthesis to determine which cells on each net can be moved closer to one another
- Reduces capacitance between nets with high switching activity by shortening them during placement and optimization
  - Physical synthesis tool performs clock tree estimation to determine the ways to shorten each net by moving its cells closer together

141

## BACK-END DESIGN

### Placement – Impact of Low Power Design Techniques on Placement

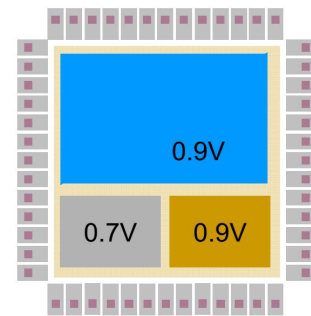
- Clock-gating
  - Turning off portions of the design not in use
  - Decreasing the switching activity
- Multi-Voltage ( $V_{DD}$ ) Optimization
  - Using two or more Vdd's on different blocks
  - Using the higher supply for gates on the critical path
  - Using the lower supply for gates off the critical path
- Multi-Threshold ( $V_{th}$ ) Optimization
  - Using two or more Vth's
  - Using the lower threshold for gates on the critical path
  - Using the higher threshold for gates off the critical path
- Low Power placement
  - Clumping registers closer to reduce clock-tree capacitance
  - Shortening high-activity nets

142

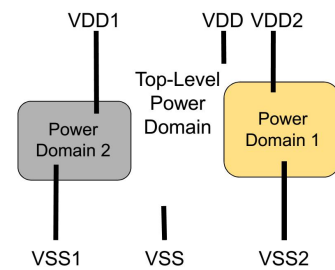
## BACK-END DESIGN

### Placement

- **Multi-voltage and Multi-supply Placements: Using Power Domains**
- Separate parts of the design which operate under certain voltage, can produce power domains. The complete design is called Top-Level power domain
- Voltage area – physical placement area for the cells of a certain power domain.
- Power nets and power ports: multi-voltage supply system for multi-block placement.



Multi-Voltage (MV)

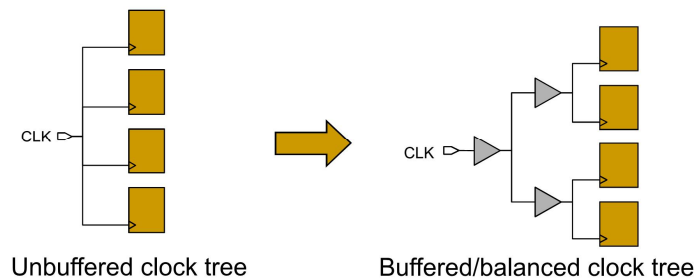


143

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- Clock Tree Synthesis helps balance clock optimization
  - Skew
  - Power
  - Area
  - Slew rates



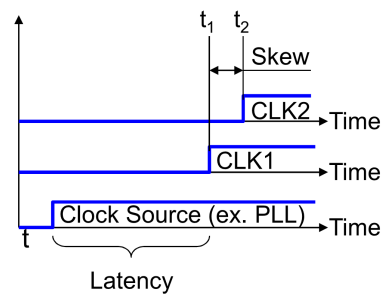
144

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- **Clock Skew: Definition**

- Clock skew is a phenomenon in synchronous digital circuit systems in which the same sourced clock signal arrives at different components at different times. The instantaneous difference between the readings of any two clocks is called their skew.

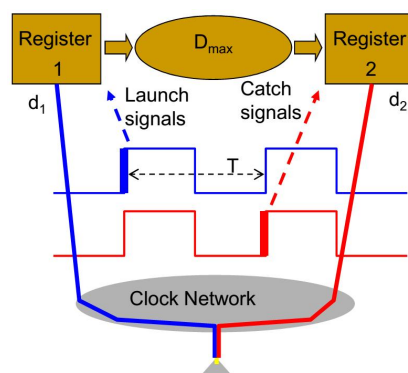


145

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- **Clock Skew: Definition**



$$\text{Skew} = d_1 - d_2$$

$$\text{Zero skew: } d_1 = d_2$$

$$\text{Useful skew, } d_1 - d_2 = \delta_{12}$$

146

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

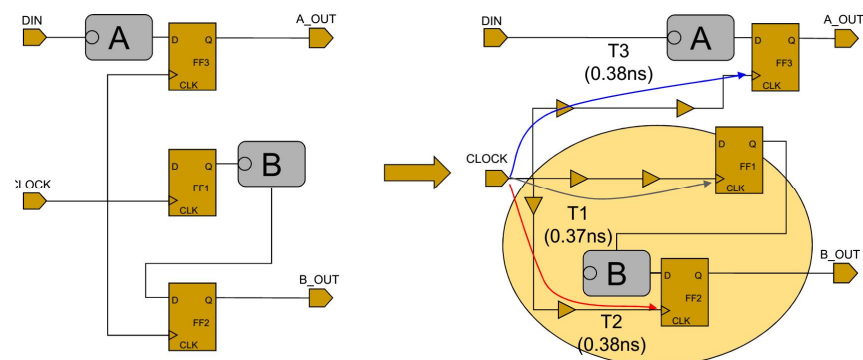
- Global skew
  - Clock difference between any pair of sequential elements of the whole design
  - Global skew is recommended - fastest
- Local skew
  - Same as global but only for registers that have a datapath in between
  - No local skew between 2 registers that have no path between each other
  - Longer runtime
- Useful skew
  - Can be added to the clock of one register in the timing optimization phase to fix hold/setup violations
  - Used to fix small violations where local or global failed

147

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- Clock Skew: Global

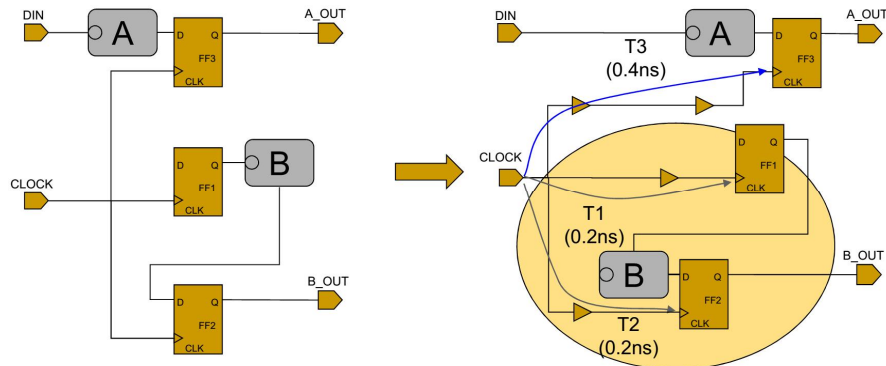


148

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- Clock Skew: Local

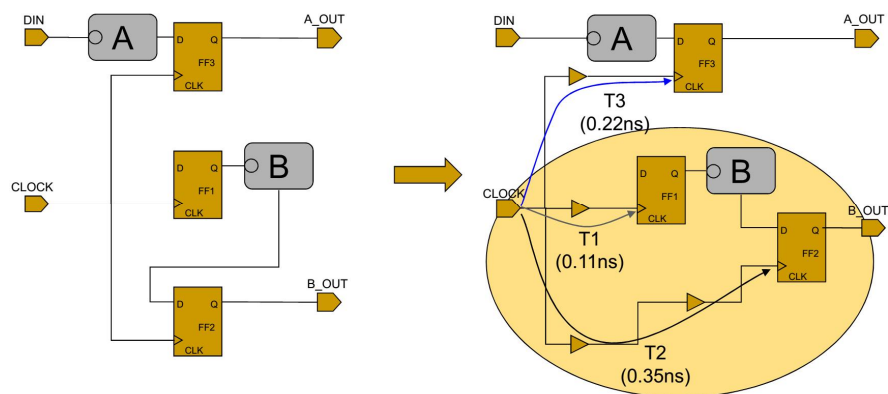


149

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- Clock Skew: Useful

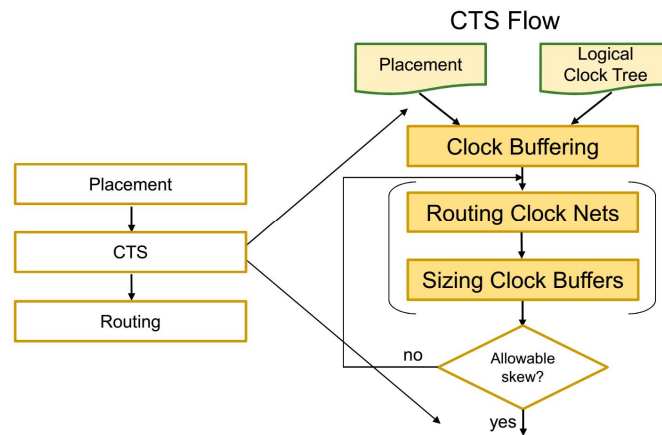


150

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

#### CTS Design Flow and Steps

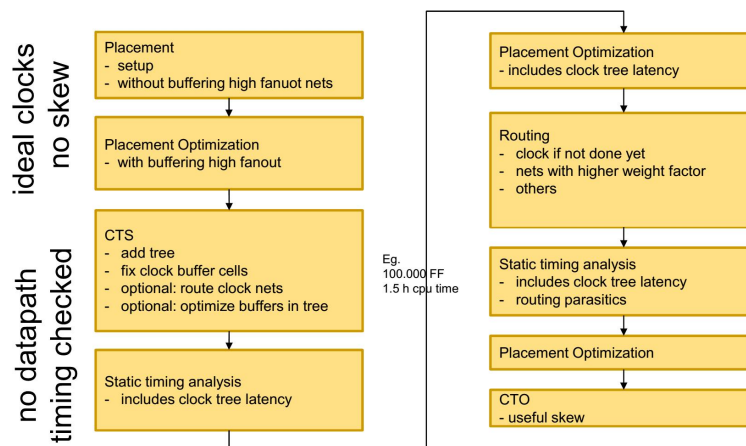


151

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

#### CTS in a real P&R Flow



152

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- **CTS Goals**
- Meeting the clock tree with design rule constraints
  - Maximum transition delay
  - Maximum load capacitance
  - Maximum fanout
  - Maximum buffer levels
- Meeting the clock tree targets
  - Maximum skew
  - Min/Max insertion delay (latency)

153

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- **CTS Prerequisites**
- The design is placed and optimized
  - Placement – completed, Power and ground nets – prerouted
  - Estimated congestion – acceptable
  - Estimated timing – acceptable (small negative slack)
  - Estimated max cap/transition – no violations
  - High fanout nets
    - Reset, Scan Enable synthesized with buffers
    - Clocks are still not buffered
- No large blockages between clock root and its sinks.

154

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

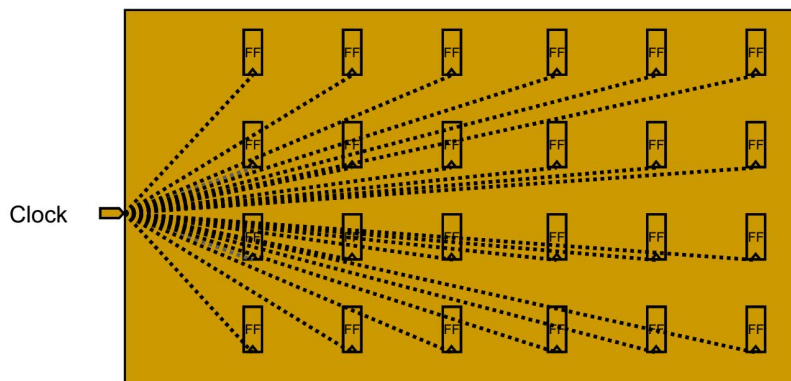
- **Clock Tree Buffer List**
- Buffer List types
  - Clock tree synthesis
  - Sizing
  - Delay insertion
- Buffers in lists
  - Reduced list (less CPU time to make selection)
  - No very large buffers (too much power)
  - No very small buffers (too sensitive for output load)
- Buffer quality
  - Sometimes only inverters for better for power
  - Symmetrical rise and fall times to keep the duty cycle at 50 - 50 %

155

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- **CTS Buffering**

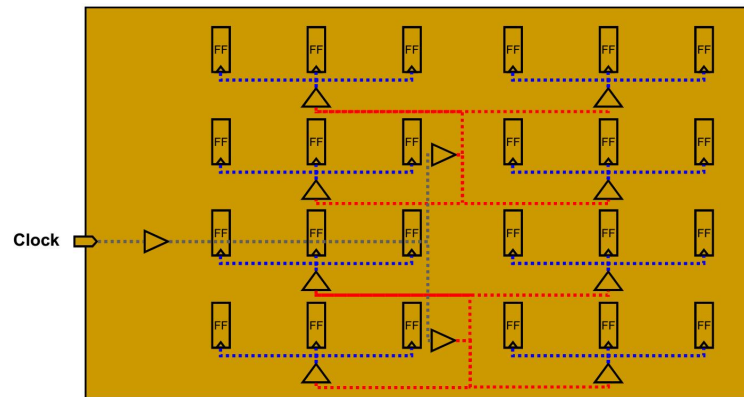


156

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

#### ■ CTS Buffering - Build



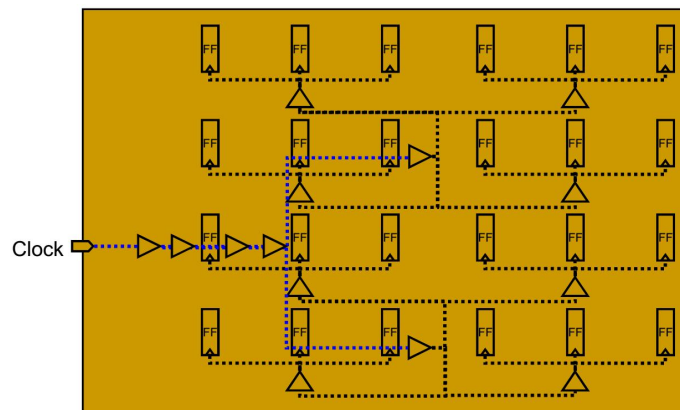
A buffer tree is built to balance the loads and minimize the skew

157

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

#### ■ CTS Buffering - Insertion Delay



A delay line is added to meet the minimum insertion delay

158

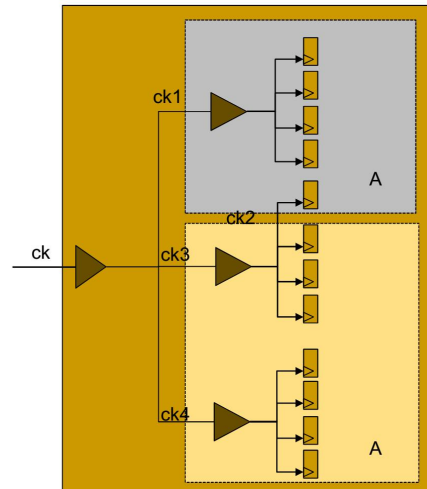
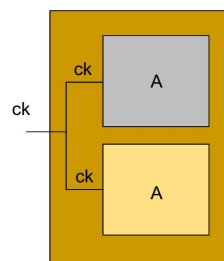
## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- CTS Buffering

#### Logical Hierarchy

CTS always on flat design

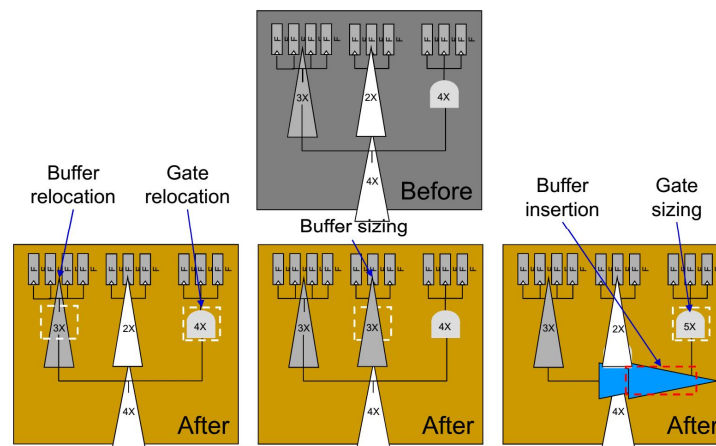


159

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- Clock Tree Optimization (CTO)

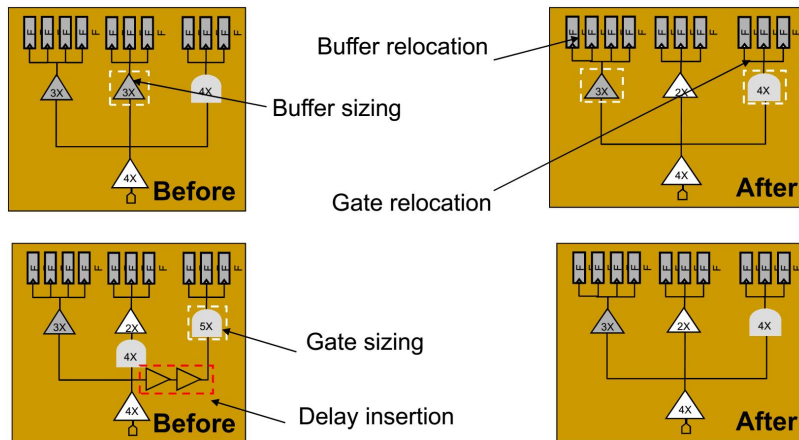


160

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

#### ■ Clock Tree Optimization (CTO)



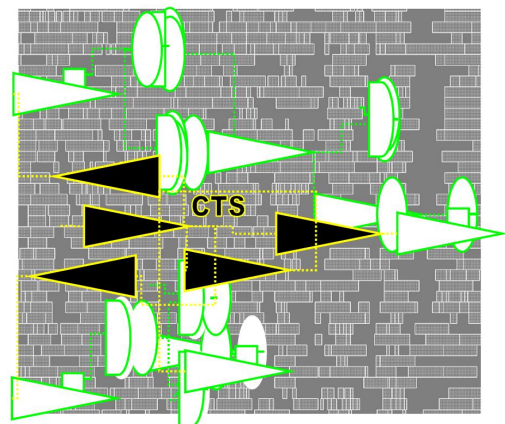
161

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

#### ■ Effects of Clock Tree Synthesis

- Clock buffers added
- Congestion may increase
- Non clock cells may have been moved to less ideal locations
- Inserting clock trees can introduce new timing and max. tran/cap violations

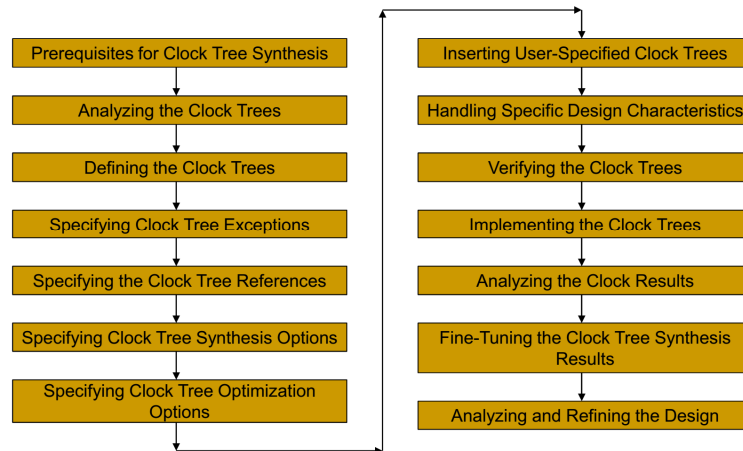


162

## BACK-END DESIGN

### Clock Tree Synthesis (CTS)

- Basic steps IC Compiler CTS



163

## BACK-END DESIGN

### Clock distribution

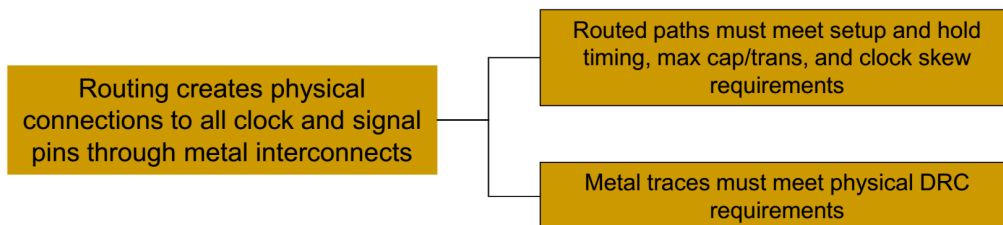
- Clock is the most important signal/wire***
- Digital systems require the same clock at everywhere in the chip at the same time: **skew**
- Clock must be connected to all flip-flops:
  - Very big fanouts
- EDA P&R tools specially insert multi-level of buffers (load gain) and time balance (delay) to make sure all the clock signal length are equal.

164

## BACK-END DESIGN

### Routing

- Routing is the physical implementation of all those interconnects between pins which are connected by an electrical circuit.
- Those interconnects provide both signal and clock/power circuit implementation and meet physical (DRC) and electrical (timing, capacitance/transmission, clock, etc.) requirements.
- Globally interconnects must provide minimal distortions from circuit operation by ideal connection lines to pass the operation through physical interconnects.

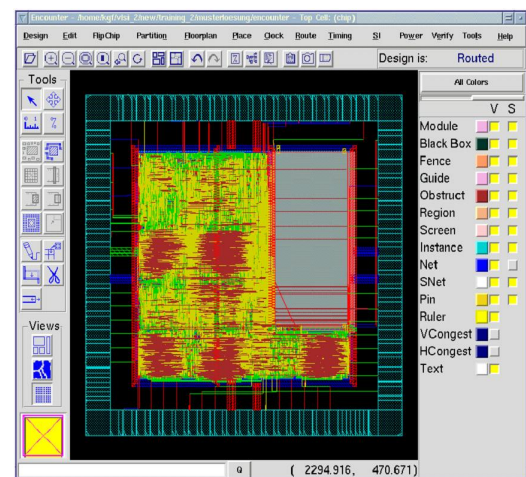


165

## BACK-END DESIGN

### Routing

- *Connect standard cells/blocks/macros together in a layout.*
- Employs wires on different metal layers for routing. The signals on the different layers should not make any intersections.
- Employs vias to connect metal layers → but increase parasitic resistor.
- If the interconnection wires are (too) long,
  - Large parasitic values
  - Large delay value
  - Large power consumption

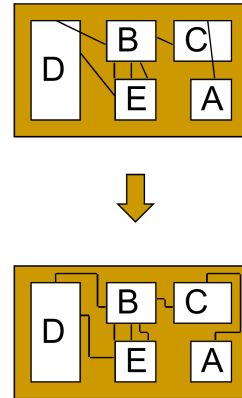


166

## BACK-END DESIGN

### Routing - Routing Goals

- Determination of component wiring
  - Assignment of wires to routing areas
    - Restricted routing problems, e.g. routing box
  - Detailed wiring within areas
    - Layer assignment, Vias
- Goal
  - Minimizing routing area
  - Minimizing wiring length
  - Minimizing channel height
  - Minimizing lengths of critical paths
  - 100% completion in allocated space
  - Minimizing crosstalk



167

## BACK-END DESIGN

### Routing

- **Routing Input and Output**
- Routing is needed after floorplanning/placement and CTS
- Input:
  - Netlist
  - Timing constraints for critical nets
  - Locations of blocks and locations of pins
- Output:
  - Geometric layouts of all nets
- Objective:
  - Minimizing the total wire length, the number of vias
  - Each net meeting its timing constraints

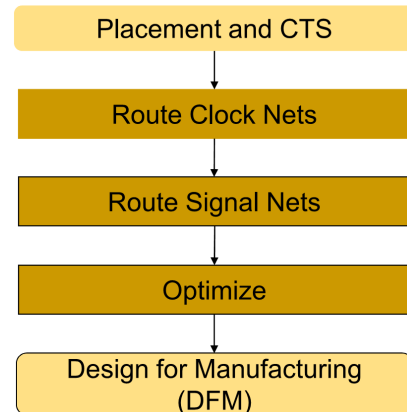
168

## BACK-END DESIGN

### Routing

#### General flow of Routing

- In all the design stages, preceding routing (floorplan, placement and CTS), provision of better conditions has been the most important for further routing. This is also called routability.
- Generally, IC electrical circuits in the sense of functionality are divided into 3 groups – signal nets, clock nets and power nets.
- In the stage of routing, the physical design of clock nets and power nets are also performed.

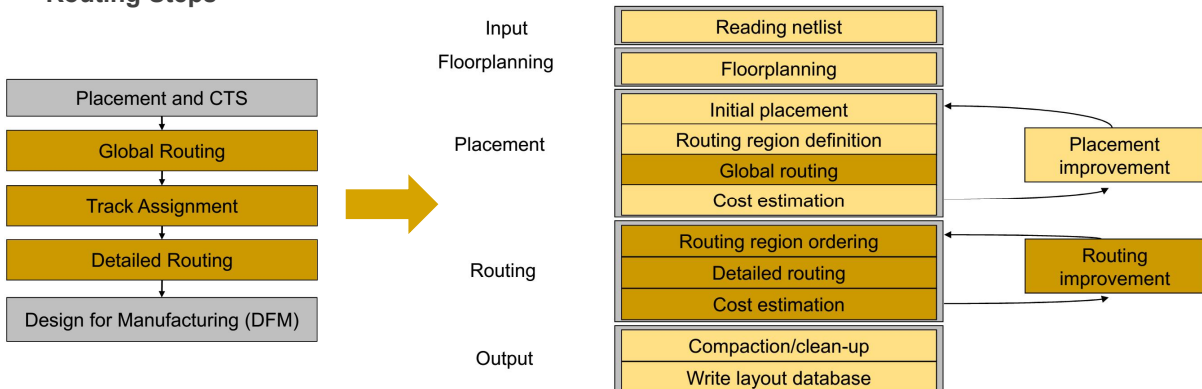


169

## BACK-END DESIGN

### Routing

#### Routing Steps



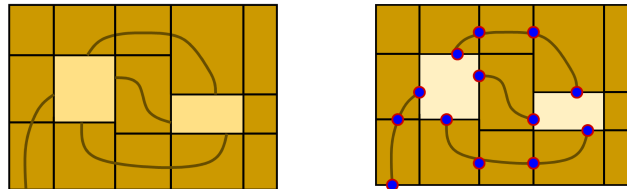
170

## BACK-END DESIGN

### Routing

#### Global Routing

- Region definition: Division of the routing area into routing regions of simple shape (rectangular):
- Region assignment: Assigning routing regions to each net
- Pin assignment to routing regions: Assigning pins on routing region boundaries for each net



- Assigns net to specific metal layers and global routing cells (Gcells)
- Avoid congestion, P/G (power rings/straps/rails), routing blockages

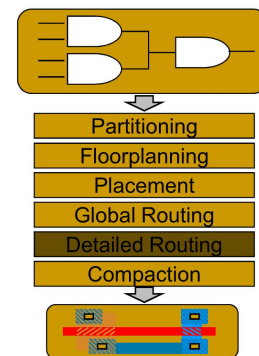
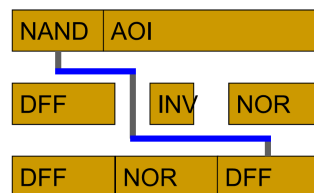
171

## BACK-END DESIGN

### Routing

#### Detailed Routing

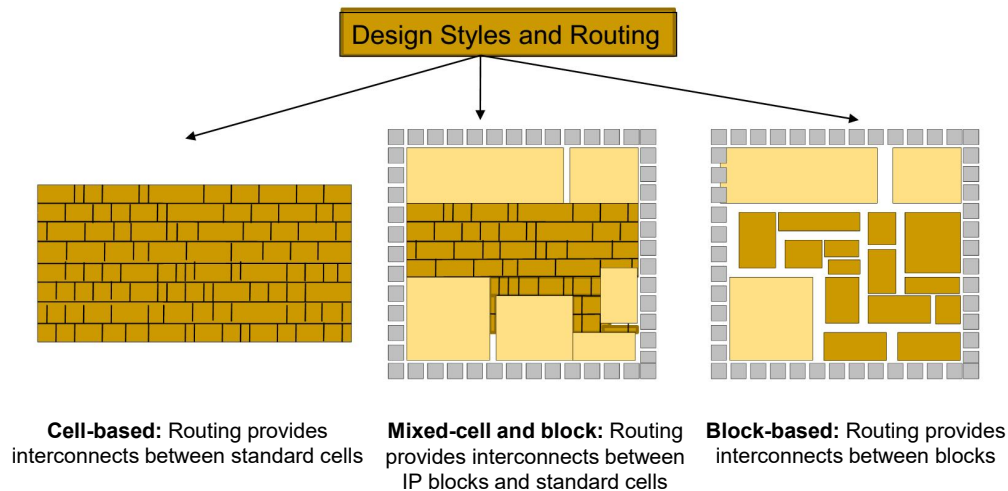
- Performing interconnection between each connected pair of pins in the region, which has been defined with the result of global routing.
- Determining exactly how each signal is routed through each region
- Seeking to reduce routing area
- Routing completion



172

## BACK-END DESIGN

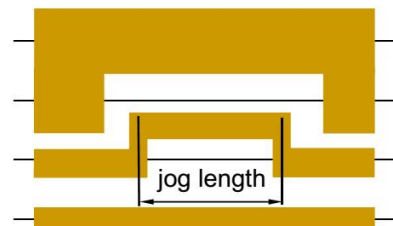
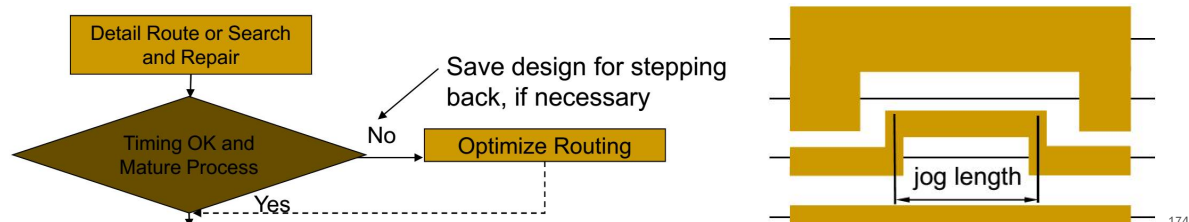
### Routing - Design styles and routing



## BACK-END DESIGN

### Routing

- **Routing optimization**
- Route optimization can be done by reducing wire length and number of vias, and also by removing unnecessary jogs
- If the process is new or unproven then the reduction in via counts and the increase in long straight routes may improve yield



## BACK-END DESIGN

### Routing

#### ■ Post-Routing optimization

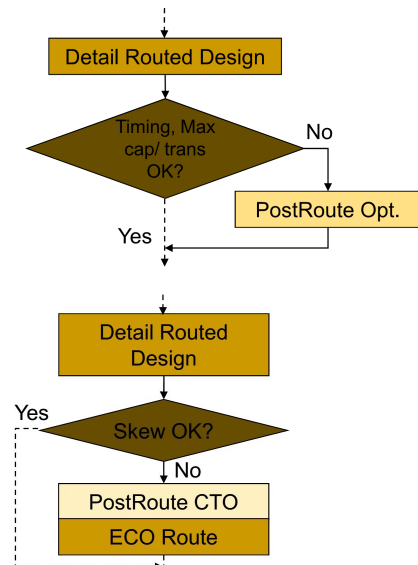
- Performing cell sizing, buffer and inverter insertion
- Powerful hold time fixing
- Topology based optimization

#### ■ Post-Routing timing optimization

- Account for mismatch between pre-route and post-route delays
- Actual net delays are used instead of estimates done at pre-route
- Actual clock tree delays are propagated
- Remaining violations are fixed using standard- $v_{th}$  cells

#### ■ Post-Routing Clock Tree Optimization

- Clock skew may have been disturbed by previous routing and route optimization activity --> improve the skew on clock nets



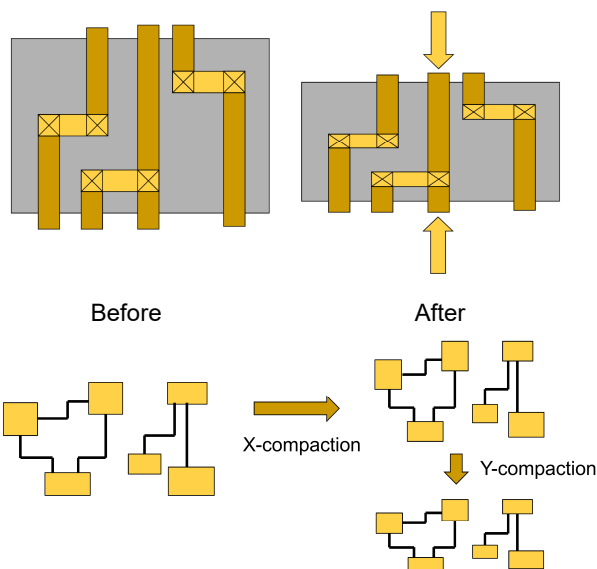
175

## BACK-END DESIGN

### Routing

#### ■ Compaction

#### ■ Channel compaction



176

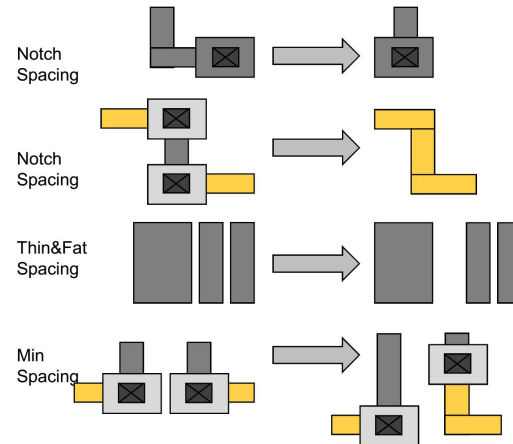
## BACK-END DESIGN

### Routing

- Checking Routing DRC Errors
- Fixing DRC Violations

(Using ICC Tools and Scripts)

Use route\_opt (fixes timing as well):  
route\_opt -incremental

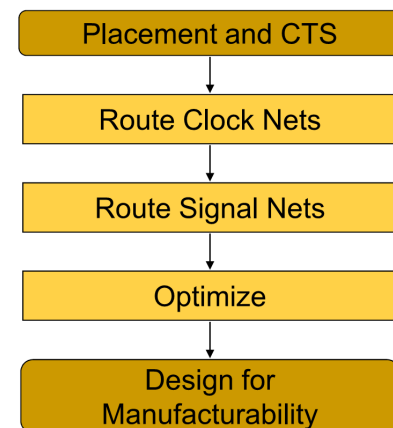


177

## BACK-END DESIGN

### Routing

- General Flow for Routing IC Compiler
- Before performing routing, the design must meet the following conditions:
  - Placement and CTS completed in the previous steps
  - Power and ground nets have been routed after design planning and before placement.
  - Clock tree synthesis and optimization have been performed.
  - Estimated congestion is acceptable.
  - Estimated timing is acceptable (about 0 ns of slack).
  - Estimated maximum capacitance and transition have no violations

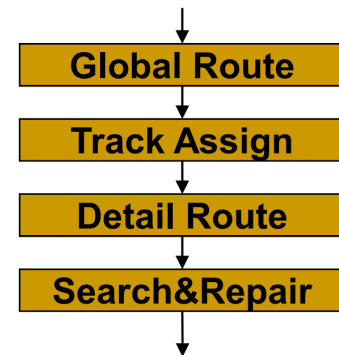


178

## BACK-END DESIGN

### Routing

- **IC Compiler performs:**
  - Global Routing
  - Track Assignment
  - Detail Routing
  - Search and Repair
- After global routing, track assignment and detail routing of all clock/signal nets will be completely routed and should meet all timing and DRC requirements
- Any remaining DRC violations can be fixed by Search & Repair

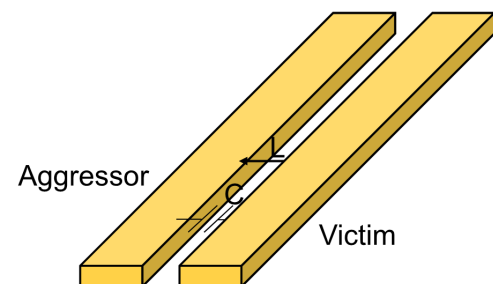


179

## BACK-END DESIGN

### Routing

- **Crosstalk**
- When a signal switches, it may affect the voltage waveform of a neighboring net.
- The switching net is typically identified as the "aggressor" and the affected net is the "victim."
- Often, a net is both an aggressor and a victim



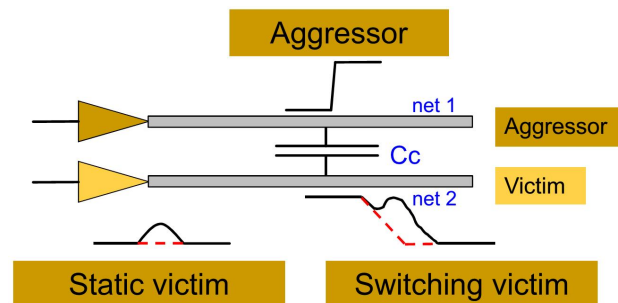
180

## BACK-END DESIGN

### Routing

#### ■ Crosstalk

- Crosstalk is the transfer of a voltage transition from one switching net (aggressor) to another static or switching net (victim) through a coupling capacitance ( $C_c$ )



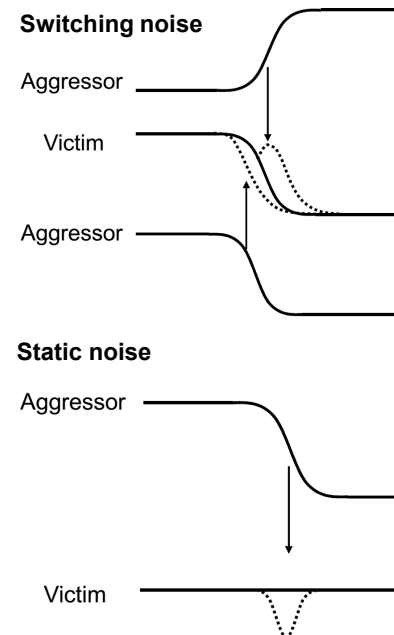
181

## BACK-END DESIGN

### Routing

#### ■ Crosstalk

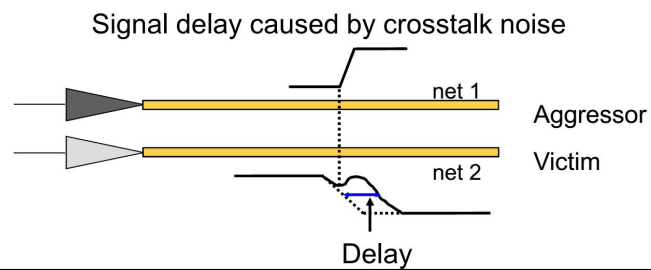
- Crosstalk can cause early and late switching behavior and can also cause timing related logic errors.
- The change on the victim net from its normal path is called **switching noise**.
- Unlike switching noise, static noise occurs when the victim is in a steady state (either high or low) and the aggressor is switching. This causes a glitch on the victim net called **static noise**.



## BACK-END DESIGN

### Routing

- **Crosstalk**
- Crosstalk noise between neighboring signal wires can cause two major problems that affect the operational integrity of IC designs:
  - Delay and Functional Failures
  - Crosstalk delay changes the signal propagation on some of the nets, reducing achievable clock speed



183

## BACK-END DESIGN

### Routing

- **Crosstalk Prevention in IC Compiler**
- During placement and optimization
- During CTS
- During Global Routing and Track Assign
- Crosstalk optimization after routing
- Crosstalk correction performs both cell-based and route-based optimization (i.e., optimizes gate placement, gate logic, and routing traces)

184

## BACK-END DESIGN

### Routing

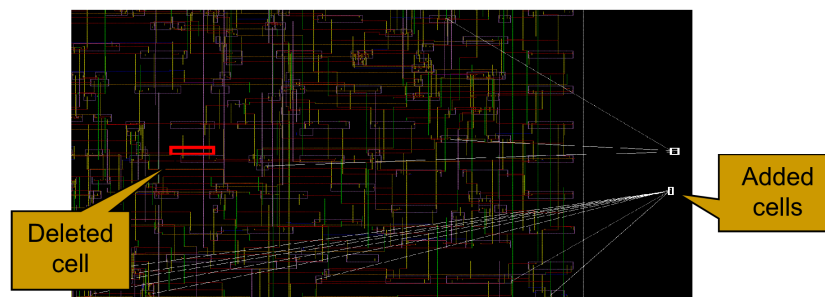
- **Crosstalk and Physical Routing Summary**
- Specify common route options before starting route activity for clocks and signals
- Route clock nets and special net groups first, then route signals for the remainder of the design
- Perform Post Route Optimizations to fix max. cap/trans (buffer insertion, upsizing cells,...) and timing violations (relating to Static Timing Analysis)
- Perform crosstalk correction during the routing flow using logic, placement and routing optimization
- Implement functional ECOs with minimal changes

185

## BACK-END DESIGN

### “ECO” - “Engineering Change Order”

- In the early days of circuit design, if a change has to be made in the design which was already defined or specified, then an “Engineering Change Order” form has to be filled out and signed before making the change.
- Example: Functional changes occur late in the design cycle



186

---

## BACK-END DESIGN

### Chip Finishing and DFM (Design for Manufacturing)

- **Chip Finishing Flow**
- Post-Route: Timing and DRC
- Antenna Fixing
- Wire Spreading
- Redundant Via Insertion
- Filler Cell Insertion
- Metal Fill Insertion
- Metal Slotting

187

---

## BACK-END DESIGN

### Chip Finishing and DFM (Design for Manufacturing)

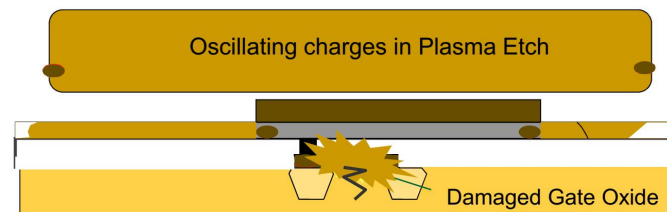
- **Chip Finishing Overview**
- To ensure manufacturability and yield with these processes, tools have partnered with the leading foundries to implement advanced process rules.
- Antenna checking and repair is needed because several high energy steps used during fabrication can cause charge to accumulate on the interconnect.
- The antenna check makes sure that the path exists and corrects it (by jumping to a higher level or inserting a reverse-biased diode).
- Metal filling are used to ensure that metal density rules are met.
- Reliability: The ability to create reliable vias decreases with each process node. If a single via fails, it creates an open and the circuit is useless.
- Signal Electromigration (EM): Because all currents must go through one narrow via, it can eventually cause electromigration creating an open that causes the circuit to fail.
- Via optimization (adding multiple vias) increases the reliability of the circuit, and increases its resistance to signal electromigration

188

## BACK-END DESIGN

### Chip Finishing and DFM (Design for Manufacturing)

- **Antenna Fixing: Gate Oxide Integrity**
- Metal wires (antennas) placed in an EM field generate voltage gradients
- During the metal etch stage, strong EM fields are used to stimulate the plasma etchant
- Resultant voltage gradients at MOSFET gates can damage the thin oxide



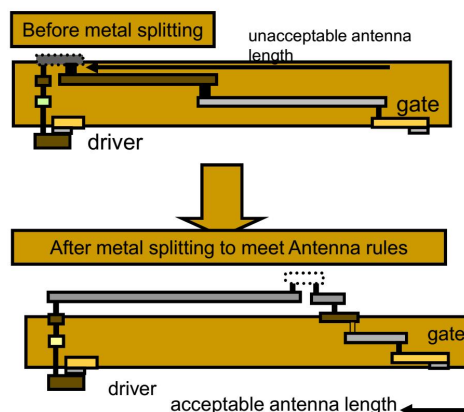
189

## BACK-END DESIGN

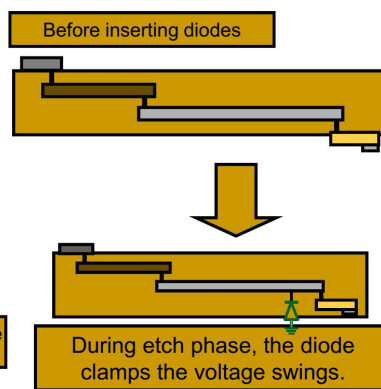
### Chip Finishing and DFM (Design for Manufacturing)

- **Fixing Antenna Problems**

#### Splitting Metal or Layer Jumping



#### Inserting Diodes

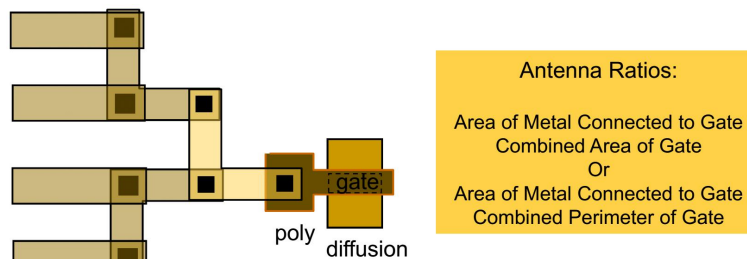


190

## BACK-END DESIGN

### Chip Finishing and DFM (Design for Manufacturing)

- **Antenna Rules**
- As total area (length) of wire increases during processing, the voltage stressing the gate oxide increases → may destroy the transistors
- Antenna rules define acceptable total area of wires

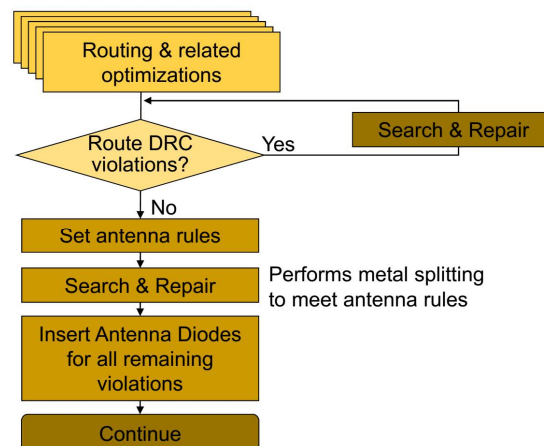


191

## BACK-END DESIGN

### Chip Finishing and DFM (Design for Manufacturing)

- **Antenna Flow**

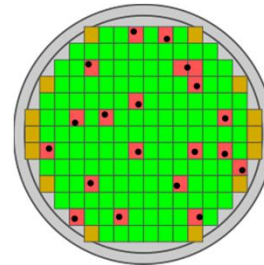


192

## BACK-END DESIGN

### Chip Finishing and DFM (Design for Manufacturing)

- **Filler Cell Insertion**
- Metal fill insertion helps to
  - Fill layers of choice including poly → fixing density errors
  - Insert floating vias, connect continuous P/G rows
  - Do area based metal fill
  - Specify required width (timing driven metal fill doesn't fill wire tracks around critical nets)
- For better yield, density of the chip needs to be uniform
- Some placement sites remain empty on some rows
  - P&R tools can fill such empty sites with standard cells/filler cells



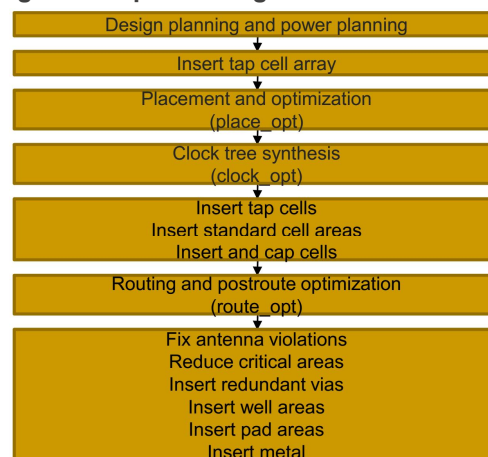
- - defect
- - defective die
- - good die
- - partial edge die

193

## BACK-END DESIGN

### Chip Finishing and DFM (Design for Manufacturing)

- **Design For Manufacturing and Chip Finishing Tasks in The Design Flow IC Compiler**

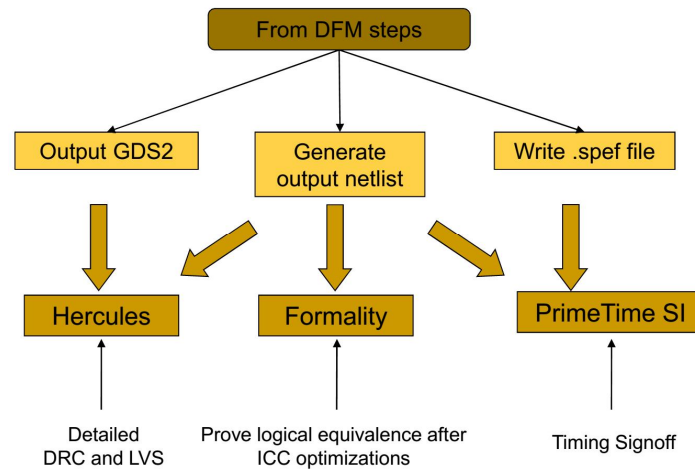


194

## BACK-END DESIGN

### Chip Finishing and DFM (Design for Manufacturing)

#### Final Validation



195

## BACK-END DESIGN

### Chip Finishing and DFM (Design for Manufacturing)

#### Final Validation

- Parasitics: Wire parasitics for PrimeTime are provided via .SPEF or .SBPF file. Use StarRC extraction for signoff
- Netlists for STA (Static Timing Analysis) do not require output of “physical only cells” like:
  - Corner pad cells
  - Pad/core filler cells
  - Unconnected cell instances
- Unconnected cell instances (e.g., spare cells) are needed for LVS
- GDS2 Output: The GDSII for external physical verification can be generated using IC Compiler

196

## BACK-END DESIGN

### Extraction

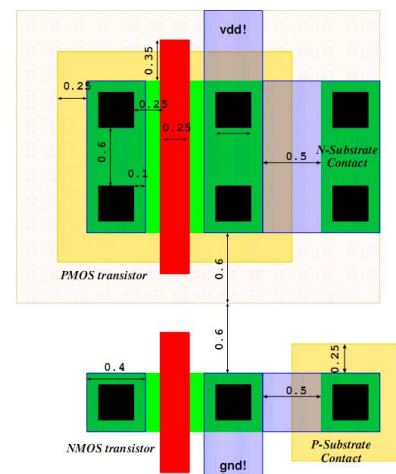
- After P&R is finished, every parameters on interconnection wires can be known by an important step called parasitic extraction. They can be:
  - Parasitic capacitors on wire (Wire capacitance)
  - Parasitic capacitors on 2-wire (Wire to wire capacitance)
  - Parasitic resistors on wire or via (Wire – via resistance)
- All these parasitic extraction parameters can be exactly calculated based on values of SDF, SPEF files.
- These parasitic extraction values can be used for post layout simulation.
- **Parasitic extraction values can affect to delay and timing.**
- **Optimization:** add buffers on long interconnection wires, select better standard cells, change placement methods, re-synthesize critical paths.

197

## BACK-END DESIGN

### Layout verification

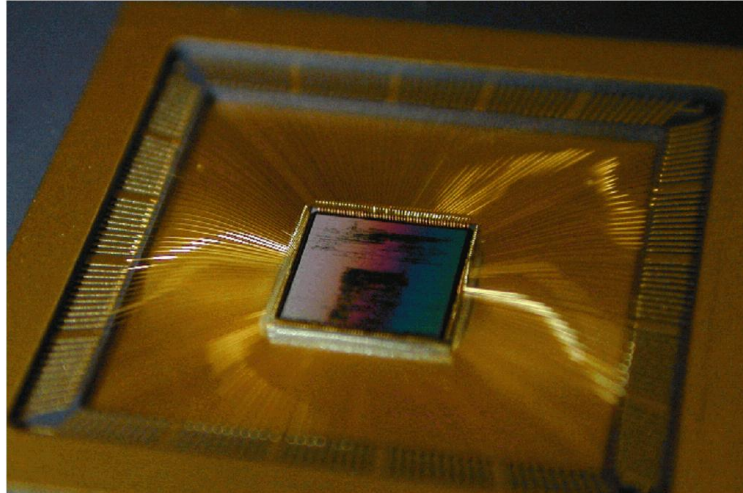
- Design rule check (DRC): All physical layers must follow design rules:
  - Minimum spacing
  - Minimum width
  - Minimum coverage
  - Minimum area
- Layout versus Schematic (LVS):
  - Layout only consists of physical geometry
  - Parameters of transistors and interconnection wires are extracted → Extracted netlist after layout
  - **Extracted layout netlist from GDSII** is compared to **Verilog netlist with P/G from layout** → CORRECT or not



198

## BACK-END DESIGN

- Tape-out



199

## BACK-END DESIGN

### Example of Synthesis scripts

```
#Thiet lap cac duong dan
set LinkProject "<home>/LAB3_DC/THUC_HANH"
set LinkLibrary_db "<home>/Lib/syn_lib/syn_lib"
set LinkSource "${LinkProject}/src"
set LinkReport "${LinkProject}/dc/dc_report"
set LinkNetlist "${LinkProject}/netlist"
#Thiet lap thu vien cong nghe
define_design_lib WORK -path "work"
set target_library [list ${LinkLibrary_db}/saed90nm_min.db \
    ${LinkLibrary_db}/saed90nm_typ.db \
    ${LinkLibrary_db}/saed90nm_max.db]
set link_library [list * ${LinkLibrary_db}/saed90nm_min.db \
    ${LinkLibrary_db}/saed90nm_typ.db \
    ${LinkLibrary_db}/saed90nm_max.db]
#Doc cac file verilog
read_verilog -rtl ${LinkSource}/counter.v
#Kiem tra neu thiet ke phu hop
uniquify
check_design > ${LinkReport}/synth_check_design.rpt
#set enviroment
```

```
set_min_library "${LinkLibrary_db}/saed90nm_max.db" -min_version
"${LinkLibrary_db}/saed90nm_min.db"
set operating_conditions -min BEST -max WORST
# Constraint cua Counter
current_design counter
set design "counter"
set time_scale 20
set tran [expr {0.05*$time_scale}]
set delay_in [expr {0.7*$time_scale}]
set delay_out [expr {0.7*$time_scale}]
create_clock -period 20 -waveform {0 10} {iClk}
# Define design constraints
set clock_uncertainty $tran iClk
set clock_latency $tran iClk
set clock_transition $tran iClk
set_input_delay $delay_in [all_inputs] -clock iClk
set_output_delay $delay_out [all_outputs] -clock iClk
# Compilation
compile -area_effort medium -map_effort medium
# Report
report_area > ${LinkReport}/${design}_synth_area.rpt
report_cell > ${LinkReport}/${design}_synth_cells.rpt
report_qor > ${LinkReport}/${design}_synth_qor.rpt
report_resources > ${LinkReport}/${design}_synth_resources.rpt
write_sdc ${LinkNetlist}/${design}_SDC.sdc
## dump out the synthesis
#write -f ddc -hierarchy -output ${LinkNetlist}/${design}_NL.ddc
write -hierarchy -format verilog -output ${LinkNetlist}/${design}_NL.v
```

200

## BACK-END DESIGN

### Example of P&R scripts

```
#Thiet lap cac duong dan den cac thu vien cong nghe
set Techfile "${LinkLibrary_mk}/astro/tech/astroTechFile.tf"
set Ref_lib "${LinkLibrary_mk}/astro/fram/saed90nm_fr"
set Tlupmax
"${LinkLibrary_mk}/star_rcxt/tluplus/saed90nm_1p9m_1t_Cmax.tluplus"
set Tlupmin
"${LinkLibrary_mk}/star_rcxt/tluplus/saed90nm_1p9m_1t_Cmin.tluplus"
set Tech2itf "${LinkLibrary_mk}/astro/tech/tech2itf.map"
set target_library [list ${LinkLibrary_db}/saed90nm_min.db /
                        ${LinkLibrary_db}/saed90nm_typ.db /
                        ${LinkLibrary_db}/saed90nm_max.db]
set link_library [list * ${LinkLibrary_db}/saed90nm_max.db]

#Tien hanh CTS
clock_opt -effort high
clock_opt -no_clock_route -only_psyn -fix_hold_all_clocks -only_hold_time

initialize_floorplan \
    -core_utilization $Core_util \
    -left_io2core $Core_space \
    -bottom_io2core $Core_space \
    -right_io2core $Core_space \
    -top_io2core $Core_space

#Tien hanh Placement
place_opt -effort high
place_opt -congestion

#Tien hanh Routing
route_opt
```

201

## STATIC TIME ANALYSIS (STA)

### Definition

- Static Timing Analysis (STA) is a method of validating the timing performance of a design by checking all possible paths for timing violations under worst case conditions.
- It considers the worst possible delay through each logic element, but not logical operation of the circuit.
- To perform STA, there are 3 main steps:
  - Design is broken down into sets of timing paths
  - Calculates the signal propagation delay along each path
  - Checks for violations of timing constraints inside the design and at the input/output interface.

202

---

## STATIC TIME ANALYSIS (STA)

### Timing paths

- Types of paths for timing analysis
  - Data Path
  - Clock Path
  - Clock Gating Path
  - Asynchronous Path
- Each timing path has a “Start point” and an “End point”

203

---

## STATIC TIME ANALYSIS (STA)

### Timing paths

- Data path
  - Start point
    - Input port of the design (because the input data can be launched from some external source)
    - Clock pin of the Flip-flop/latch/memory (sequential cell)
  - End point
    - Data input pin of the Flip-flop/latch/memory (sequential cell)
    - Output port of the design (because the input data can be launched from some external source)
- Clock path
  - Start point
    - Clock input port
  - End point
    - Clock pin of the Flip-flop/latch/memory (sequential cell)

204

## STATIC TIME ANALYSIS (STA)

### Timing paths

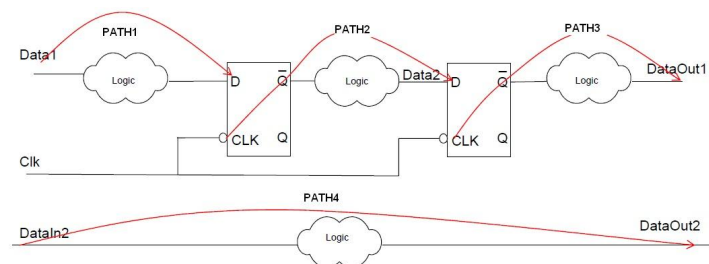
- Clock gating path
  - Start point
    - Input port of the design
  - End point
    - Input port of clock gating element
- Asynchronous path
  - Start point
    - Input port of the design
  - End point
    - Set/Reset/Clear pin of the Flip-flop/latch/memory (sequential cell)

205

## STATIC TIME ANALYSIS (STA)

### Data paths

- Input pin/port to Register (Flip-flop)
- Input pin/port to Output pin/port
- Register (Flip-flop) to Register (Flip-flop)
- Register (Flip-flop) to Output pin/port
- Example:

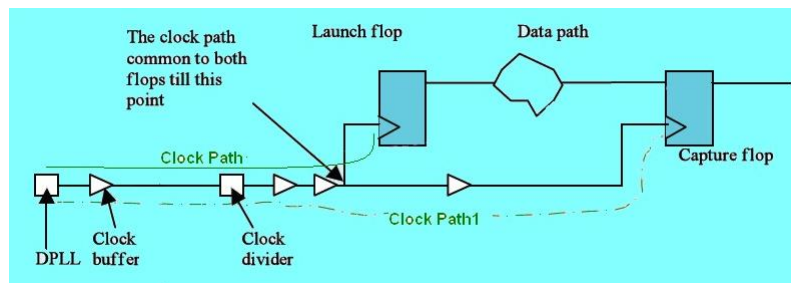


- PATH1: starts at an input port and ends at the data input of a sequential element (Input port to Register)
- PATH2: starts at the clock pin of a sequential element and ends at the data input of a sequential element (Register to Register)
- PATH3: starts at the clock pin of a sequential element and ends at an output port (Register to Output port)
- PATH4: starts at an input port and ends at an output port (Input port to Output port)

206

## STATIC TIME ANALYSIS (STA)

### Clock path

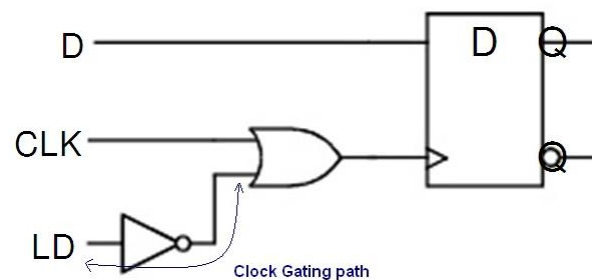


- Clock path starts from the input port/pin of the design which is specific for the Clock input and the end point is the clock pin of a sequential element. In between the start point and the end point there may be lots of Buffers/Inverters/Clock divider.

207

## STATIC TIME ANALYSIS (STA)

### Clock gating path

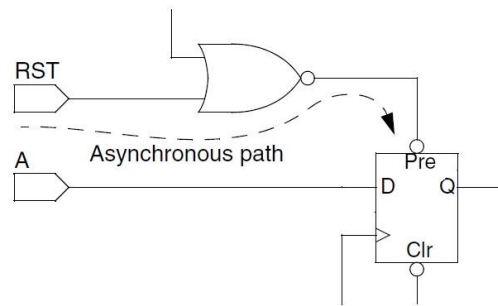


- Clock path may be passed through a “gated element” to achieve additional advantages. In this case, characteristics and definitions of the clock change accordingly. We called this type of clock path as “gated clock path.”

208

## STATIC TIME ANALYSIS (STA)

### Asynchronous path



- A path from an input port to an asynchronous set or clear pin of a sequential element.
- Functionality of set/reset pin is independent from the clock edge. Its level triggered pins and can start functioning at any time of data. So in other way, we can say that this path is not in synchronous with the rest of the circuit and that's the reason we are saying such type of path an Asynchronous path.

209

## STATIC TIME ANALYSIS (STA)

### Setup time

- Setup time is the minimum amount of time the data signal should be held steady **before** the clock event so that the data are reliably sampled by the clock. This applies to synchronous circuits such as the flip-flop.
- Or, in short, the amount of time the Synchronous input (D) must be stable **before the active edge of the Clock**
- The time when input data is available and stable **before** the clock pulse is applied is called Setup Time.

210

## STATIC TIME ANALYSIS (STA)

### Hold time

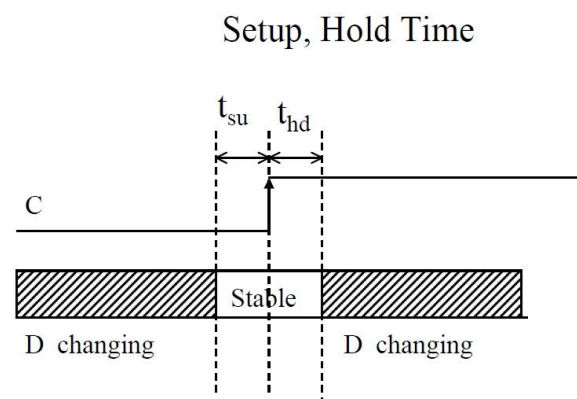
- Hold time is the minimum amount of time the data signal should be held steady **after** the clock event so that the data are reliably sampled. This applies to synchronous circuits such as the flip-flop.
- Or, in short, the amount of time the Synchronous input (D) must be stable **after** the active edge of the Clock
- The time **after** clock pulse where data input is held stable is called Hold Time.

211

## STATIC TIME ANALYSIS (STA)

### Setup and Hold Violation

- If Setup time is  $t_{su}$  for a FF and if data is not stable before  $t_{su}$  time from active edge of the clock, there is a Setup violation at that FF. So, if data is changing in the non-shaded area (in the above figure) before active clock edge, then it's a Setup violation.
- If Hold time is  $t_{hd}$  for a FF and if data is not stable after  $t_{hd}$  time from active edge of the clock, there is a Hold violation at that FF. So, if data is changing in the non-shaded area (in the above figure) after active clock edge, then it's a Hold violation.

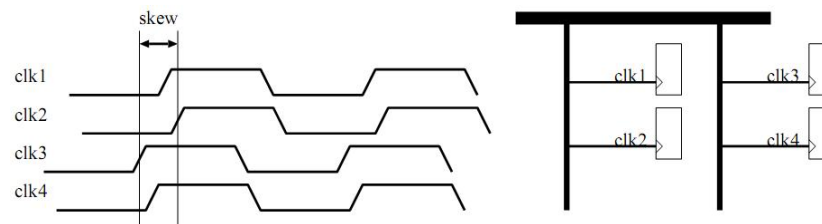
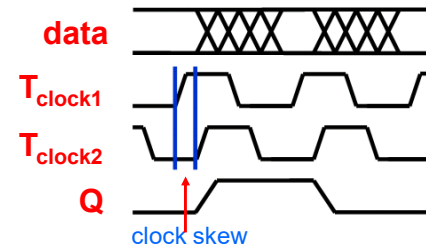


212

## STATIC TIME ANALYSIS (STA)

### Clock Skew

- Clock skew is a phenomenon in synchronous digital circuit systems in which the same sourced clock signal arrives at different components at different times. The instantaneous difference between the readings of any two clocks is called their skew.

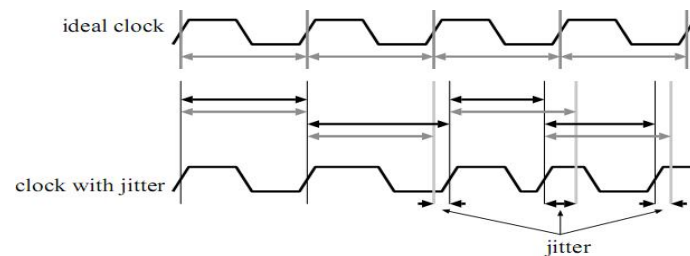


213

## STATIC TIME ANALYSIS (STA)

### Clock Jitter

- The difference between the actual clock and the desired clock (ideal)



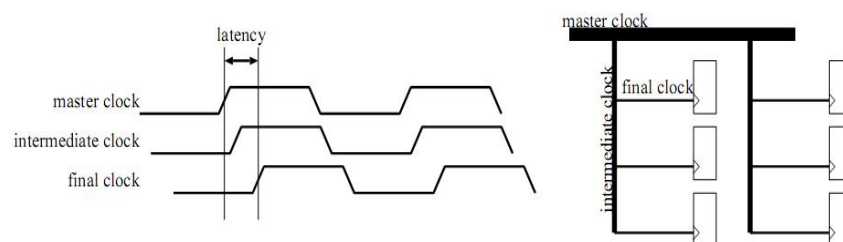
- Causes of Clock jitter:
  - Temperature and voltage (environment and on-chip) change over time
  - Fab house fabricates chip in different environments
  - etc.

214

## STATIC TIME ANALYSIS (STA)

### Clock Latency

- Clock Latency is the general term for **the delay that the clock signal takes between any two points**
- Clock Latency is the total delay that a clock signal takes to reach a sink or a destination pin, which typically is the clock pin of the flip-flops or the latches, from a clock source.

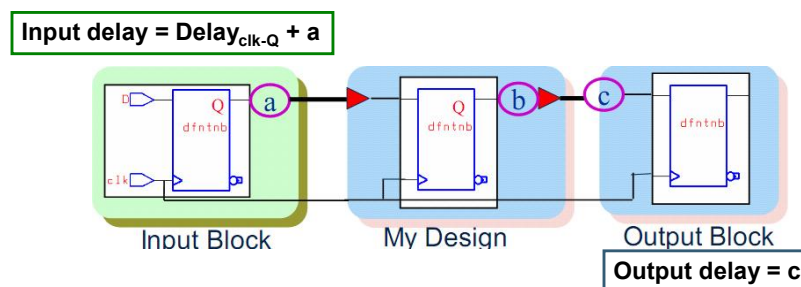


215

## STATIC TIME ANALYSIS (STA)

### Input and output delay

- Input delay is delay timing of inputs (relating to input data latch with clock edge)
- Similarly, Output delay is delay timing of outputs

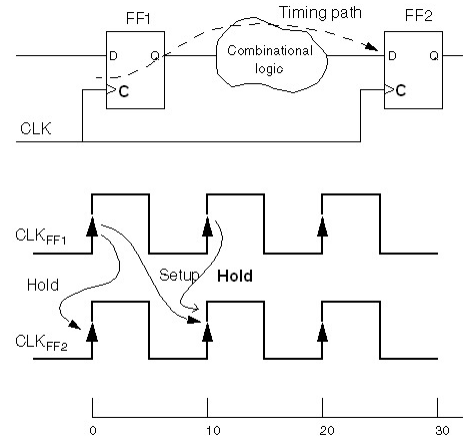


216

## STATIC TIME ANALYSIS (STA)

### Calculate Setup and Hold Violation for a design

- Data is launching from FF1/D to output FF1/Q at the positive clock edge at FF1/C
- At FF2/D, input data is coming from FF1/Q via a combinational logic
- Data is captured at FF2/D at the positive clock edge at FF2/C
- Data path is FF1/C → FF1/Q → FF2/D
- Signal propagates through data path in 1 clock cycle, so if data is launched at time = 0ns from FF1, it should be captured at time = 10ns by FF2.

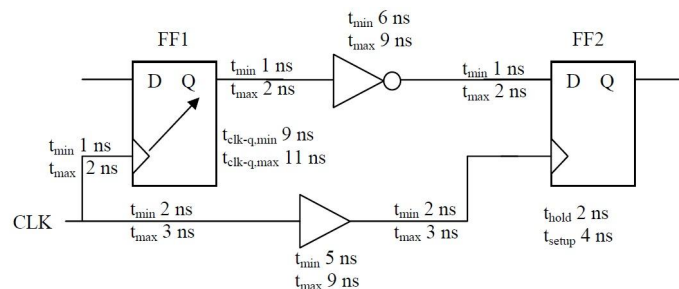


217

## STATIC TIME ANALYSIS (STA)

### Problem 1

- Find out this is Setup or Hold Violation ?
- When a **hold analysis** is performed, consider
  - Minimum delay along the data path
  - Maximum delay along the clock path
- If the difference between the data path and the clock path is negative, timing violation has occurred
- Solution: Data path is: CLK → FF1/C → FF1/Q → NOT → FF2/D
- Min. delay in data path = min(wire delay to the clock)+min(Clk-to-Q delay FF1)+min(NOT delay)+min(wire delay FF1/Q to NOT + wire delay NOT to FF2/D) =  $T_d = 1+9+6+(1+1)=18\text{ns}$
- Clock path: CLK → buffer → FF2/CLK
- Max. Clock path delay = max(wire delay from CLK to Buffer input)+max(Buffer delay)+max(wire delay from Buffer output to FF2/CLK)+(hold time of FF2) =  $T_{clk} = 3+9+3+2=17\text{ns}$
- Hold slack =  $T_d - T_{clk} = 18\text{ns} - 17\text{ns} = 1\text{ns} \rightarrow$  No Hold Violation (Because Hold slack > 0) → If hold time had been increased, there would be a hold violation
- If the hold time of FF2 is 4ns,  $T_{clk} = 3+9+3+4=19\text{ns} \rightarrow$  Hold slack =  $T_d - T_{clk} = 18\text{ns} - 19\text{ns} = -1\text{ns} \rightarrow$  Hold Violation

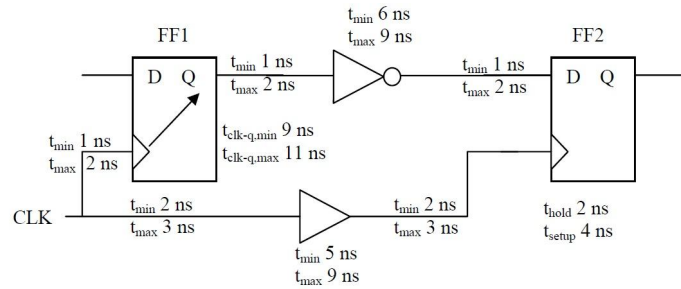


218

## STATIC TIME ANALYSIS (STA)

### Problem 1

- Find out this is Setup or Hold Violation ?
- When a **setup analysis** is performed, consider
  - Maximum delay along the data path
  - Minimum delay along the clock path
- If the difference between the data path and the clock path is negative, timing violation has occurred
- Solution: Data path is: CLK → FF1/C → FF1/Q → NOT → FF2/D
- Max. delay in data path = max(wire delay to the clock)+max(Clk-to-Q delay FF1)+max(NOT delay)+max(wire delay FF1/Q to NOT + wire delay NOT to FF2/D) =  $T_d = 2+11+9+(2+2)=26\text{ns}$
- Clock path: CLK → buffer → FF2/CLK
- Min. Clock path delay = **Clock period** + min(wire delay from CLK to Buffer input)+min(Buffer delay)+min(wire delay from Buffer output to FF2/CLK)-(setup time of FF2) =  $T_{clk} = 15+2+5+2-4=20\text{ns}$  (**Setup is checked at the next clock cycle**)
- Setup slack =  $T_{clk} - T_d = 20\text{ns} - 26\text{ns} = -6\text{ns} \rightarrow$  Setup Violation (Setup slack < 0) → Fixing by increase clock period or less max. delay of inverter
- If Clock period is 22ns,  $T_{clk} = 27\text{ns}$ ,  $T_d = 26\text{ns} \rightarrow$  Setup slack =  $T_{clk} - T_d = 27\text{ns} - 26\text{ns} = 1\text{ns} \rightarrow$  No Setup Violation

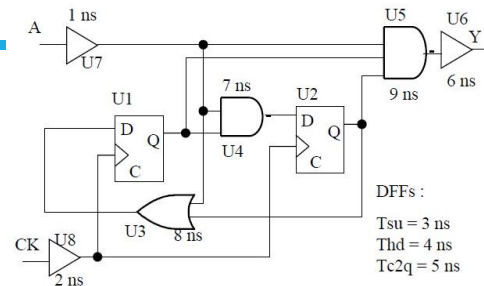


219

## STATIC TIME ANALYSIS (STA)

### Problem 2

- Find the maximum operating frequency for this circuit.
- (Note: Ignore wire delay), where  $T_{su}$  = setup time,  $T_{hd}$  = Hold time,  $T_{cq}$  = Clock-to-Q delay
- Step 1: Find the max. Register to Register delay
- Max Register to Register Delay = (Clk-to-Q delay of U2) + (Cell delay of U3) + (all wire delay) + (setup time of U1) =  $5+8+3=16\text{ns}$
- There are 2 register to register paths
  - U2 → U3 → U1 (delay =  $5+8+3=16\text{ns}$ )
  - U1 → U4 → U2 (delay =  $5+7+3=15\text{ns}$ )
  - We must pick max. value.



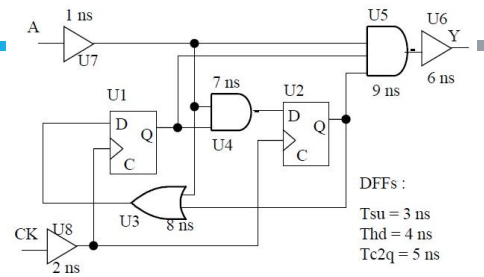
220

## STATIC TIME ANALYSIS (STA)

### Problem 2

#### Step 2: Find Setup time

- Setup time = Setup time of FF + Max(Data path delay) - min(Clock path delay) = (Setup time of FF + A2D max delay) - min(Clock path delay) =  $T_{su} + (T_{pd} U7 + T_{pd} U3 + \text{wire delay}) = 3 + (1 + 8) - 2 = 10\text{ns}$
- Here we are not using the Clock period. Because we are not supposed to calculate the Setup violation. We are calculating Setup time.
- All the wire delay is ignored. If wire delay is considered, we have to consider those one.
- There are 2 data paths
  - $A \rightarrow U7 \rightarrow U4 \rightarrow D \text{ of } U2$  (Data path delay =  $1 + 7 = 8\text{ns}$ )
  - $A \rightarrow U7 \rightarrow U3 \rightarrow D \text{ of } U1$  (Data path delay =  $1 + 8 = 9\text{ns}$ )
- For setup calculation we need max. data path delay, we must choose the 2<sup>nd</sup> value.



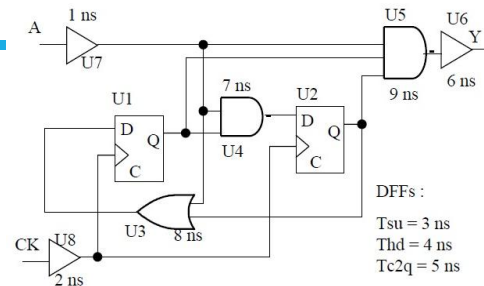
221

## STATIC TIME ANALYSIS (STA)

### Problem 2

#### Step 3: Find Hold time

- Hold time = Hold time of FF + Max(Clock path delay) - min(Data path delay) = (Hold time of FF + Clk path max delay) - (A2D max delay) =  $T_{hd} + T_{pd} U8 - (T_{pd} U7 + T_{pd} U4 + \text{wire delay}) = 4 + 2 - (1 + 7) = -2\text{ns}$
- All the wire delay is ignored. If wire delay is considered, we have to consider those one.
- There are 2 data paths
  - $A \rightarrow U7 \rightarrow U4 \rightarrow D \text{ of } U2$  (Data path delay =  $1 + 7 = 8\text{ns}$ )  $\rightarrow$  Hold time =  $4 + 2 - (1 + 7) = -2\text{ns}$
  - $A \rightarrow U7 \rightarrow U3 \rightarrow D \text{ of } U1$  (Data path delay =  $1 + 8 = 9\text{ns}$ )  $\rightarrow$  Hold time =  $4 + 2 - (1 + 8) = -3\text{ns}$
- For hold calculation we need min. data path delay, we must choose the 1<sup>st</sup> value.

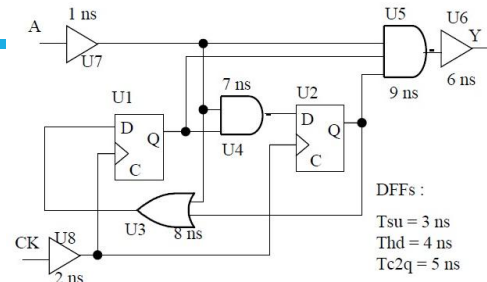


222

## STATIC TIME ANALYSIS (STA)

### Problem 2

- Step 4: Find Clock to out time
- Clock to Out = Cell delay of U8 + Clk-to-Q delay of FF + Cell delay of U5 + Cell delay of U6 + (all wire delay) =  $T_{pd} U8 + T_{c2q} U2 + T_{pd} U5 + T_{pd} U6 = 2 + 5 + 9 + 6 = 22\text{ns}$
- There are 2 Clock to out paths: From FF U1 and FF U2
- In this case, the Clk-to-Q path for both FF is the same, we can consider any paths, but for different delay path we should consider max. delay path.
- Step 5: Find Pin to Pin Combinational Delay (A to Y delay)
- Pin to pin Combinational Delay (A to Y Delay) =  $T_{pd} U7 + T_{pd} U5 + T_{pd} U6 = 1 + 9 + 6 = 16\text{ns}$



223

## STATIC TIME ANALYSIS (STA)

### Problem 2

- Step 6: Find Max Clock Frequency
- Max Clock Frequency =  $1/\text{Max}(\text{Reg2reg}, \text{Clk2out}, \text{Pin2pin}) = 1/\text{Max}(16, 22, 16) = 1/22\text{ns} = 45.5\text{MHz}$
- **Summary table:**

| Parameter | Description     | Min. | Max. | Units |
|-----------|-----------------|------|------|-------|
| Tclk      | Clock Period    | 22   |      | ns    |
| Fclk      | Clock Frequency |      | 45.5 | MHz   |
| Atsu      | A setup time    | 10   |      | ns    |
| Athd      | A hold time     | -2   |      | ns    |
| A2Y       | A to Y Tpd      |      | 16   | ns    |
| Clk2Y     | Clock to Y Tpd  |      | 22   | ns    |

224

## STATIC TIME ANALYSIS (STA)

### Points to remember

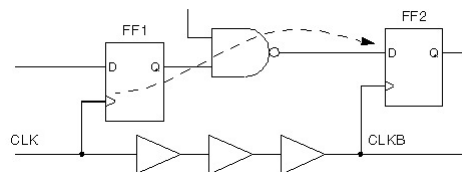
- Setup is checked at next clock edge
- Hold is checked at same clock edge
- For Hold Check (Checking of Hold Violation)
  - **Minimum Delay** along the **data path**
  - **Maximum Delay** along the **clock path**
- For Setup Check (Checking of Setup Violation)
  - **Maximum Delay** along the **data path**
  - **Minimum Delay** along the **clock path**

225

## STATIC TIME ANALYSIS (STA)

**Calculation of Setup Violation Check:** Consider the circuit of 2 FF connected to each other

- **Setup Slack = Required time – Arrival time (since we want data to arrive before it is required)**
- Where
  - Arrival time (max) = clock delay FF1 (max) + Clock-to-Q delay FF1 (max) + comb. delay (max)
  - Required time = Clock adjust + Clock delay FF2 (min) – setup time FF2
  - Clock adjust = Clock period (since setup is analyzed at next edge)

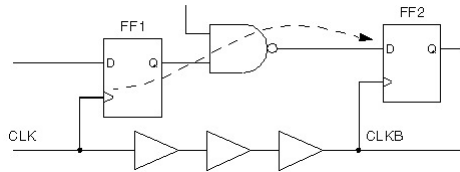


226

## STATIC TIME ANALYSIS (STA)

**Calculation of Hold Violation Check:** Consider the circuit of 2 FF connected to each other

- **Hold Slack = Arrival time – Required time (since we want data to arrive after it is required)**
- Where
  - Arrival time (min) = clock delay FF1 (min) + Clock-to-Q delay FF1 (min) + comb. delay (min)
  - Required time = Clock adjust + Clock delay FF2 (max) + hold time FF2
  - Clock adjust = 0 (since hold is analyzed at same edge)

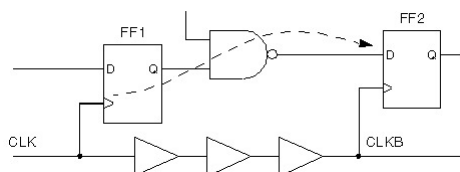


227

## STATIC TIME ANALYSIS (STA)

**Calculation of Max. Clock Frequency:** Consider the circuit of 2 FF connected to each other

- **Max. Clock Freq = 1/Max(Reg2Reg delay, Clk2Out delay, Pin2Pin delay)**
- Where
  - Reg2Reg delay = Clk-to-Q delay of FF1 (max) + comb. delay (max) + setup time of FF2
  - Clk2Out delay = Clock delay with reference to FF (max) + Clk-to-Q delay of FF1 (max) + comb. delay (max)
  - Pin2Pin delay = Comb. delay between input pin to output pin (max)



228

---

## STATIC TIME ANALYSIS (STA)

### Fixing Setup and Hold Violation Summary

- Min. delay of Combinational Path > Hold time of Capture FF
- Max. delay of Combinational Path < Clock period - Setup time of Capture FF
- Clock period  $\geq$  Setup time + Hold time

229

---

## STATIC TIME ANALYSIS (STA)

### Fixing Setup Time Violations

- Clear buffers on path
- Replace buffers by 2 NOT gates which are placed far apart
- Decrease capacitors on path
- Ensure that “clock uncertainty” is not too large compared with the current library
- Increase clock cycle or decrease operating frequency

### Fixing Hold Time Violations

- Add buffer/2 inverters
- Select cell with the larger delay value
- Reduce the size of certain blocks on the path, especially those near the flipflop for data latch
- Add a capacitor to the output

230

## STATIC TIME ANALYSIS (STA)

### PVT

- Process: transistors with shorter gate lengths have faster switching time (ex.: 0.5 $\mu$ m technology vs. 0.35  $\mu$ m technology).
- Voltage: if voltage is higher, switching time of gate is getting faster.
- Temperature: the lower temperature, the higher mobility, so switching time of gate is getting faster.
- In a specific process, there are
  - Fast (NMOS, PMOS)
  - Slow (NMOS, PMOS)
  - Typical (NMOS, PMOS)

231

## STATIC TIME ANALYSIS (STA)

### PVT

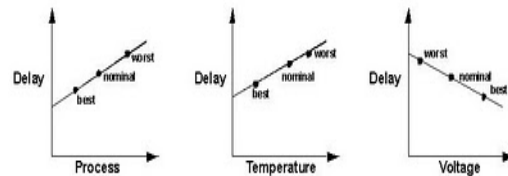
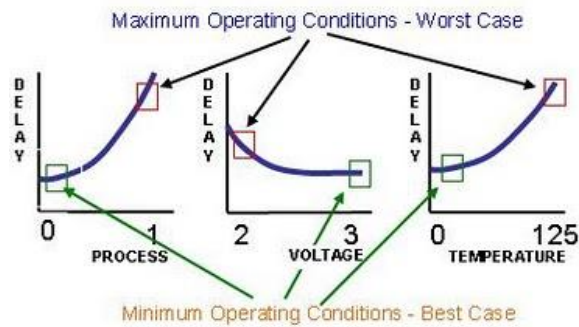
- Library is included by the WORST, TYPICAL and BEST operating conditions, and defined in technology library.
- **Best case:** Fast NMOS, Fast PMOS, high voltage, low temperature.
- **Worst case:** Slow NMOS, Slow PMOS, low voltage, high temperature.
- **Typical/Nominal case:** Fast NMOS, Slow PMOS or Slow NMOS, Fast PMOS, average voltage, average temperature.
- $V_{DD}$  can be low (Slow devices), Typical, or high (Fast devices)
- Temperature can be cold (Fast devices), Typical, or hot (Slow devices)

232

## STATIC TIME ANALYSIS (STA)

### PVT

- PVT affects to timing and delay.

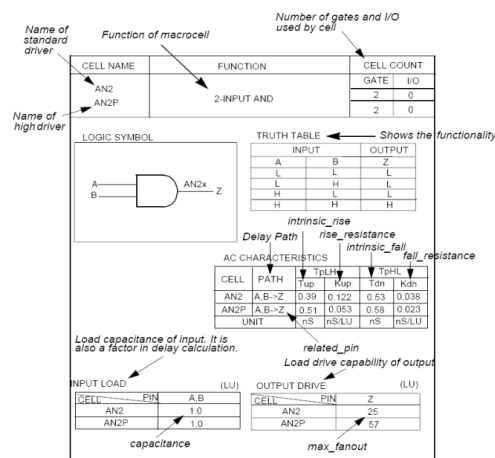


233

## STANDARD CELL LIBRARY

### Technology Library

- Correlation between specification (datasheet) and .lib file of 2-input AND gate



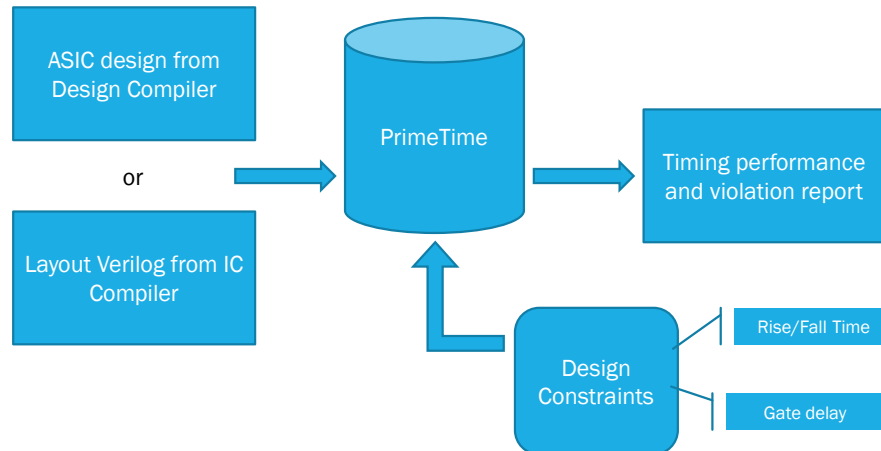
```
pin(A,B) {
    direction : input ;
    capacitance : 1 ;
}

pin(Z) {
    direction : output ;
    function : "A * B" ;
    timing() {
        intrinsic_rise : 0.39 ;
        intrinsic_fall : 0.53 ;
        rise_resistance : 0.122 ;
        fall_resistance : 0.038 ;
        related_pin : "A B" ;
    }
}
```

234

## STATIC TIME ANALYSIS (STA)

### STA Implementation with PrimeTime (Synopsys)



235

## STATIC TIME ANALYSIS (STA)

### STA Implementation with PrimeTime (Synopsys)

```
#####
set_driving_cell -lib_cell NBUFFX16 -pin Z [get_ports clock]
set_driving_cell -lib_cell SDDFARX1 -pin Q [get_ports reset]
set_load -pin_load 0.004 [get_ports {counter_out[3]}]
set_load -pin_load 0.004 [get_ports {counter_out[2]}]
set_load -pin_load 0.004 [get_ports {counter_out[1]}]
set_load -pin_load 0.004 [get_ports {counter_out[0]}]

create_clock [get_ports clock] -period 20 -waveform {0 5}
set_clock_uncertainty 0.4 [get_clocks clock]
set_propagated_clock [get_clocks clock]
set_clock_transition -rise 0.8 [get_clocks clock]
set_clock_transition -fall 0.8 [get_clocks clock]
set_input_delay -clock clock 0.85 [get_ports reset]

set_output_delay -clock clock 8 [get_ports {counter_out[3]}]
set_output_delay -clock clock 8 [get_ports {counter_out[2]}]
set_output_delay -clock clock 8 [get_ports {counter_out[1]}]
set_output_delay -clock clock 8 [get_ports {counter_out[0]}]
```

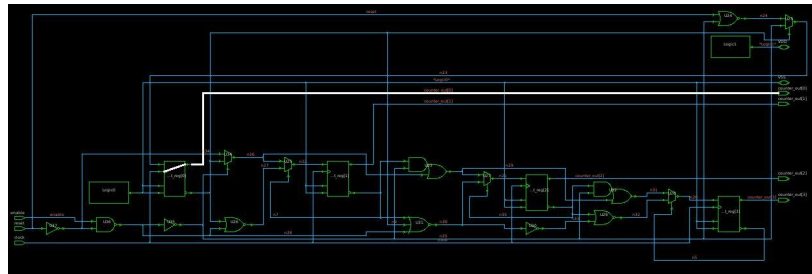
```
Startpoint: counter_out_reg[1]
(rising edge-triggered flip-flop clocked by clock)
Endpoint: counter_out_reg[1]
(rising edge-triggered flip-flop clocked by clock)
Path Group: clock
Path Type: max
```

| Point                            | Incr    | Path      |
|----------------------------------|---------|-----------|
| clock clock (rise edge)          | 0.0000  | 0.0000    |
| clock network delay (propagated) | 0.0348  | 0.0348    |
| counter_out_reg[1]/CLK (SDFFX1)  | 0.0000  | 0.0348 r  |
| counter_out_reg[1]/QN (SDFFX1)   | 0.2241  | 0.2589 f  |
| U25/Q (MUX2I1X1)                 | 0.2122  | 0.4711 r  |
| counter_out_reg[1]/D (SDFFX1)    | 0.0326  | 0.5037 r  |
| data arrival time                |         | 0.5037    |
| clock clock (rise edge)          | 20.0000 | 20.0000   |
| clock network delay (propagated) | 0.0348  | 20.0348   |
| clock uncertainty                | -0.4000 | 19.6348   |
| counter_out_reg[1]/CLK (SDFFX1)  |         | 19.6348 r |
| library setup time               | -0.1875 | 19.4473   |
| data required time               |         | 19.4473   |
| data required time               |         | 19.4473   |
| data arrival time                |         | -0.5037   |
| slack (MET)                      |         | 18.9436   |

236

## STATIC TIME ANALYSIS (STA)

### STA Implementation with PrimeTime (Synopsys)

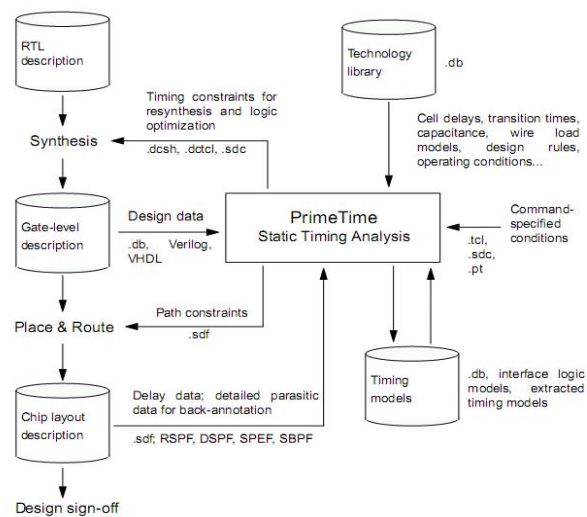


## DataTableView.12 - Path Table

| Startpoint Pin Name  | Endpoint Pin Name | Path Group | Slack    | Endpoint Cloc | Arrival | Required | Startpoint | Startpoint Cloc | Endpoint | Endpoint Cloc |
|----------------------|-------------------|------------|----------|---------------|---------|----------|------------|-----------------|----------|---------------|
| counter_out_reg[...] | counter_out[0]    | clock      | 11.27541 | -0.03483      | 0.32459 | 11.60000 | clock      | 0.03483         | clock    | 0.00000       |

237

## STATIC TIME ANALYSIS (STA)



238

## STATIC TIME ANALYSIS (STA)

### INPUTS

- Design file
  - Synopsys database files (.db)
  - Verilog netlist files
  - Electronic Data Interchange Format (EDIF) netlist files
  - VHDL/Verilog netlist files
- Set up the operating conditions, wireload models, port load, drive, and transition time.
- Define the clock period, waveform, uncertainty, and latency.
- Specify the input and output port delays.
- Set multicycle paths.
- Set false paths.
- Specify minimum and maximum delays, path segmentation, and disabled arcs.



### OUTPUTS

#### Flip-flops with violations

- Setup and hold violations
- Timing paths
- Rising edge input timing and falling edge input timing of all paths in the design
- Critical path for each clock group
- Timing report for each clock group
- Electrical design rule violations

239

**THE END  
(THIS PART)**

240